SIEMENS EDA

Tessent™ Multi-die User's Manual

Software Version 2024.4
Document Revision 9

SIEMENS

About Siemens Digital Industries Software

Siemens Digital Industries Software is a global leader in the growing field of product lifecycle management (PLM), manufacturing operations management (MOM), and electronic design automation (EDA) software, hardware, and services. Siemens works with more than 100,000 customers, leading the digitalization of their planning and manufacturing processes. At Siemens Digital Industries Software, we blur the boundaries between industry domains by integrating the virtual and physical, hardware and software, design and manufacturing worlds. With the rapid pace of innovation, digitalization is no longer tomorrow’s idea. We take what the future promises tomorrow and make it real for our customers today. Where today meets tomorrow. Our culture encourages creativity, welcomes fresh thinking and focuses on growth, so our people, our business, and our customers can achieve their full potential.

Support Center: support.sw.siemens.com
Send Feedback on Documentation: support.sw.siemens.com/doc_feedback_form

Revision History ISO-26262

Revision	Changes	Status/Date
9	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the *Tessent Release Notes* for this product are reflected in this document. Approved by Ron Press.	Released Dec 2024
8	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the *Tessent Release Notes* for this product are reflected in this document. Approved by Ron Press.	Released Aug 2024
7	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the *Tessent Release Notes* for this product are reflected in this document. Approved by Ron Press.	Released Jun 2024
6	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the *Tessent Release Notes* for this product are reflected in this document. Approved by Ron Press.	Released Mar 2024

Author: In-house procedures and working practices require multiple authors for documents. All associated authors for each topic within this document are tracked within the Siemens documentation source. For specific topic authors, contact the Siemens Digital Industries Software documentation department.

Revision History: Released documents include a revision history of up to four revisions. For earlier revision history, refer to earlier releases of documentation on Support Center.

Table of Contents

List of Figures

List of Tables

Chapter 1
Multi-Die DFT Introduction

System on a Chip (SoC) technology enabled the expansion from simple devices comprised entirely of I/O, logic, and memories to include third-party IP cores such as serializer/deserializer (SerDes), double data rate (DDR) memory, physical layer architecture (PHY), and special I/Os such as low-voltage differential signaling (LVDS), and positive emitter-coupled logic (PECL). This created a new IP core marketplace. These IP cores necessarily came with design-for-test (DFT) libraries, embedded DFT logic such as scan, or both.

Through creative packaging techniques, SoC has evolved into system in a package (SiP) where you can instantiate multiple dies inside a single chip package. This has created a new *Chiplet* market where IP vendors sell chiplets with specific functions to third-party SiP aggregators who build the SiP with chiplets and their own chips.

In a SiP, each die connects to other dies using either 2.5D or 3D interconnect technology. SiPs have the following advantages over multi-chip modules (MCMs) and printed circuit boards (PCBs):

- Better performance
- Smaller size
- More functional capability
- Cost savings

SiP advantages over a very large SoC design with the same logic include:

- Better silicon yield as a result of breaking large SoC functions up into smaller dies
- The capability to manufacture packaged ICs much larger than the maximum die size for the target process
- Better profitability as the packaging of bad parts is avoided

SiP challenges compared to a very large SoC include:

- Higher power density, IR drop, and noise
- Thermal load and cooling issues
- Lower package yield due to increased packaging steps and complexity
- Dies must be fully tested at wafer probe before packaging
- Wafer probe is more difficult for SiP chiplets and SoCs than for standard ICs

DFT in 3D Designs

Assumptions

There are certain assumptions and prerequisites for using the Tessent Multi-die flow.

- You must have a working knowledge of Tessent DFT insertion, vector generation, and verification, including IJTAG. Refer to Tessent documentation and training courses.
- All chips inside the SiP must have DFT and IP test implementations that use the standard Tessent tools and flows.
- The SSN bus transports scan data between the primary die and the chiplets.
- Each chip that supports hard repair must have its own eFuse or One-Time Programmable (OTP) non-volatile memory to store the repair signature. Dies cannot share eFuse or OTP.
- You must individually test each die using Known Good Die (KGD) wafer probe techniques.
- You should have a good understanding of basic IP tests such as SerDes loopback.
- You must have a basic understanding of wafer probe and sort, and final package test.
- You must have an understanding of RTL and SDC timing constraints.
- IEEE-1149.6-compliant AC boundary scan is supported only for boundary scan at the SiP package pins. The unified package-level BSDL flow supports AC boundary scan.

Restriction:
The Tessent Multi-die flow does not support boundary scan for AC-coupled interfaces between dies.

DFT in 2.5D Designs

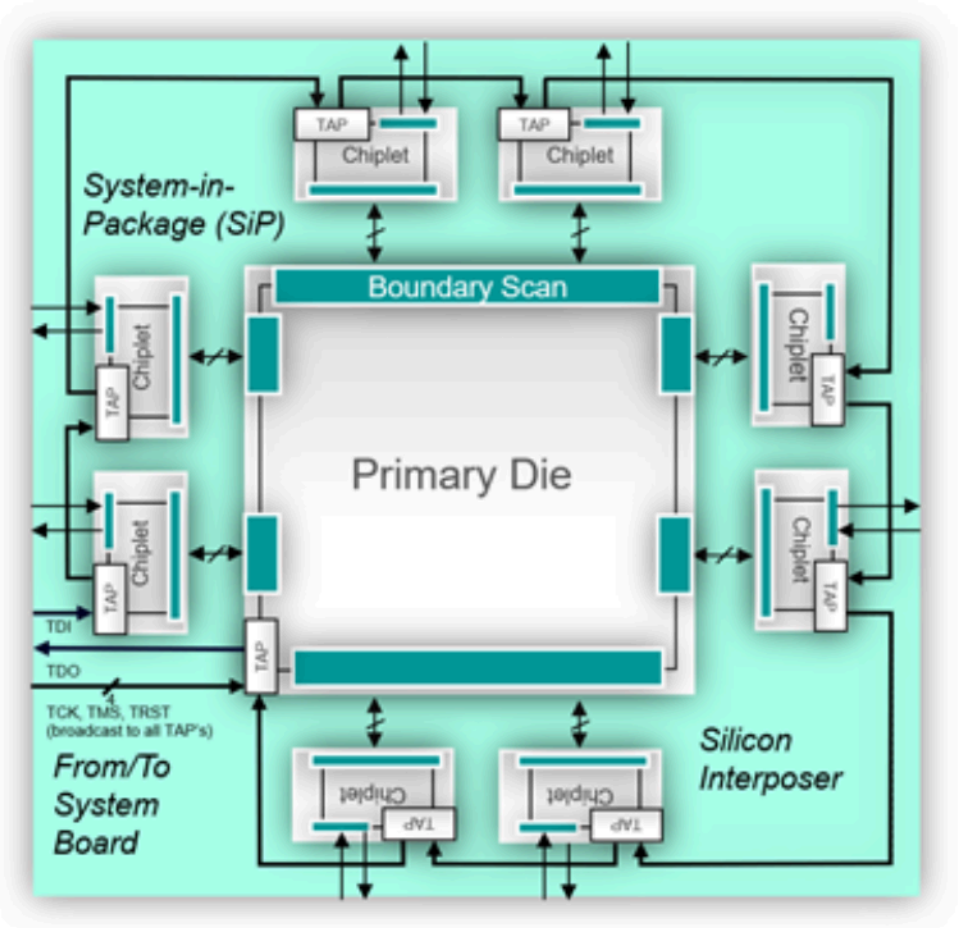
A 2.5D design includes one or more logic dies connected to a base SOC die on the same plane in a chip.

2.5D Design Overview

Before the advent of System in a Package (SiP), the primary means for interconnecting multiple chips was using printed circuit boards (PCBs), which are considered 2D technology. With 2.5D, we are moving the board, or subsystems of the board, into the IC package.

The following figure shows an example with a large primary die. The primary die provides the TDO signal, and the TDI connects to a nearby chiplet. The chiplets are considered to be secondary devices.

Figure 1. Example 2.5D Interconnect With Boundary Scan



You can also incorporate new technologies such as High Bandwidth Memory (HBM). Technologies such as HBM3 have their own DFT and IP test requirements. Some of these test requirements are built into the HBM3 3D memory stack, like DRAM memory BIST. Others, such as I/O BIST and lane repair, are in the

base die of the memory stack. IP cores such as HBM3 PHYS are incorporated in the ASICs or SoCs and mirror the test behavior of the HBM3 3D stack base die. These technologies may require thousands of I/Os between dies in the package. A much finer bump pitch than the standard bump pitch is required to accommodate large numbers of lanes.

Standard substrates cannot accommodate such fine bump pitches, so silicon is typically used as the substrate. The silicon substrate (or “interposer”) is manufactured in an IC fab. The PCB is moved to the interposer and into the package. You can instantiate many chips on the interposer; thus, the term “System-in-Package”. With this methodology, the circuitry of a conventional large PCB can fit into a relatively small package.

Multi-Chip Modules (MCMs) are a variation of SiPs with multiple dies instantiated on a PCB or ceramic substrate. Most of the 2.5D capabilities described also apply to MCMs, and this documentation highlights any differences where they exist.

This document describes use of the standard Tessent DFT insertion flows for the individual dies. Boundary scan insertion capabilities and flows have been enhanced for die-to-die interconnect testing and for *Unified BSDL*.

2.5D DFT Overview

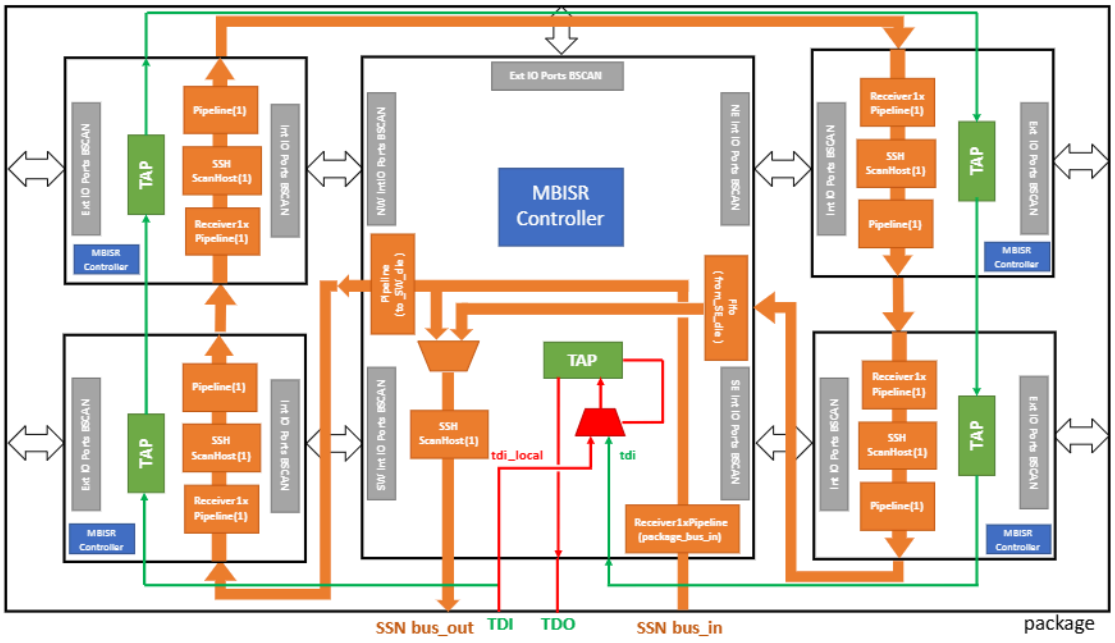
This section describes the requirements for the Tessent Shell multi-die DFT flow.

Each die in the package has its own DFT logic to provide testability for the die itself and package/ interposer connections. The package level permits only wire connections. The wires connect the dies to each other and to the package pins.

The functional clock tree is localized within each die. This enables independent testing of the dies, retargeting of die internal modes, and creation of consolidated patterns to detect faults in die-to-die connections. Clocks may also come from other dies.

The following figure shows a block diagram of an example 2.5D design with DFT inserted.

Figure 2. 2.5D Design With DFT Inserted



Each die in the 2.5D integrated package has a TAP interface and a boundary scan (BScan) chain to provide interconnect testability. You can use the Tessent Streaming Scan Network (SSN) as logic test infrastructure to integrate the logic test of all dies in the package. You can use a daisy-chained SSN bus to access the scan and EDT logic in all dies.

The die with a TAP connected directly to the TDO port at the package level is the primary die (the center die in Figure 2). The other dies are considered non-primary when assembled in the package.

The above example shows that the SSN bus starts at the package I/O, goes to the inputs of the primary die, through other dies in the daisy-chain topology, back to the primary die, and finally to the package SSN bus output ports. The SSN paths in Figure 2 are designed such that the SSN interface that connects to the ATE comes from the primary die.

Note:
This configuration of the SSN bus is not mandatory. For example, you can place the SSN input and output interfaces in separate dies.

IEEE 1149.1 standard compliance is required to extract the unified package-level BSDL description and create JTAG boundary scan patterns. A flow for a package that is not compliant with the IEEE 1149.1 standard is limited to generating the MultiDieInterconnect patterns, which use the BSDL descriptions of the individual dies to test the interconnect between dies in the SiP. In this case, the board test uses the individual BSDLs, and the SiP appears to be an auxiliary board with multiple devices during system or board testing.

CAUTION:
An integrated package with daisy-chained TAPs is not compliant with the IEEE 1149.1 standard for board test with a single package-level BSDL. To be compliant with a single package-level BSDL, your design must meet the following requirements at the package TAP pins:

- There must be a 1-bit BYPASS instruction.
- There must be a 32-bit DEVICE_ID instruction.
- The TAP instruction length must be constant.

When you connect the TAP controllers inside the dies in series, the above requirements are not fulfilled because the DEVICE_ID and BYPASS registers are concatenated in series. For the package-level BSDL to comply with the IEEE 1149.1 standard, add the tdi_local mux and port to the primary die.

The TDI port directly connects to the primary die through the additional tdi_local port. By setting the attribute for the TDI local port, the tool can automatically generate the necessary muxing logic to bypass the non-primary dies during the BYPASS and DEVICE_ID instructions. For example, the IDCODE for the primary die is the only IDCODE that can be read from the package during the DEVICE_ID instruction when the TDI_local mux is active. Figure 2 shows how the tdi_local mux reroutes TDI so that the primary die's TAP is used for the DEVICE_ID and BYPASS instructions.

The constant TAP instruction length requirement means that all TAP instruction registers must be in series to configure the SiP boundary scan registers.

Multi-Die Topologies for 2.5D

The Tessent Multi-die tool supports the following 2.5D topologies for interconnect patterns: daisy-chained, multiple chain scan interface, and separate scan interfaces.

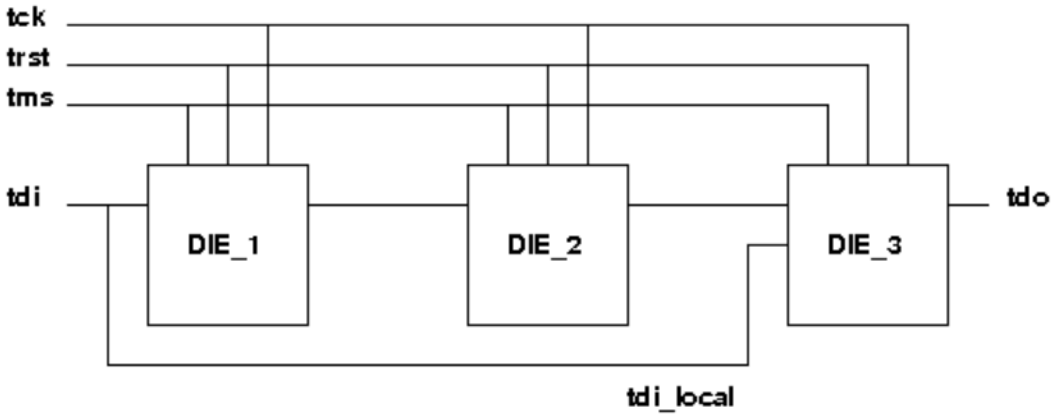
Tip
For TDI local and TDO daisy topologies, we recommend that the TAP controller provide the BYPASS_MD instruction and that you register the IDCODE_MD instruction if you have a IDCODE instruction. When Tessent generates the TAP controller and the TDI or TDO mux, it adds those instructions automatically.

The BYPASS_MD and IDCODE_MD are identical to the BYPASS and IDCODE instructions except that they do not activate the TDI local or TDO daisy mux. This enables access to the data register of other dies in the package through the BYPASS and DEVICE_ID register of the die that provides the *_MD instructions. For more information, refer to DftSpecification/IjtagNetwork/HostScanInterface/Tap/bypass_instruction_codes_md and DftSpecification/IjtagNetwork/HostScanInterface/Tap/DeviceIDRegister/instruction_codes_md.

Daisy-Chained Topology

Figure 3 and Figure 4 show daisy-chained topologies. These topologies result in an IEEE 1149.1-compliant package and a relatively small number of package ports are needed.

Figure 3. Daisy-Chained Topology With tdi_local

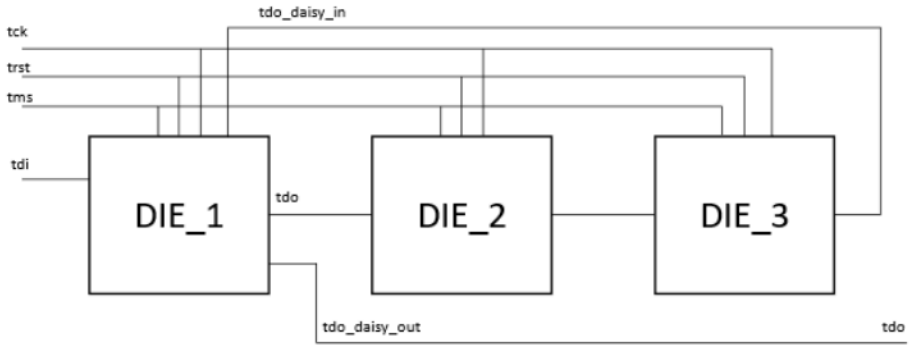


Using an ICL description is optional. If you require an ICL description, the following description of the daisy-chained topology with tdi_local is the ICL representation for the package:

```
Module package {
   TCKPort tck;
   ScanInPort tdi;
   ScanOutPort tdo {
      Source Die_3.tdo;
   }
   TMSPort tms;
   TRSTPort trst;
   ScanInterface TC0 {
      Port tdi;
      Port tdo;
      Port tms;
      Port trst;
      Port tck;
   }
```

```
    Instance Die_1 Of die1 {
       InputPort tck  = tck;
       InputPort tdi  = tdi;
       InputPort tms  = tms;
       InputPort trst = trst;
    }
    Instance Die_2 Of die1 {
       InputPort tc k = tck;
       InputPort tdi  = Die_1.tdo;
       InputPort tms  = tms;
       InputPort trst = trst;
    }
    Instance Die_3 Of die2 {
       InputPort tck       = tck;
       InputPort tdi       = Die_2.tdo;
       InputPort tdi_local = tdi;
       InputPort tms       = tms;
       InputPort trst      = trst;
    }
}
```

Figure 4. Daisy-Chained Topology With tdo_daisy



Similarly, you could optionally use an ICL description such as the following for the daisy-chained topology with tdo_daisy:

```
Module package {
   TCKPort tck;
   ScanInPort tdi;
   ScanOutPort tdo {
      Source Die_1.tdo_daisy_out;
   }
   TMSPort tms;
   TRSTPort trst;
   ScanInterface TC0 {
      Port tdi;
      Port tdo;
      Port tms;
      Port trst;
      Port tck;
   }
```

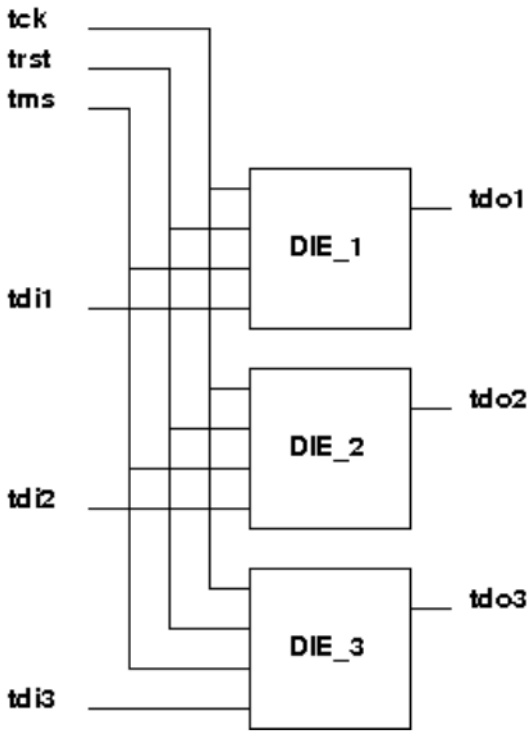
```
   Instance Die_1 Of die3 {
      InputPort tck          = tck;
      InputPort tdi          = tdi;
      InputPort tdo_daisy_in = Die_3.tdo;
      InputPort tms          = tms;
      InputPort trst         = trst;
   }
   Instance Die_2 Of die1 {
      InputPort tck  = tck;
      InputPort tdi  = Die_1.tdo;
      InputPort tms  = tms;
      InputPort trst = trst;
   }
   Instance Die_3 Of die1 {
      InputPort tck  = tck;
      InputPort tdi  = Die_2.tdo;
      InputPort tms  = tms;
      InputPort trst = trst;
   }
}
```

Multiple Scan Chain Interface

The following figure shows the multiple scan chain interface topology. With this topology, all dies can be operated in parallel, reducing the MultiDieInterconnect pattern shift time.

Note:
The package that is created using the multiple scan chain interface topology is not IEEE-1149.1 compliant.

Figure 5. Multiple Scan Chain Interface



For this topology, optionally use an ICL description such as the following:

```
Module package {
   TCKPort tck;
   ScanInPort tdi1;
   ScanInPort tdi2;
   ScanInPort tdi3;
   ScanOutPort tdo1 {
      Source Die_1.tdo;
   }
   ScanOutPort tdo2 {
      Source Die_2.tdo;
   }
   ScanOutPort tdo3 {
      Source Die_3.tdo;
   }
   TMSPort tms;
   TRSTPort trst;
   ScanInterface TC0 {
      Port tck;
      Port tms;
      Port trst;
      Chain chain0 {
         Port tdi1;
         Port tdo1;
      }
      Chain chain1 {
```

```
         Port tdi2;
         Port tdo2;
      }
      Chain chain2 {
         Port tdi3;
         Port tdo3;
      }
   }
   Instance Die_1 Of die1 {
      InputPort tck  = tck;
      InputPort tdi  = tdi1;
      InputPort tms  = tms;
      InputPort trst = trst;
   }
   Instance Die_2 Of die1 {
      InputPort tck  = tck;
      InputPort tdi  = tdi2;
      InputPort tms  = tms;
      InputPort trst = trst;
   }
   Instance Die_3 Of die1 {
      InputPort tck  = tck;
      InputPort tdi  = tdi3;
      InputPort tms  = tms;
      InputPort trst = trst;
   }
}
```

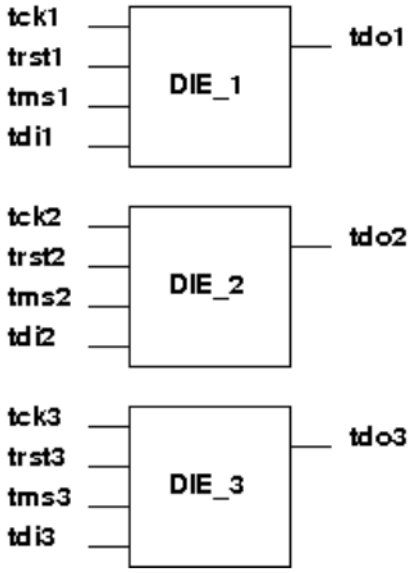
Separate Scan Chain Interfaces

Figure 6 shows a topology with separate scan chain interfaces. With this topology, all dies must be operated independently. Additionally, the MultiDieInterconnect pattern shift time is greater compared to the multiple scan chain interface solution because the chains are operated sequentially. To have the required values in the first-operated chains when the later chains capture, every value is loaded twice into each chain. This doubles the test time.

Note:
The package that is created using the separate scan chain interfaces topology is not IEEE-1149.1 compliant. Also, more package ports are required compared to other topologies.

Tip
Although this option increases the test time, you may want to consider using it for the simplicity of testing each die individually, which also enables you to identify a failing die. It may be easier to identify a failing die if the test does not interfere with the other dies.

Figure 6. Separate Scan Chain Interfaces



For this topology, optionally use an ICL description such as the following:

```
Module package {
   TCKPort tck1;
   TCKPort tck2;
   TCKPort tck3;
   ScanInPort tdi1;
   ScanInPort tdi2;
   ScanInPort tdi3;
   ScanOutPort tdo1 {
      Source Die_1.tdo;
   }
   ScanOutPort tdo2 {
      Source Die_2.tdo;
   }
   ScanOutPort tdo3 {
      Source Die_3.tdo;
   }
   TMSPort tms1;
   TMSPort tms2;
   TMSPort tms3;
   TRSTPort trst1;
   TRSTPort trst2;
   TRSTPort trst3;
   ScanInterface TC0 {
      Port tdi1;
      Port tdo1;
      Port tms1;
      Port trst1;
      Port tck1;
   }
   ScanInterface TC1 {
```

```
      Port tdi2;
      Port tdo2;
      Port tms2;
      Port trst2;
      Port tck2;
   }
   ScanInterface TC2 {
      Port tdi3;
      Port tdo3;
      Port tms3;
      Port trst3;
      Port tck3;
   }
   Instance Die_1 Of die1 {
      InputPort tck  = tck1;
      InputPort tdi  = tdi1;
      InputPort tms  = tms1;
      InputPort trst = trst1;
   }
   Instance Die_2 Of die1 {
      InputPort tck  = tck2;
      InputPort tdi  = tdi2;
      InputPort tms  = tms2;
      InputPort trst = trst2;
   }
   Instance Die_3 Of die1 {
      InputPort tck  = tck3;
      InputPort tdi  = tdi3;
      InputPort tms  = tms3;
      InputPort trst = trst3;
   }
}
```

Clocking in Internal and External Scan Test Modes

The 2.5D flow enables the simultaneous testing of dies in internal mode and at-speed die-to-die testing in external mode. These two modes require separate clock control configurations. You must add clock multiplexers at every clock-out port to support multiple test modes in the integrated design.

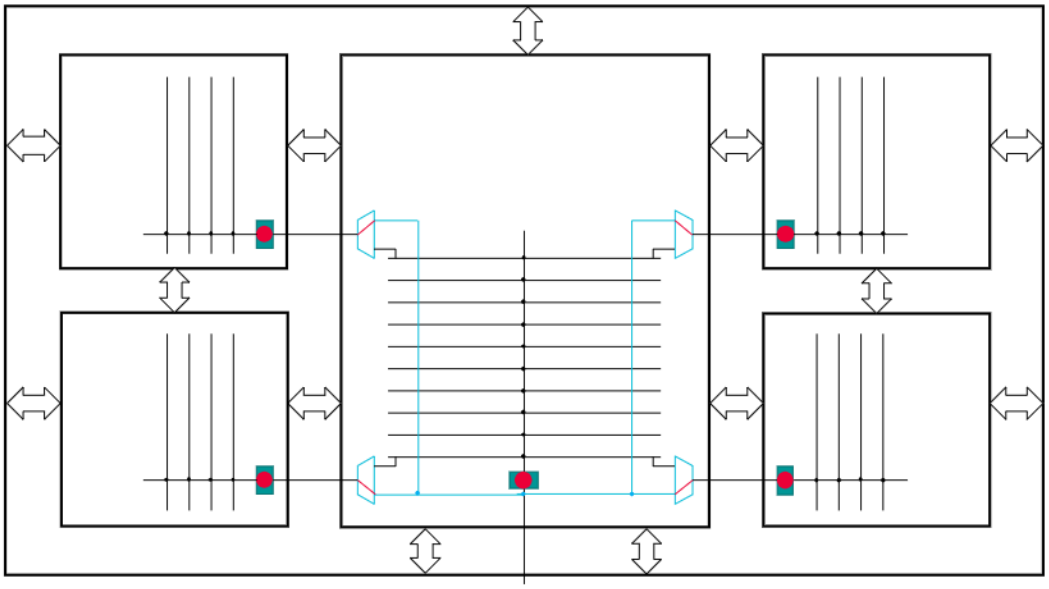
"Logic Test Instruments Insertion (EDT and OCC)" on page 55 describes the process for adding the muxing logic in the primary die.

Clock Control in Internal Mode

You must insert OCC bypass muxes to run scan on any combination of dies in parallel. This enables non-primary dies to receive a free-running clock, even when the OCCs in the Primary die are active.

If you do not inject the muxes, you cannot run the internal test mode of the non-primary dies in parallel with the primary die. Each die must have its own clock control in internal test mode. The additional clock muxes are required to provide asynchronous clocks directly to dedicated OCCs in dies and to avoid cascaded OCCs. The red dots in the following figure show active OCCs in internal test mode.

Figure 7. Clocking in Internal Test Mode

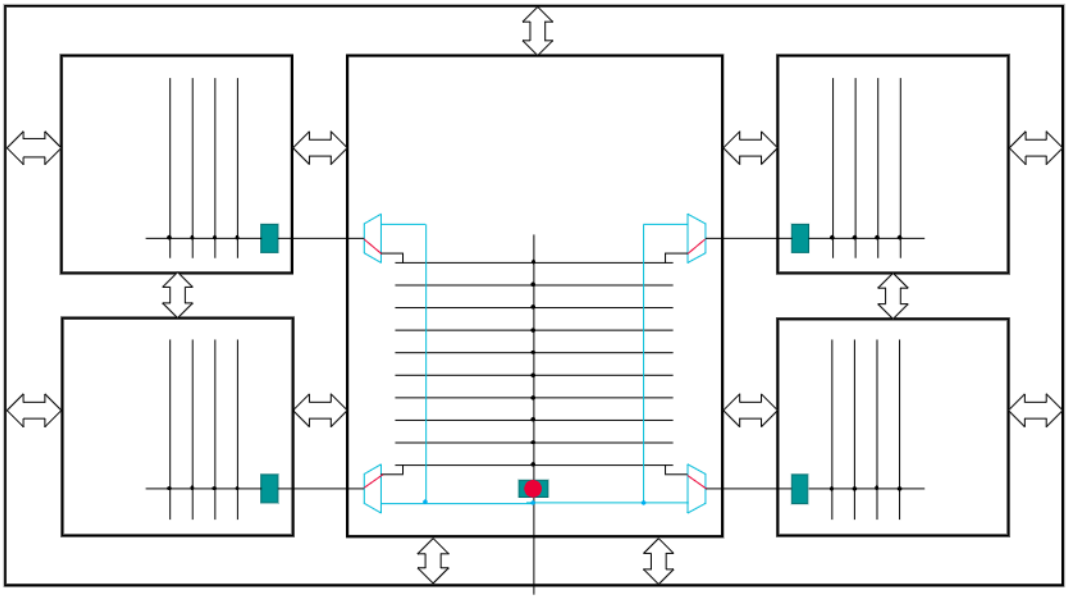


Clock Control in External Mode

The OCC bypass clock muxes steer the at-speed functional clocks to each die, ensuring that the inter-die logic paths are scan-tested with their native functional clocking.

You must synchronize the wrapper chains in the dies to test die-to-die connections. The OCC in the primary die is active during its external test mode and is promoted to the external test mode of the primary die during scan insertion. The controlled clock is delivered to all dies through additional clock multiplexers in external mode using the functional clock connections. The red dot in the following figure shows the active OCC in the primary die for this configuration.

Figure 8. Clocking in External Test Mode



Boundary Scan Requirements for 2.5D Dies

Dies used in 2.5D designs require TAP client interfaces. You can reuse an off-the-shelf chip die in a 2.5D design without modification of the BoundaryScan infrastructure. The unified package-level BSDL file describes the BoundaryScan cells not connected to the package level as internal cells.

The main disadvantage of an off-the-shelf boundary scan approach is that shifting may be unnecessarily long. Therefore, you should modify the BoundaryScan and add at least two bonding configurations created from the BoundaryScan logical groups. You should know the physical layout of the ports before grouping them into BoundaryScan logical groups and creating the bonding configurations.

The required bonding configurations are:

- **external** — A group of ports connected directly to the package boundary.
- **all ports** — Used in the standalone BoundaryScan test before package integration and to test the die interconnections at the package level.

The external group consists of ports connected directly to the package boundary. The tool extracts the external groups from all dies to the final BSDL description at the package level. The “all ports” group consists of all BoundaryScan cells in the die.

One method of generating bonding configuration enable signals is to use a custom DftSignal. Set the DftSignal reset value such that after the logic reset, the external configuration is the default active bonding configuration. You can define as many bonding configurations as needed.

Additional Boundary Scan Requirements for Primary Dies

The BoundaryScan infrastructure requirements for the primary die are the same as those for the non-primary die. Additionally, the primary die TAP should have a local TDI port to bypass the TAP clients in the other non-primary dies.

Use the TDI local port in the case of the BYPASS or DEVICE_ID instructions to ensure the 2.5D design is compliant with the IEEE 1149.1 standard. Only the 1-bit BYPASS and 32-bit DEVICE_ID registers from the primary die are accessible and used at the package level, matching the unified package BSDL.

Design Prerequisites for 2.5D Boundary Scan Pattern Generation

You must satisfy several conditions before generating boundary scan test patterns for a 2.5D design.

TAP Controllers

You must configure TAP controller connections for test mode, and the compliance enable pins on the dies to their active values. If the compliance enable pins are connected to package-level ports, Tessent Shell applies the correct values automatically; otherwise, you must set them explicitly.

Required Input Data

The following input data is needed:

- A netlist description of the interposer in either Verilog or VHDL format.
- A Verilog or VHDL interface view netlist description.
- A BSDL description of each die that includes boundary scan.
- An ICL description of the chip (optional).

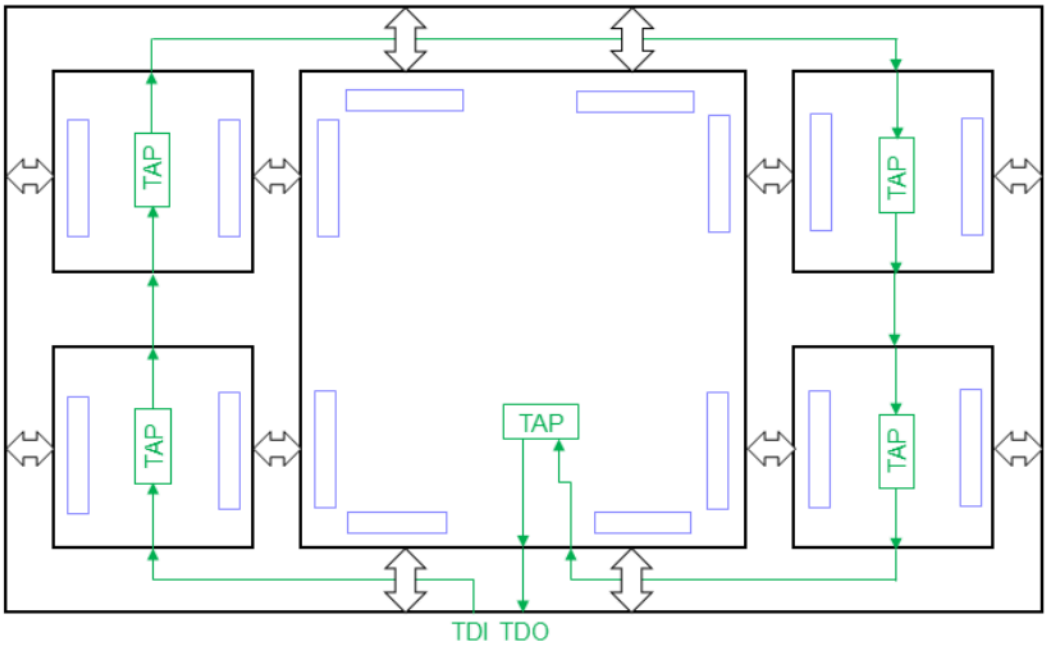
If you do not supply the die descriptions, Tessent Shell derives these from the package netlist and the BSDL files.

Note:
In addition to the required input data, each die must have an IEEE-compliant TAP and boundary scan inserted for all digital I/Os.

Supported Design Topology

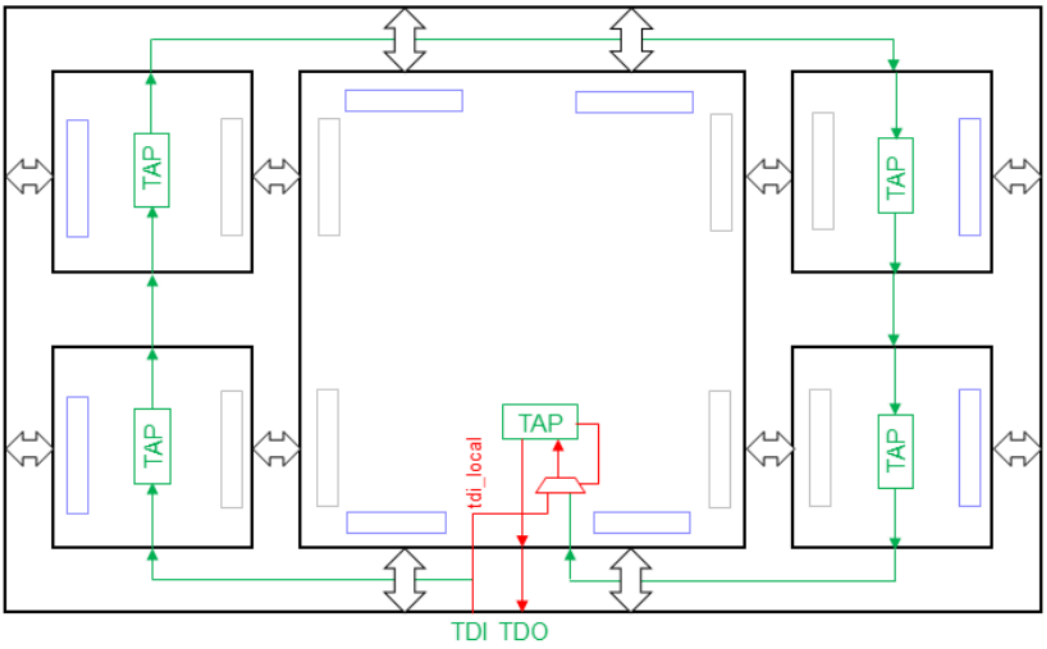
The following figure shows that the dies can be daisy-chained, meaning that the TDO of each die connects to the TDI of the next die. The TDI of the first die connects to the package TDI port, and the TDO of the last die connects to the package TDO port. In this case, you must connect all TCK, TMS, and (optionally) TRST die pins to the same package-level TCK, TMS, and TRST ports.

Figure 9. Daisy-Chained Configuration



The following figure shows a variation on this topology where you can also connect the package TDI to the tdi_local of the last die in the chain. This configuration enables the generation of a single, package-level unified BSDL for use at the board or system level for this device. To facilitate a unified BSDL, the TAP controller of the last die controls the select input pin of the tdi_local mux.

Figure 10. Daisy-Chained Configuration With Bypass



This configuration enables you to bypass all other dies and has only the BYPASS and DEVICE_ID registers of the last die in the data shift path. All other register shifts use the normal TDI. This makes the package IEEE 1149.1 compliant with a 1-bit BYPASS register and a 32-bit DEVICE_ID register in the path between the package TDI and TDO.

To insert the TAP controller into the die closest to the package TDO with Tessent Shell, you can automate the insertion of the mux by specifying the tdi_local property in the DftSpecification/IjtagNetwork/HostScanInterface/Interface wrapper and point it to a top-level port.

To specify the location and port name for the tdi_local mux, use the Tap/Interface/TdiMux wrapper with the properties as shown in the following:

```
Tap(main) {
  Interface {
    TdiMux {
        parent_instance      : <instance_path>; //for the tdi mux
        leaf_instance_name   : <leaf_name>;     //for the tdi mux
        tdi_local            : <port_name>;
        bypass_device_select : <port_name>;
        ir_select            : <port_name>;
        tdi_selected         : <port_name>;
    }
  }
}
```

Generating Boundary Scan Interconnect Test Patterns for a 2.5D Design

This section describes how you can create boundary scan test patterns to test the interconnect between devices in a 2.5D design.

A 2.5D design is one type of multiple-die SiP and consists of two or more dies instantiated on an interposer substrate. Each die has a TAP controller and a BoundaryScan chain, described in a BSDL file.

The interposer provides signal connectivity between the dies, and from the dies to the package pins. In a passive interposer, the connectivity consists of wires only; there is no logic outside the dies. The interposer also provides power and ground to the dies, but this process does not test these. The tool uses boundary scan patterns to test the signal connectivity between dies and to the package pins.

DFT in 3D Designs

A 3D design includes one or more logic dies stacked on a base SOC die in a chip. Each die is designed and tested using the Known Good Die (KGD) wafer probe technique to limit the possibility of assembling a faulty die or chiplet into the 3D package.

3D Design Overview

The IEEE 1838 standard defines the test access architecture for 3D stacked integrated circuits.

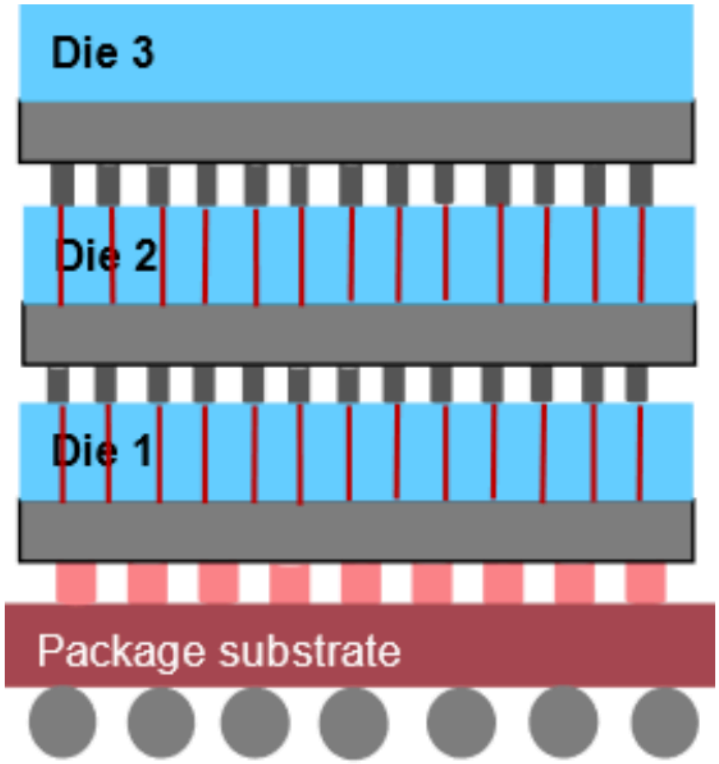
In addition to the test access architecture, the IEEE 1838 standard also sets requirements for test access. You should create the test access with a mandatory serial test access interface and a parallel interface (optional for the standard but required for the Tessent flow). The serial access uses the Primary Test Access Port (PTAP) and the Secondary Test Access Port (STAP). The PTAP and STAP control the test and debug features in each die. The Tessent Multi-die flow uses the PTAP/STAP interface as the die-to-die IJTAG interface. The parallel access is called the “Flexible Parallel Port” (FPP). The FPP covers the multi-bit test access between the primary and secondary interfaces and the core functional logic in the die. The FPP is also the interface test access for the die stack under test.

The individual dies use the standard Tessent DFT insertion flows. IJTAG insertion can insert the PTAP and STAP for serial IJTAG support, and the FPP is implemented with the Tessent SSN bus (required for Tessent 3D support). Wrapper analysis reuses physical block wrapper cells and incorporates them as the Die Wrapper Registers (DWR). The DWR enables you to perform die-to-die scan captures to test the through-silicon vias (TSV) connections between dies.

Upper dies connect only to other dies in the stack with TSVs. The upper die ports are typically not equipped with full-sized bumps and Boundary Scan and use fine-pitch micro-bumps with TSVs for die-to-die interconnect. The wrapper cells provide die isolation. You must add probe pads for at least the TAP interface IOs and SSN bus for Known Good Die tests. These sacrificial probe pads are not used after stacking.

The following figure shows a design with a base die and two upper dies. The base die provides the test interfaces to the upper dies in the stack. All dies above the base die are considered the upper dies. The DWR is the die’s external mode wrapper chains that interface to the die’s ports. The DWR bypasses the EXTEST wrapper chains that connect to other die blocks.

Figure 11. 3D Design Stack



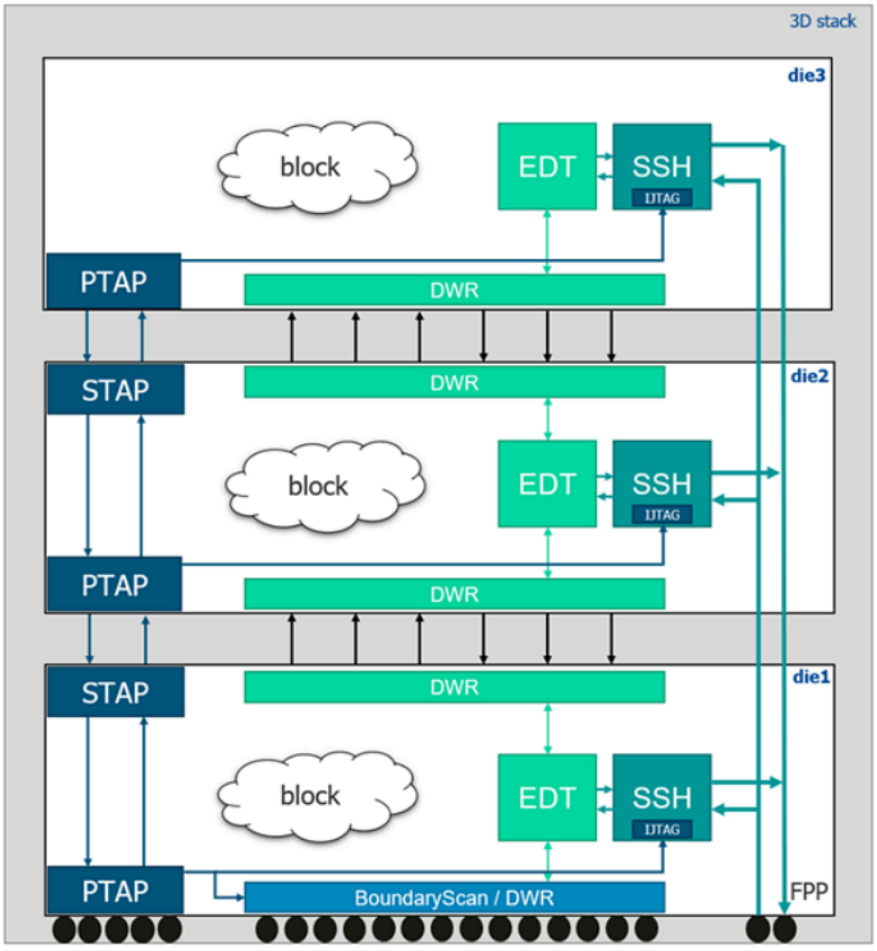
You can use 3D and 2.5D methodologies together in the same package. For example, creating a 3D stack on top of the die used in 2.5D creates a 5.5D design (per the IEEE 1838 standard), which benefits from both methods. Refer to "DFT in 3D Designs" on page 28 for more information.

3D DFT Overview

Due to the nature of the 3D die stacking, there are some limitations in accessing the DFT pins of the upper die from the package level. Challenges exist for both wafer test and package test.

Each die in the package has its own DFT logic to provide testability for the die and the package after the integration. Wire connections connect the dies to each other and connect the base die to the package pins. No other structures can exist at the package level.

Figure 12. 3D Die Stack With DFT



CAUTION:
Do not use assign statements for bidirectional die-to-die connections. Create these connections by using the same net name for both pins as in the following:

```
wire net1;
M1 U1(.pin(net1));
M2 U2(.pin(net1));
```

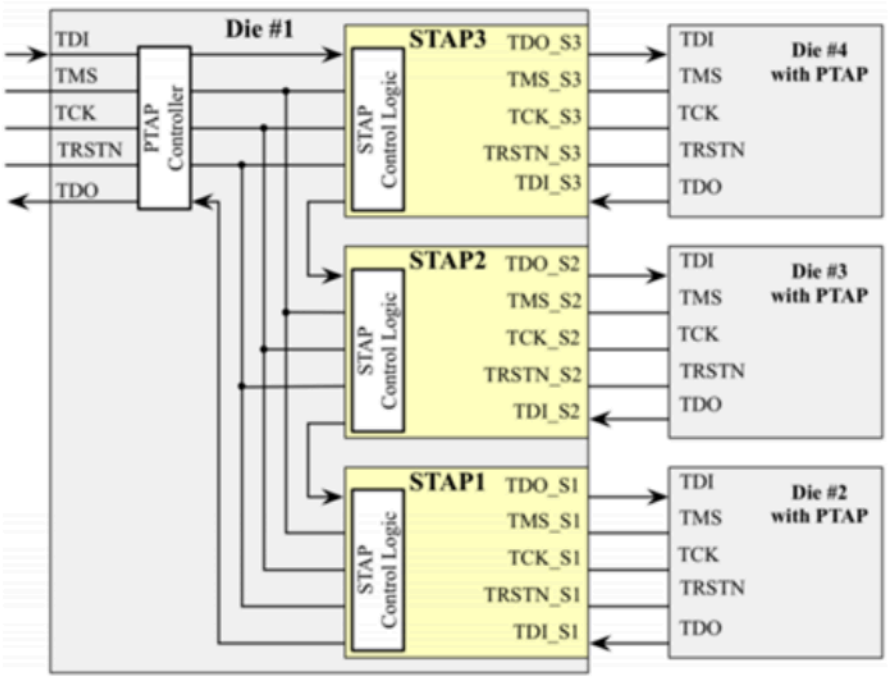
Each die in the 3D integrated package has a primary test access port (PTAP), and all dies that are not the top of the stack (middle dies) should also have a secondary test access port (STAP) inserted. Die wrapper cells handle the interconnect testing, and are part of the die external mode scan chains.

Use Tessent Streaming Scan Network (SSN) as the logic test infrastructure to integrate the logic tests for all dies in the 3D stack.

Each die must have a serial PTAP to provide an interface and access to all on-die test and debug features. This PTAP is a normal IEEE1149.1 TAP with a special set of host PTAP ports: stap_to_ce, stap_to_se, stap_to_ue, stap_to_si, and stap_from_so. Also, a two-bit test data register (3DCR) is added to control PTAP operations. The two bits of 3DCR are reset_suppression and STAP_EN. When STAP_EN is 1, the PTAP allows the STAP module in front to become in series with the current PTAP. If STAP_EN is 0, this PTAP only works as an 1149.1-compliant TAP. The reset_suppression bit enables a five-pulse TMS-driven reset.

Insert STAPs facing next to each die's PTAP to drive and control the selection of these dies. Every STAP includes a five-pin interface that is selectable to the current die's previous die. The die that contains the STAP is the host die to the die whose PTAP accepts signals from the STAP. Refer to Figure 13.

Figure 13. PTAP/STAP Network With Four Dies



STAP signals are selected and configured by a 3DCR. If you configure a 3DCR to disable its die, the STAP gates off the TMS signal and the serial data path to deactivate the die.

Note:
The 3D stack is compliant with the IEEE 1149.1 standard only if the base die's PTAP does not include the STAPs into the serial path. This means that if the stap_en signal is low (the STAPs are not enabled), the PTAP bypasses the STAPs in the die and all downstream JTAG networks. The IEEE 1149.1 standard requires a 1-bit- BYPASS and a 32-bit DEVICE_ID instruction that cannot be created in this situation.

Scan Mode Requirements in 3D Designs

Scan Mode Requirements in 3D Designs

In a 2D design, the core uses internal scan mode to test the logic inside the core and external scan mode to simulate the top-level scan in the unwrapped mode. You require an additional scan mode in a 3D design for patterns at the package level.

Some core ports may be connected directly to the die boundary. You can reuse the wrapper cells on these ports as wrapper cells of the die. Group these wrapper cells into the parent external mode of a core. Considering the 3D stack scan modes from top to bottom of the stack, the die's wrapper cells, together with the parent external mode of a core, create the die graybox needed to simulate the die-to-die interconnections in the package unwrapped mode.

The following table shows the hierarchy of modules and scan modes in a 3D stacked design:

Table 1. Scan Modes Inserted in Each Module

Core	Die	Package
Internal	Retargeting of core internal	Retargeting of core internal
External	Internal	Retargeting of core internal
Parent external	External	Unwrapped

You can retarget the internal modes from any hierarchy level to any parent hierarchy level. For example, a core’s internal mode patterns can be retargeted to the die and package levels.

The following figures show the scan modes of each module in a 3D stack during internal and external scan tests.

Figure 14 shows the core in internal scan mode (grayed-out logic is not in the scope of this test). The core internal scan mode can be retargeted to the die and package levels. You can run the retargeted core-level internal tests in parallel at the package level because of isolation from the die.

Figure 14. Core Internal Scan Mode Test

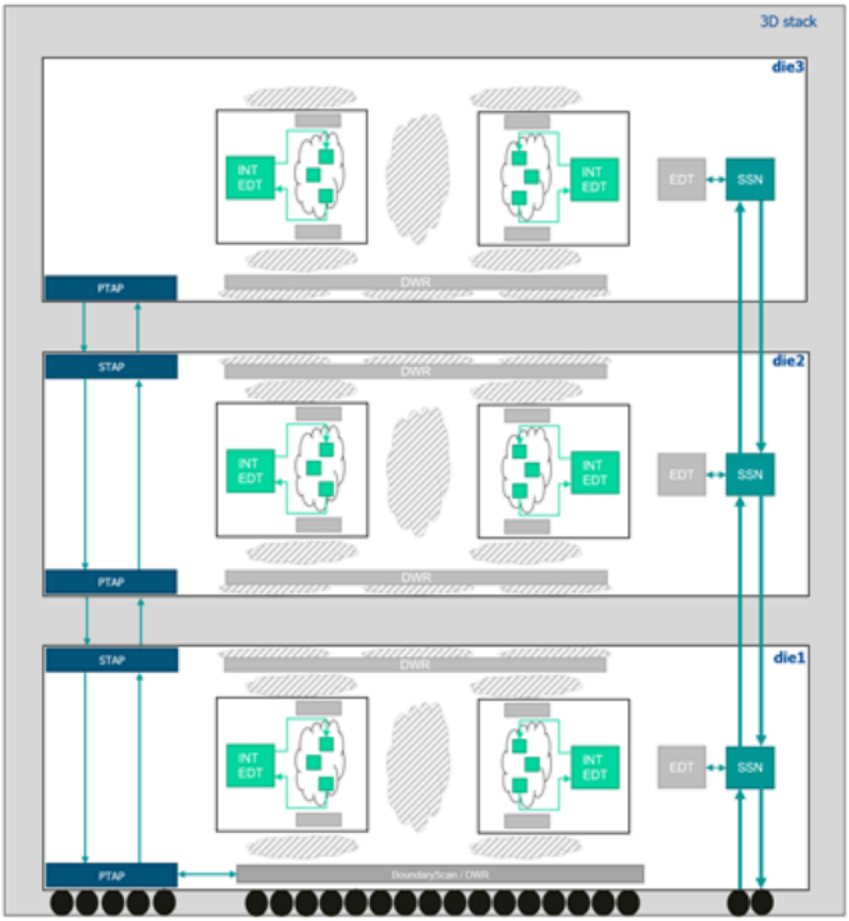


Figure 15 shows the cores in external scan mode during the die internal scan mode test (grayed-out logic is not in the scope of this test). You can retarget the die internal scan mode to the package level. You can run the retargeted die-level internal scan tests in parallel at the package level because of isolation from the package.

Figure 15. Die Internal Scan Mode Test

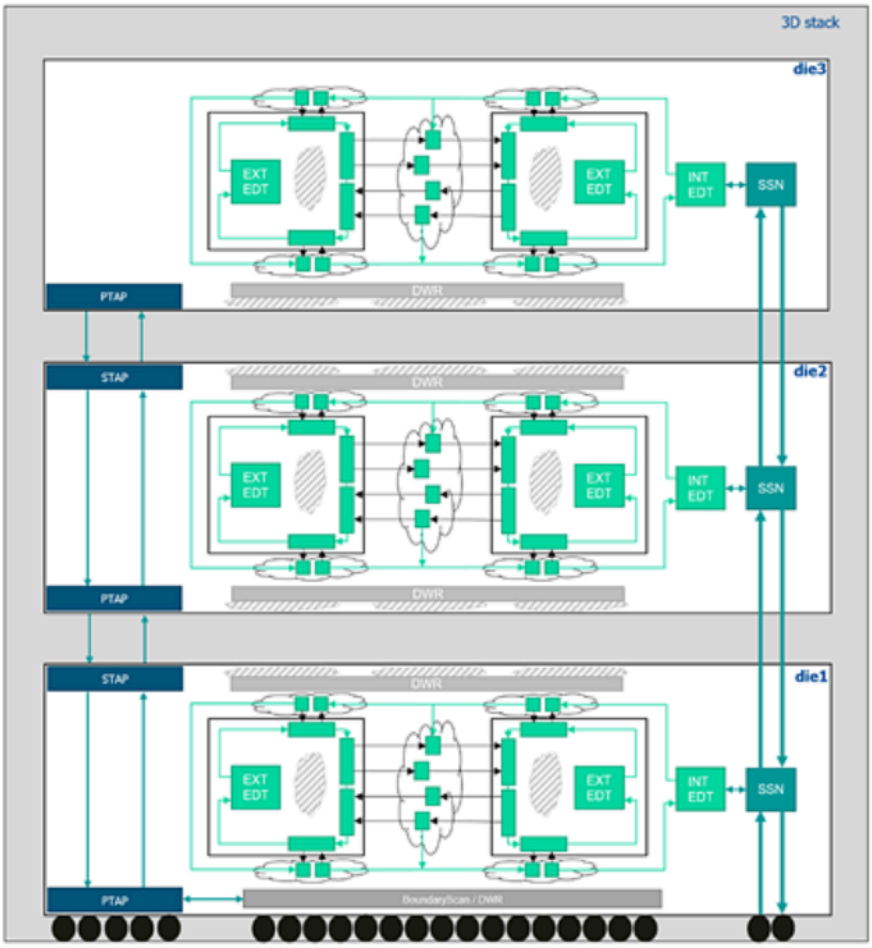
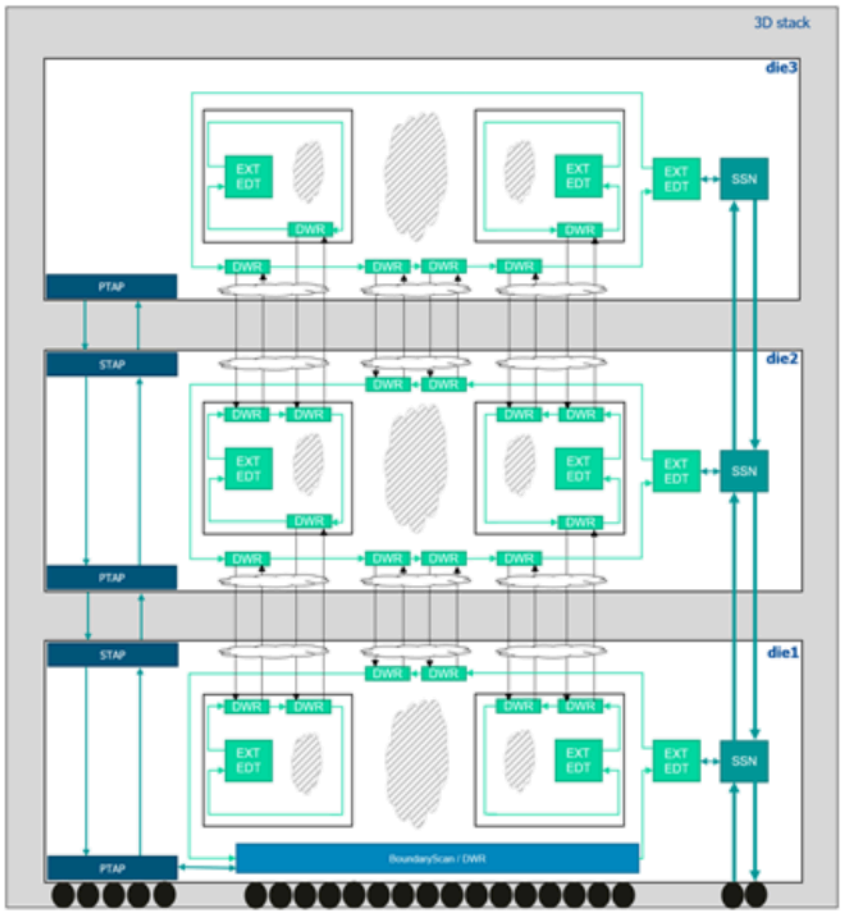


Figure 16 shows the cores in parent external scan mode and dies in external scan mode during the package-level die-to-die test (grayed-out logic is not in the scope of this test).

Figure 16. Package Unwrapped Scan Mode and Die-to-Die Test



Chapter 2
DFT and Design Considerations

This section describes some of the architecture and design requirements associated with 2.5D designs and SiP DFT planning.

SiP DFT Inputs and Considerations

Certain considerations are needed to support SiP DFT planning. You must define the placement and orientation of the dies on the interposer and the test-related package pins. For example, the die-to-die boundary scan chain should follow the die placement ordering on the interposer.

You should consider the following items for 2.5D and 3D die testing:

- Wafer probe: die-level sacrificial pads must exist for all I/Os needed for test. Add as many die-level power and ground sacrificial pads as possible.
  - The primary limitation is area.
  - Add a proportionate number of pads for each IC supply based on projected current requirements.
  - Ensure your probe card can handle the current without burning needle tips through the sacrificial power pads.
  - Package test: add three or four sacrificial voltage observation balls per IC supply. These are used with the tester supply's sense and force capabilities to mitigate IR drop during test. Add a safe current limit per supply to avoid overcurrent conditions that could burn the probecard (wafer probe), socket, DUT, or loadboard.

CAUTION:
Overcurrent can damage the probe card if there are open observation paths between the DUT and the tester power supply.

- All I/Os needed for package test should have direct access to package-level pins. Any unused chip inputs must be bonded in the package to be statically high or low. This can be validated with the add_pin_constraint command or modified with the PatternsSpecification.

Power and thermal considerations are important for test because the vectors are optimized for efficiency. This means they may exceed functional activity factors leading to IR drop, noise, and temperature excursions during test. DFT tools incorporate features such as power-aware EDT capabilities that limit simultaneous switching activity during scan. Also, SSN has inherently lower power because of non-overlapping shift and capture clocks, so SSN may also improve power and thermal factors during scan. We recommend that you always analyze the DFT modes for static and dynamic IR drop.

Chiplet Vendor Handoffs

Chiplet vendors already have DFT and IP test included in their chip design because they provide fully-tested bare dies. The chiplet vendors typically provide a design kit to implement their chiplet in a multi-chip package.

At a high level, you must have the following chiplet vendor handoffs to support DFT testing:

- The chiplet must consist of a fully-tested bare die (Known Good Die).
- They should provide Tessent design files. These include ICL, PDL, TSDB, TCD, and supporting files such as SDC and min/max SDF.
- With the DFT netlist:
  - Full chip netlist (used for package testing of individual dies).
  - Die-to-die ATPG (reduced top-level with wrapper chains, SSN bus, SSN clock, IJTAG, TAP, and boundary scan only).
  - All required libraries: ATPG, standard cell I/O, MBIST, timing, and verification.
  - ATPG scripts.
  - A test integration guidelines document.

Integrating multiple ASIC and chiplet devices into a single package presents several testing challenges for the individual chiplet devices and the die-to-die interfaces between the chiplets. Their respective suppliers deliver all 2.5D ICs as pre-tested, known good die devices. Die or interposer interconnect may be damaged during the handling and SiP assembly process. There may be opens or resistive connections between the dies and interposer, so die-to-die testing is needed to check all die-to-die connections.

You need post-assembly package testing for testing the individual chiplets. To facilitate in-package testing, the chiplet suppliers must provide the respective manufacturing test patterns to ensure that the device is operational after assembly into the SiP package. Advanced test access methods such as IJTAG are required to initialize and force static test signals if static IC test pins are not available through external package pins.

Traditional ASIC devices use structured DFT production testing as the primary test method. Boundary scan is used for I/O and slow-speed interconnect testing, MBIST and repair for internal memory testing, and scan testing for internal digital logic. Scan testing also facilitates slow-speed and at-speed die-to-die testing. Analog or IP testing uses additional testing techniques, including functional patterns, loopback, and other BIST logic tests. You should use IJTAG to initialize IP testing. Many IP vendors provide IJTAG interfaces with PDL and ICL files for IP tests. IJTAG is preferred for test access, as it simplifies top-level or full SiP simulation and pattern generation. Individual chiplet tests, die-to-die tests, and top-level IP tests are combined into a complete ATE production test program for the multi-die SiP device. The SiP integrator may also include some overall functional tests and DC parametric tests using package-level boundary scan.

The BIST and IP-related ATE tests are accessed and initialized through a standard IEEE 1149.1 test access port (TAP) serial interface, which typically runs at a clock rate of 10 MHz to 50 MHz. FastIJTAG can significantly increase the TCK and IJTAG clock rate. FastIJTAG is beneficial for long IJTAG

sequences such as SerDes firmware loading. IJTAG is the preferred method for internal test access of DFT and IP testing.

Chiplet SSN scan testing must accommodate I/O overrides at the package pins, and the interposer must connect the SSN bus out to the SSN bus in of the next die on the interposer. The SSN bus clock is similarly routed with the SSN bus interfaces. In SiP designs with multiple chiplets, the interposer serially connects the TAP TDI and TDO test pins between chiplets. Additionally, SiP designs with large interposers and many chiplets may have interposer routing parasitic delays, which could further reduce the operational test speed of the SIP device. You should carefully plan the interposer test routing and SiP planning to minimize these delays. Also, consider signal integrity analysis, static timing analysis, and SDF timing simulations to verify the SiP interconnect delays.

Chiplet devices operating in their respective test modes may use significantly higher power than the functional mode power. Careful power planning, analysis, and control are required. IR-drop analysis should be performed for all test modes to ensure good yield and to help plan and limit the yield-limiting effects of IR drop during test.

The following lists several of the published test standards and chiplet deliverables:

- Boundary Scan Description Language (BSDL) – IEEE 1149.1 or IEEE-1149.6.

  This is the chiplet boundary scan model used for slow-speed I/O and interconnect testing inside the package and external to the package at the system and board levels. Supports DC and AC coupled interfaces, as required. It describes the TAP controller, typically used as a convenient test access mechanism for transferring data to and from a chiplet's various test modes.

- Tessent FastScan ATPG models or Verilog User-Defined Primitive (UDP) models for standard cell and I/O models to test logic and interconnect are required. We recommend Tessent FastScan models.

- Internal JTAG (IJTAG) IEEE 1687 architecture is required. This provides test set up and patterns for the chiplet IP and DFT tests.

  - Instrument Connectivity Language (ICL). This describes the internal test hardware structures of the device under test.

  - Procedural Description Language (PDL): This creates test patterns and specialized test for IP such as SerDes, and creates chiplet test initialization sequences such as PLL initialization for die-to-die at-speed scan tests. You can also use PDL for Memory BIST and repair patterns as part of an embedded chiplet delivery.

- (Optional) IEEE 1500 Core Test Language (CTL) description. We recommend IJTAG. This is required for HBM, but IJTAG can interface to an IEEE-1500 interface as IEEE-1500 is a subordinate standard.

- (Optional) IP firmware, if required, for test initialization of chiplets or IP core physical layers (PHYs) embedded in a chiplet. Typically, load this into the design using IJTAG.

- IJTAG graybox. We strongly recommended this.

  This is an IJTAG graybox description of test logic interfaces to support die-to-die ATPG and simulation of chiplet production test patterns by removing the bulk of the chip logic other than hierarchical wrapper scan chains and IEEE 1838-related logic. This significantly improves full SiP die-to-die test run time. It should also include a Verilog wrapper netlist that contains the following:

    - Wrapper scan
    - Boundary scan
    - SSN bus and clock

- Minimum, typical, and maximum SDF. These simulation timing files are needed for all of the supplied netlists, including the interposer.
- Full-chip ATPG vectors. These should be STIL (IEEE 1450.1) or parallel waveform generation language (WGL) files and consist of the ATPG vectors generated by the DFT tools to test the internal logic of the chiplet device through the chiplet I/O pins.
- Full-chip MBIST repair vectors. These consist of STIL or parallel WGL test patterns and are generated by the DFT test tools to test the internal memories or logic of the chiplet device through the chiplet I/O pins. Die-level IJTAG vectors for IP test, including PDL, ICL, STIL, and Verilog or VHDL testbenches.
- UPF – IEEE 1801 or CPF (optional). These models for the chiplet define the power intent and implementation of the chiplet device, including all unique power domains and supplies.

Timing

You should use SDC timing constraints generated by Tessent tools for the various DFT modes for DFT timing closure. You can supplement them with additional disabling constraints to prevent the activation of false paths between DFT logic and functional logic.

We recommend Min/Typ/Max SDF and Standard Parasitic Exchange Format (SPEF) timing files for 2.5D die-to-die interposer connections to support timing simulation and Static Timing Analysis (STA). The packaging signal integrity team typically generates these files.

The following are primary die-to-die interfaces for DFT:

- DFT
    - SSN bus
    - TAP and boundary scan
- Functional I/O
    - TAP and boundary scan
    - Scan capture

Chapter 3
Testing and Diagnosis

This section describes some of the testing and diagnosis procedures used for multi-die designs.

Known-Good-Die Test

Known-Good-Die (KGD) wafer testing is one method to ensure that the dies bonded into the package are fully functional. With KGD testing, a final test program for each individual die is run, including all at-speed DFT and IP tests.

The following are some challenges for KGD testing:

- Most wafer probe cards do not support the fine microbump pitch (spacing) used for the chips in multi-die packages. This requires special techniques and trade-offs for signal and power delivery. The wafer probe environment tends to be noisy, with high tester loading, which affects at-speed testing. Technologies such as direct-dock probing may help to alleviate this problem.

- When you use mixed-process lithography, special characterization techniques such as speed binning to match dies from different process nodes and corners may be needed.

Sacrificial Pads

Sacrificial Pads

Most probe cards are designed to support standard bump and pad pitches, and cannot support the many thousands of microbumps used in multi-die packages. You can use a subset of probe pads with standard spacing to support wafer probe to work around this problem. These are known as sacrificial pads because they take up valuable area but are unused after test.

A successful sacrificial pad strategy includes defining a super set of signal bumps to support all test modes and a subset of power and ground bumps as sacrificial pads. These signal pads include IJTAG and SSN pins plus functional clock and reset inputs, at a minimum. The packaging and signal integrity teams should carefully consider the power and ground sacrificial pad assignments to avoid issues such as severe IR drop (which causes yield loss) and damaged wafer probecard tips caused by excessive power.

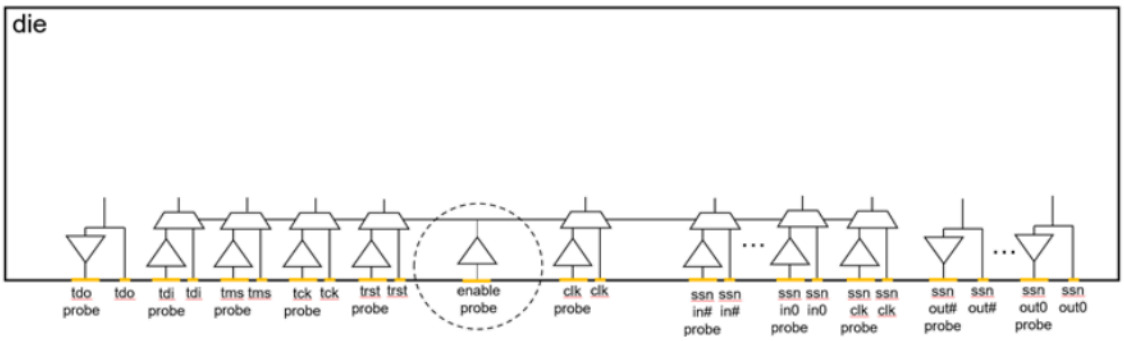
Muxed pads and shorted pads are sacrificial pad strategies that provide duplicate signal, power, and ground pads with standard bump pitch for test.

Muxed Pads

The muxed pad implementation duplicates each signal probe pad by muxing the functional and test paths together. I/O buffers may exist for all muxed inputs and shorted outputs.

Figure 17 shows an adequate sampling of power and ground with standard probe pads are shorted to microbump power and ground.

Figure 17. Muxed Pad Implementation



The muxes require an extra enable_probe signal to act as the probe pad and microbump mux select. You must drive the enable_probe signal to be inactive when the dies are packaged, so that the functional paths are selected on all the muxes. This is typically accomplished by connection through the lower dies, or bonding to the interposer.

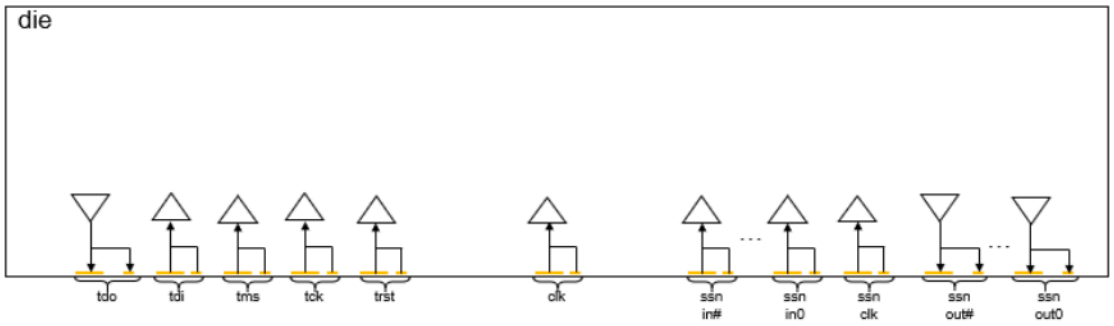
Shorted Pads

The shorted pad strategy shorts the sacrificial pad to the microbump.

Figure 18 shows a shorted pad implementation. The advantages of this technique are simplicity and minimal area impact. The disadvantage is that it creates dangling nets that could generate noise. These issues can be identified and corrected during package signal integrity analysis. Test signals tend to be

relatively low speed compared to functional mode, so the impact should be minimal. The muxed I/O technique may be better if you use very fast I/Os for test.

Figure 18. Shorted Pad Implementation



Die-to-Die Testing

The die interconnect tests in multi-die designs differ depending on the technology. The die-to-die testing considerations for 2.5D and 3D designs are covered in the following sections.

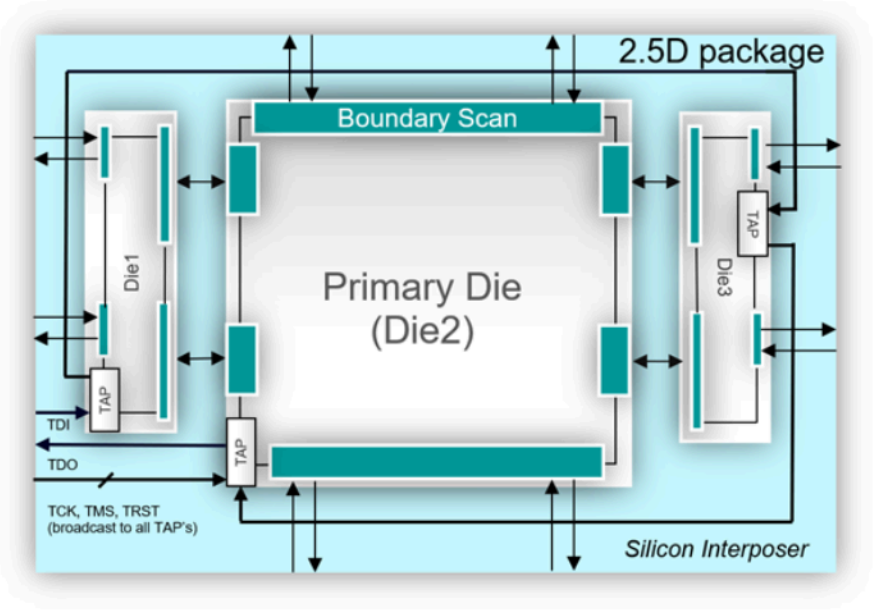
Die-To-Die Testing in 2.5D Designs

A 2.5D design is a type of multiple-die SiP design consisting of two or more dies instantiated on an interposer substrate. In a passive interposer, the connectivity consists of wires only, and there is no logic outside the dies. The interposer provides signal connectivity between the dies and from the dies to the package pins.

Note:
Tessent Diagnosis does not support diagnosis of die-to-die test failures. Refer to "Diagnosis of Multi-Die Designs" on page 48 for more information.

Die-to-die testing in 2.5D designs tests the interposer connections to verify that the connections between the dies and from the die to the package boundary are working correctly. It also validates the package assembly process.

Figure 19. Die-To-Die Testing (2.5D)



Use boundary scan to test these connections. Each die must have a TAP controller and a boundary scan chain, as shown in Figure 19. The BSDL file should describe the boundary scan chain.

Refer to “Non-Primary Die-Level DFT Flow” on page 52 and “Generating the Die-to-Die Boundary Scan Patterns” on page 70 for more details about boundary scan insertion and die-to-die pattern generation in 2.5D designs.

Related Topics

Non-Primary Die-Level DFT Flow

Die-To-Die Testing in 3D Designs

A 3D design is a type of multiple-die SiP design consisting of one or more dies instantiated on top of a base die that connects to the package boundary. The stack-level die-to-die connections consist of wires only, and there is no logic outside the dies. Through Silicon Vias (TSVs) provide the signal connectivity between the dies. The connectivity from the base die to the package is tested with ATPG patterns and conventional boundary scan.

Note:
Tessent Diagnosis does not support diagnosis of die-to-die test failures. Refer to “Diagnosis of Multi-Die Designs” on page 48 for more information.

Die-to-die testing in 3D focuses on testing the TSV connections to verify that the connections between the dies are working correctly. It also verifies the glue logic between the last flop in one die and the first flop in another die. These flops must be scannable cells and be part of the Die Wrapper Register (DWR).

Figure 20. Die-To-Die Testing (3D)

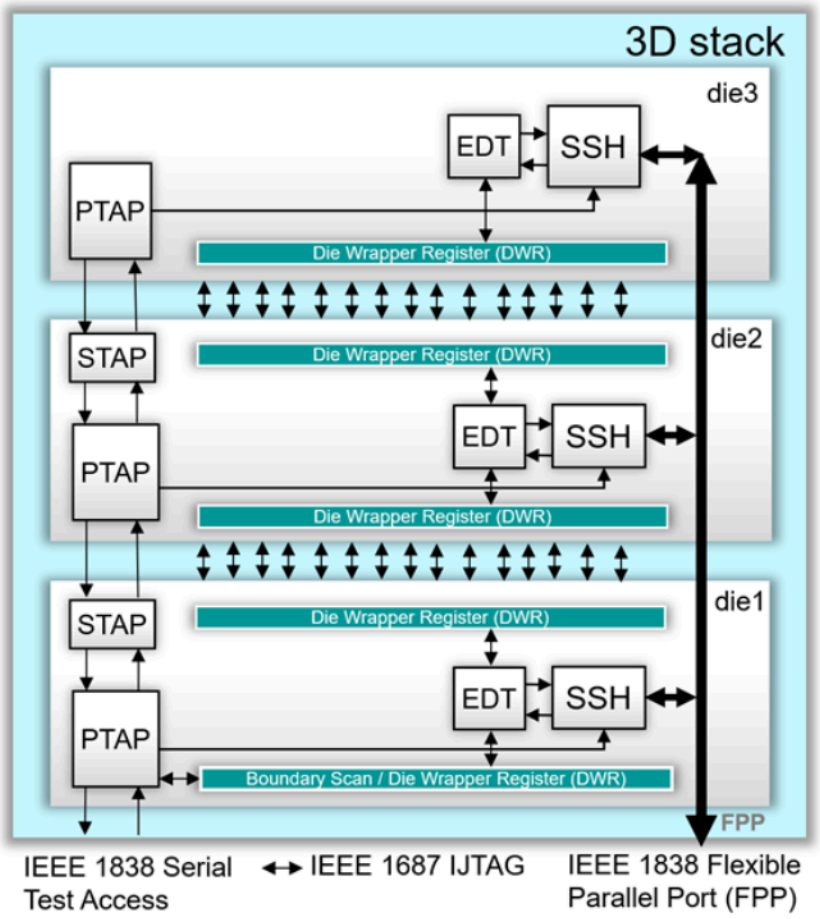


Figure 20 shows how the EDT is used to test the logic between one die's DWR and another die's DWR. In some cases, the scan cells in physical blocks are reused as part of the DWR. Shift the ATPG patterns through the EDT into the DWRs. Each die should have a wrapper chain to isolate the die in internal test mode and to perform the die external test. The SSN bus provides the parallel test access.

Refer to "Tessent Shell Flow for 3D Stacked Designs" on page 114 for more information.

Related Topics

- Generating the Die-to-Die Boundary Scan Patterns
- Second DFT Insertion Pass: Inserting Core-Level EDT, OCC, and SSN
- Performing Scan Insertion With Tessent Scan (Core Level)
- Second DFT Insertion Pass: Inserting Die-Level EDT, OCC, and SSN
- Performing Scan Insertion With Tessent Scan (Die Level)
- Generating Stack-Level ATPG Patterns

Diagnosis of Multi-Die Designs

The manufacturing process of multi-die designs uses known good die (KGD). A KGD is a die that has passed scan testing at the wafer sort level. The multi-die design package does not introduce new logic. The package establishes connectivity of package pins to die and between the known good die. Multi-die design testing focuses on these die-to-die connections.

Note:
Tessent Diagnosis does not currently support diagnosis of die-to-die tests for the location of package interconnect defects.

Tessent Diagnosis fully supports single-die designs. This is the standard diagnosis flow that uses a Verilog view in the flat models and layout databases (LDB) constructed from LEF/DEF.

The standard diagnosis flow uses ATE failure logs and hierarchical netlists based on the flat model to determine symptoms and suspects. These electrical results are more useful for failure analysis when you use them with LDBs of the physical design to cross-map results. Multi-die designs do not typically have LEF/DEF files of the package interconnect.

Diagnosis runs at the same hierarchical level corresponding to the level at which ATPG generated the patterns. If you retarget scan patterns, you must reverse-map any pattern fails on the ATE before running diagnosis for yield learning. This is the standard flow for each single-die design that goes on to make a multi-die design.

For yield learning of multi-die designs, use the standard diagnosis flow for single-die design failures during wafer test. Use volume diagnosis of single-die wafer-level tests to optimize the yield of the known good die feeding the multi-die design packaging.

Chapter 4
Tessent Shell Workflows for Multi-Die Designs

This section describes some of the flow requirements for multi-die designs.

Tessent Shell Flows for 2.5D Packaged Designs

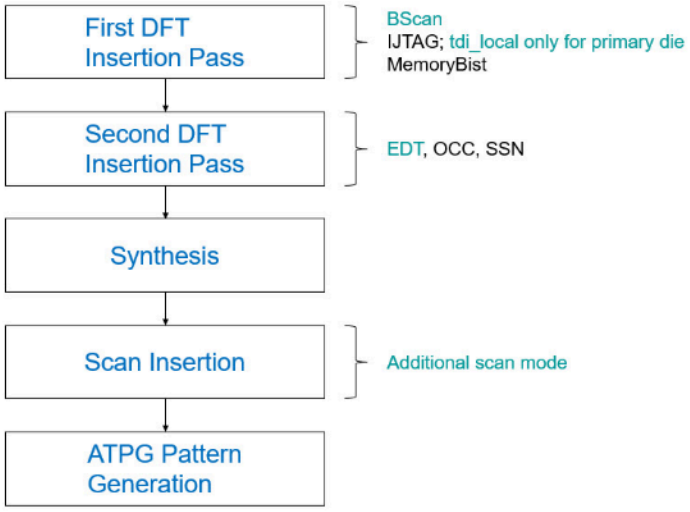
This section describes the DFT flow for dies in a 2.5D design.

2.5D Example Flow

This workflow example covers the insertion and verification flow for 2.5D non-primary and primary die designs.

The Tessent Shell flow enables DFT and scan insertion, independent testing of the dies, retargeting internal modes from dies, and creating consolidated patterns to detect faults on die-to-die connections.

Figure 21. 2.5D Workflow



Non-Primary Die-Level DFT Flow
Primary Die-Level DFT Flow
Die-Level ATPG Pattern Generation
Package-Level Integration and Verification

Non-Primary Die-Level DFT Flow

This section describes the DFT flow for non-primary dies in a 2.5D design, including BoundaryScan, MBIST, OCC, EDT, and SSN DFT logic insertion. Additionally, it covers the preparation of a die for at-speed die-to-die interconnection tests.

First DFT Insertion Pass: Inserting Die-Level Boundary Scan, IJTAG, and MemoryBIST

During the first DFT insertion pass at the 2.5D non-primary die level, insert BoundaryScan, IJTAG, and MemoryBIST.

BoundaryScan DFT Signals

Before inserting boundary scan, you must add the following DFT signals:

- **bscan_input_isolation_enable** — Isolates the die inputs in the internal scan mode using the boundary scan.
- **output_pad_disable** — Disables the output pads of a die in the internal and external scan modes.
- **bscan_clamp_enable** — Enables the tool to explicitly turn off boundary scan chains during the external scan mode. This prevents value propagation failure by a boundary scan cell during the die-to-die at-speed test.

BondingConfigurations need an enable signal source. The following example sources them with a DftSignal. To define this enable signal, register and add a new DftSignal. Specifying the reset_value of this signal as 1 assures that the die is in the BondingConfiguration that enables only the ports directly connected to the package boundary after the reset. This ensures that the TAP is reset as described by the package-level BSDL file for PCB testing.

```
register_static_dft_signal_names external_bonding_config_en \
  -reset_value 1
add_dft_signals external_bonding_config_en
```

In 2.5D designs, there are two EDT modes (internal and external). Including BoundaryScan cells in both EDT modes enables the reuse of the boundary scan cells as wrapper cells for later use during ATPG. However, a tool limitation prohibits adding the connect_bscan_segments_to_lsb_chains property to both EDT controllers.

So, to achieve this, you must specify the handle_logictest_segments_during_scan_insertion property in the BoundaryScan DftSpecification wrapper:

```
set spec [create_dft_specification -replace]
set_config_value $spec/BoundaryScan/handle_logictest_segments_during_scan_insertion on
```

After this, a tcd_scan and a CTL file describe the logic test segments per logical group. During scan insertion, you must connect them to the internal (int_edt) and external (ext_edt) EDT controllers. This setup enables an active BoundaryScan chain during both internal and external modes.

Pinorder and DEF File

The Boundary Scan tool needs the package-level I/O buffer ordering to implement BoundaryScan. Tessent Shell stores the order of pins in the pinorder file, which the tool automatically generates during BoundaryScan insertion. The default order is inherited from the HDL netlist, but you can modify the ordering and pin types.

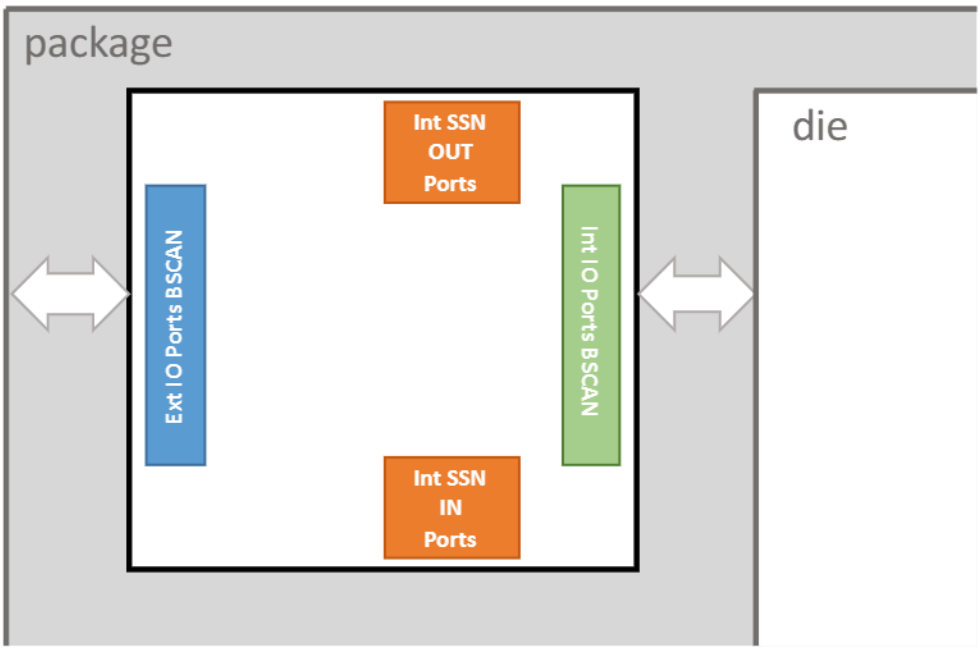
You can optionally provide a DEF file with the port placement, and the Boundary Scan tool generates the pinorder file from the coordinates given in the DEF file. You can also provide a custom pinorder file. The following is an example of a pinorder file:

```
// PortName                 PinName OptionList LogicalGroups
// --------                 ------- ---------- -------------
   g1_bisr_trigger          I1      -          -
   g1_bisr_prog_voltage     I2      -          -
   g1_AC_data_in_0          IO1     -          -
   g1_AC_data_in_0_inv      IO2     -          -
   g1_AC_data_in_1          IO3     -          -
   g1_AC_data_in_1_inv      IO4     -          -
   g1_AC_data_out_0         IO5     -          -
   g1_AC_data_out_0_inv     IO6     -          -
   g1_AC_data_out_1         IO7     -          -
   g1_AC_data_out_1_inv     IO8     -          -
   g3_SSN_in_0              I3      -          -
   g3_SSN_in_1              I4      -          -
   ...
   g3_SSN_out_0             O7      -          -
   g3_SSN_out_1             O8      -          -
   g3_SSN_out_2             O9      -          -
   g3_SSN_out_3             O10     -          -
   g3_SSN_out_4             O11     -          -
   g3_SSN_out_5             O12     -          -
   g3_SSN_out_6             O13     -          -
   g3_SSN_bus_clock_out_7   O14     -          -
   tdi                      I21     -          -
   tms                      I22     -          -
   trst                     I23     -          -
   tck                      I24     -          -
   tdo                      O15     -          -
```

LogicalGroups and BondingConfigurations

To decrease chain length, you can enhance the standard insertion of BoundaryScan DFT by defining LogicalGroups and BondingConfigurations. The following figure shows non-primary die ports assigned to four LogicalGroups. After the integration, the tool groups the ports connected to the package boundary into the external ports (Ext IO) BSCAN group (marked in blue). The next logical group, marked in green, is the internal ports (Int IO) BSCAN group, consisting of the ports that connect the dies.

Figure 22. BoundaryScan Logical Groups in a Chiplet (Satellite Die)



If you do not need the extra SSN-only interconnect patterns, you must group the SSN ports with the other interconnected ports to create an internal BoundaryScan group.

The following example shows how to define the LogicalGroups in the DftSpecification:

```
LogicalGroups {
    LogicalGroup(external_ports_bscan_group) {
      first_port : g1_bisr_trigger;
    }
    LogicalGroup(internal_ports_SSN_ins_bscan_group) {
      first_port : g3_SSN_in_0;
    }
    LogicalGroup(internal_ports_bscan_group) {
      first_port : g2_clk;
    }
    LogicalGroup(internal_ports_SSN_outs_bscan_group) {
      first_port : g3_SSN_out_0;
    }
  }
```

The following example shows how to define the BondingConfiguration for the external_ports_bscan_group:

```
  BondingConfigurations {
    BondingConfiguration(external_bonding_config) {
      enable_signal : DftSignal(external_bonding_config_en);
      active_logical_groups : external_ports_bscan_group;
```

```
    }
    BondingConfiguration(all_ports_bonding_config) {
    }
  }
```

The BondingConfiguration wrapper describes the bypassed logical groups when the enable_signal is active. The bypassed_logical_groups and active_logical_groups properties are mutually exclusive. The tool does not bypass any logical groups specified as active_logical_groups but bypasses all other logical groups specified as bypassed_logical_groups.

Each BondingConfiguration bypassing any previously defined LogicalGroups must have its own enable signal source. The signal source, in this case, is a DftSignal.

The last bonding configuration, which consists of all ports, does not require a dedicated enable signal. This BondingConfiguration is active when all bonding enable signals are inactive.

Note:
The BondingConfiguration(external_bonding_config) enable signal resets to 1, so you must explicitly set the BondingConfiguration(all_ports_bonding_config) signal to 0 while testing.

MemoryBIST Insertion

You can insert Tessent MemoryBIST logic in parallel with the BoundaryScan and IJTAG insertion. The "Implementing and Verifying Memory Repair" section in the *Tessent MemoryBIST User's Manual* describes this flow. For the Tessent Multi-die flow, each die with repair circuitry has its own on-chip repair storage, so eFuse/OTP sharing between dies is not supported. Other than this restriction, the Tessent Multi-die flow does not impact the MemoryBIST insertion flow.

Second DFT Insertion Pass: Inserting Die-Level EDT, OCC, and SSN

During the second DFT insertion pass at the 2.5D non-primary die level, insert any EDT, OCC, and SSN elements your design requires.

Logic Test Instruments Insertion (EDT and OCC)

The following example shows the insertion of two EDT controllers connected to the EDT Sib. The controller_int signal connects the scan chains related to the internal scan mode. The controller_ext signal connects the scan chains related to the external mode. Refer to the Tessent Shell Reference Manual for details on EDT logic insertion.

```
EDT {
    ijtag_host_interface : Sib(edt);
    Controller(controller_int) {
      longest_chain_range  : 40, 60;
      scan_chain_count     : 15;
      input_channel_count  : 3;
      output_channel_count : 2;
      Connections {
        mode_enables       : DftSignal(int_edt_mode);
      }
    }
    Controller(controller_ext) {
```

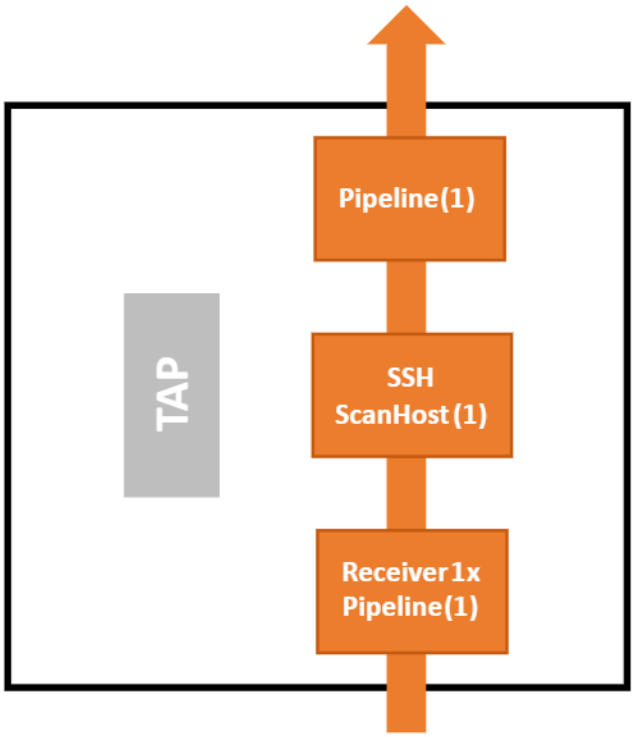
```
      longest_chain_range  : 5, 15;
      scan_chain_count     : 2;
      input_channel_count  : 1;
      output_channel_count : 1;
      Connections {
        mode_enables       : DftSignal(ext_edt_mode);
      }
    }

}
```

All ATPG clocks must have their own OCC controllers. The dedicated mini-OCC in the STI Sib and the BoundaryScan OCC in the inserted BoundaryScan Logic control the TCK clock. The BoundaryScan OCC enables scan testing of the boundary scan logic, and additional OCCs for logic test are not required. Slow-speed scan isolation between the package pins and the dies also uses the BoundaryScan chain.

SSN Insertion

In a non-primary die, the SSN bus goes through the die without additional output paths. In the following example, the SSN data bus is 7 bits wide. The order of elements on the data path according to the DftSpecification is from the bus_data_out to the bus_data_in. The Pipeline(1) wrapper is closest to the output.

Figure 23. SSN Network in a Non-Primary Die



The Receiver1xPipeline(1) wrapper is closest to the input. These pipelines ensure a safe posedge-to-negedge forward timing interface. You can test the die standalone or after integration into the 2.5D package. If the path requires multiple delay cycles, consider using a FIFO instead of the input pipeline.

The following is a DftSpecification representation of the SSN in a non-primary die:

```
# Read the DftSpecification for Scan Streaming Network
read_config_data -in_wrapper $spec -from_string {
  SSN {
    ijtag_host_interface : Sib(ssn);
    DataPath(1) {
      output_bus_width :7;
      Connections {
        bus_clock_in  : g3_SSN_bus_clock_in_7;
        bus_clock_out : g3_SSN_bus_clock_out_7;
        bus_data_in   : g3_SSN_in_%d;
        bus_data_out  : g3_SSN_out_%d;
      }
      Pipeline(1) {
      }
      ScanHost(1) {
        OnChipCompareMode {
          present: on;
        }
        support_from_scan_out_le_strobing : yes;
      }
      Receiver1xPipeline(1) {
      }
    }
  }
}
```

Gate-Level DFT Insertion

After synthesis of the DFT-inserted design, you can add scan modes to the EDT controllers, analyze the scan chains, insert the test logic, and write the graybox view of the design into the *TSDB*. Before creating the scan modes, specify the wrapper analysis options and perform wrapper analysis. Use boundary scan as isolation between the package pins and the dies to avoid using shared wrapper cells. For at-speed ATPG test between dies, use the shared isolation cells instead of the boundary scan cells as wrapper cells. The following example shows ports that connect to the package boundary should be excluded:

```
set ports_accessible_from_package_boundary [get_ports g1_*]
set_wrapper_analysis_options \
  -exclude_ports $ports_accessible_from_package_boundary
set_dedicated_wrapper_cell_options off -ports {list of ports} #optional
analyze_wrapper_cells
```

Add the int_edt_mode and insert the test logic:

```
# Create a scan mode and specify EDT instance to connect the
# scan chains to
set int_edt_instance [get_name_list [get_instance -of_module \
  [get_single_name [get_icl_module *controller_int* -of_instances \
  satellite_die* -filter  tessent_instrument_type==mentor::edt]]]]
add_scan_mode int_edt_mode -edt_instance $int_edt_instance
```

Add the ext_edt_mode scan mode, including the OCCs, wrapper cells, and boundary scan cells. Review and analyze the scan chains and modes and insert the test logic. The highlighted command in the following is specific to multi-die scan stitching for package-level wrapper chains:

```
set ext_elements [add_to_collection \
  [get_scan_elements *occ*] [get_scan_elements -class wrapper]]
set ext_edt_instance [get_name_list [get_instance -of_module \
  [get_single_name [get_icl_module *controller_ext* -of_instances
  satellite_die* -filter  tessent_instrument_type==mentor::edt]]]]
add_scan_mode ext_edt_mode -edt_instance $ext_edt_instance \
  -include_elements $ext_elements
analyze_scan_chains
# Insert scan chains
insert_test_logic
```

Write the scan graybox view of the design after test logic insertion. The graybox view is the design netlist with only the necessary scan-test-related logic (primary inputs and outputs, wrapper chains, and the glue logic outside the wrapper chains).

```
# Create the ijtag graybox for the scan inserted netlist
set_context patterns -scan
import_scan_mode ext_edt_mode
check_design_rules
analyze_graybox -scan
write_design -tsdb -graybox
```

Write the ijtag_graybox view of the design after test logic insertion. The ijtag_graybox view is the design netlist with only the necessary IJTAG-related logic.

```
# Create the ijtag graybox for the scan inserted netlist
set_context patterns -ijtag
set_system_mode analysis
analyze_graybox -ijtag
# Write the graybox view of the scan inserted design into TSDB directory
write_design -tsdb -graybox
```

Tip
Using the graybox view improves the performance of ATPG simulations.

Primary Die-Level DFT Flow

This section describes the DFT flow for a primary die in a 2.5D design, including BoundaryScan, MBIST, OCC, EDT, and SSN DFT logic insertion. Additionally, it covers the preparation of a die for at-speed die-to-die interconnection tests.

First DFT Insertion Pass: Inserting Die-Level Boundary Scan, IJTAG, and MemoryBIST

During the first DFT insertion pass at the 2.5D primary die level, you insert BoundaryScan, IJTAG, and MemoryBIST.

BoundaryScan DFT Signals

The DFT signals for the primary die are the same as for the non-primary die. Refer to "BoundaryScan DFT Signals" on page 52.

Setting TAP Interface Port Attributes

To add the TDI muxing logic to the primary die, specify the attribute value for the TAP interface port:

```
set_attribute_value tdi_local -name function -value tdi_local
```

Set this value in the setup mode before calling the create_dft_specification command.

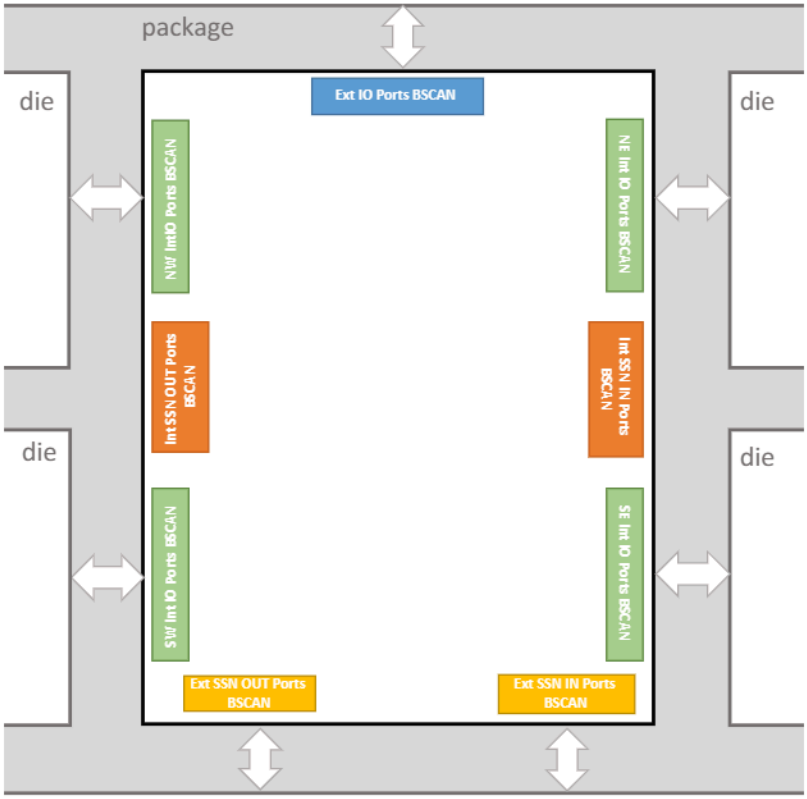
Pinorder and DEF File

The order of pins in the layout of the primary die is equally important as in the non-primary die case. Refer to "Pinorder and DEF File" on page 53.

LogicalGroups and BondingConfigurations

The LogicalGroup and BondingConfiguration wrappers should also enhance the boundary scan description in the DftSpecification. The following figure shows the center (primary) die ports assigned to nine logical groups:

Figure 24. Primary Die BScan Logical Groups



- **Blue**

  The external boundary scan group consists of the ports that connect to the package boundary after the 2.5D design integration.

- **Green**

  These four LogicalGroups comprise the functional ports interconnecting the center die with the four satellite dies. The tool considers these groups as internal because they interconnect the dies at the package level.

- **Orange**

  These two groups comprise the internal SSN bus input and output ports. The tool later uses these ports to interconnect the SSN bus between dies.

- **Yellow**

  This boundary-connected group of ports is the primary external SSN bus input and output ports.

The following DftSpecification defines the LogicalGroups:

```
LogicalGroups {
 LogicalGroup(external_ports_SSN_ins_bscan_group) {
  first_port : g4_SSN_primary_in_0;
 }
 LogicalGroup(internal_ports_SE_bscan_group) {
  first_port : SE_g2_to_clk;
 }
 LogicalGroup(internal_ports_SSN_ins_bscan_group) {
```

```
  first_port : g3_SSN_secondary_in_0;
 }
 LogicalGroup(internal_ports_NE_bscan_group) {
  first_port : NE_g2_to_clk;
 }
 LogicalGroup(external_ports_bscan_group) {
  first_port : g1_clk;
 }
 LogicalGroup(internal_ports_NW_bscan_group) {
  first_port : NW_g2_to_clk;
 }
 LogicalGroup(internal_ports_SSN_outs_bscan_group) {
  first_port : g3_SSN_secondary_out_0;
 }
 LogicalGroup(internal_ports_SW_bscan_group) {
  first_port : SW_g2_to_clk;
 }
 LogicalGroup(external_ports_SSN_outs_bscan_group) {
  first_port : g4_SSN_primary_out_0;
 }
}
```

The following DftSpecification defines two BondingConfigurations:

```
BondingConfigurations {
 BondingConfiguration(external_bonding_config) {
  enable_signal : DftSignal(external_bonding_config_en);
  active_logical_groups : external_ports_SSN_ins_bscan_group,
   external_ports_bscan_group, external_ports_SSN_outs_bscan_group;
 }
 BondingConfiguration(all_ports_bonding_config) {
 }
}
```

- **external_bonding_config**

  Integrates the 2.5D design at the package level. It contains the functional and SSN ports that connect to the boundary of the package.

- **all_ports_bonding_config**

  Standalone tests and MultiDieInterconnect patterns at the package level use this configuration.

Tip
In the primary die, the number of LogicalGroups increases significantly, so a descriptive naming convention can be helpful when creating the BondingConfigurations.

MemoryBIST Insertion

Refer to "MemoryBIST Insertion" on page 55.

Second DFT Insertion Pass: Inserting Die-Level EDT, OCC, and SSN

During the second DFT insertion pass at the die level, insert any EDT, OCC, and SSN elements your design requires.

Logic Test Instruments Insertion (EDT and OCC)

In the primary die, you must add clock muxes. The following commands add the clock muxes for two clock sources, clk and clk_mem:

```
add_dft_clock_mux SE_g2_to_clk_pad/I -test_clock_source g1_clk \
  -dft_signal_source_name int_ltest_en
add_dft_clock_mux NE_g2_to_clk_pad/I -test_clock_source g1_clk \
  -dft_signal_source_name int_ltest_en
add_dft_clock_mux NW_g2_to_clk_pad/I -test_clock_source g1_clk \
  -dft_signal_source_name int_ltest_en
add_dft_clock_mux SW_g2_to_clk_pad/I -test_clock_source g1_clk \
  -dft_signal_source_name int_ltest_en

add_dft_clock_mux SE_g2_to_clk_mem_pad/I -test_clock_source g1_clk_mem \
  -dft_signal_source_name int_ltest_en
add_dft_clock_mux NE_g2_to_clk_mem_pad/I -test_clock_source g1_clk_mem \
  -dft_signal_source_name int_ltest_en
add_dft_clock_mux NW_g2_to_clk_mem_pad/I -test_clock_source g1_clk_mem \
  -dft_signal_source_name int_ltest_en
add_dft_clock_mux SW_g2_to_clk_mem_pad/I -test_clock_source g1_clk_mem
  -dft_signal_source_name int_ltest_en
```

This adds clock muxes to the four output ports that deliver clocking to the four non-primary dies and the same for the clk_mem signal.

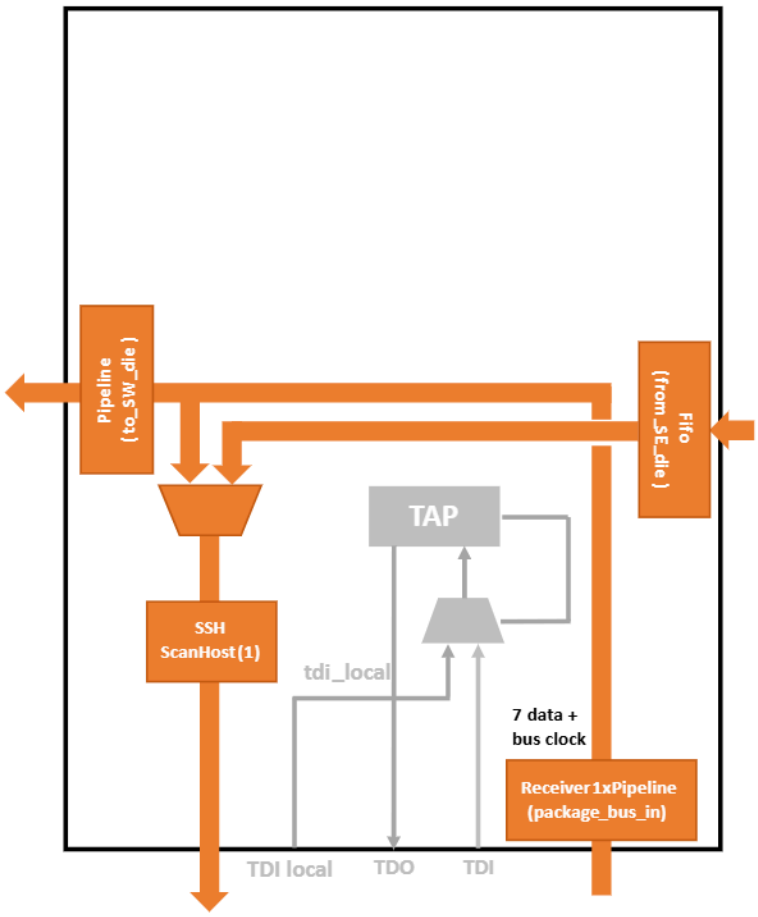
The add_dft_clock_mux command correctly connects to the specified -test_clock_source after the OCC intercepts the same specified clock source. The int_ltest_en DFT signal controls the multiplexer selects. The tool automatically sets this DFT signal high whenever an internal scan_mode is active, ensuring that the OCC bypass clock multiplexing is active during the internal test modes.

The remainder of the EDT and OCC insertion for the primary die is the same as for the non-primary dies.

SSN Insertion

The SSN interface of the primary die connects to the package boundary, and you can access the entire SSN infrastructure in the 2.5D package.

Figure 25. SSN Network in a Primary Die



ScanHost(1) is closest to the output and is always present on the SSN path, no matter the current state of the multiplexer (from_SE_die). This mux is responsible for opening the SSN path to include the SSN interface that sources the data from the last one of the non-primary dies in a chain.

A FIFO (from_SE_die) pipelines the secondary SSN input interface. This is required because the tool considers the SSN bus and clock from another die to be in a different clock domain after crossing all dies in the chain.

The ExtraOutputPathwrapper specifies the secondary SSN output interface with an output pipeline. The last Receiver1xPipeline (package_bus_in) is pipelining the input data.

The following is a DftSpecification representation of the SSN in a primary die:

```
read_config_data -in_wrapper $spec -from_string {
 SSN {
  ijtag_host_interface : Sib(ssn);
  DataPath(1) {
   output_bus_width :7;
   Connections {
    bus_clock_in    : g4_SSN_primary_bus_clock_in_7;
    bus_data_in     : g4_SSN_primary_in_%d;
    bus_data_out    : g4_SSN_primary_out_%d;
    bus_clock_out   : g4_SSN_primary_bus_clock_out_7;
```

```
    }
    ScanHost(1) {
    }
    Multiplexer(from_SE_die) {
     Connections {
      secondary_bus_data_in : g3_SSN_secondary_in_%d;
     }
     Fifo(from_SE_die) {
     frequency_ratio : 4;
     input_retimed                                  : on;
     input_retiming_cell_type                       : latch ;
     in_clock_to_out_clock_skew                     : delayed_only;
     in_clock_to_out_clock_skew_programmable : off;

      Connections {
       bus_in_clock_in    : g3_SSN_secondary_bus_clock_in_7;
       bus_out_clock_in   : g4_SSN_primary_bus_clock_in_7;
      }
     }
    }

    Receiver1xPipeline(package_bus_in) {
     ExtraOutputPath {
      Connections {
       bus_data_out   : g3_SSN_secondary_out_%d;
       bus_clock_out : g3_SSN_secondary_bus_clock_out_7;
      }
      Pipeline(to_SW_die) {
      }
     }
    }
   }
  }
 }
```

Gate-Level DFT Insertion

You can insert the scan modes in the same way as the non-primary die. Refer to "Gate-Level DFT Insertion" on page 57.

Die-Level ATPG Pattern Generation

This section describes how to generate stuck-at ATPG patterns for primary and non-primary dies and how to simulate them with testbench simulations.

After reading the design from the previous step, import the scan mode using the import_scan_mode command. Use the set_current_mode command to set a new name:

```
# Import the desired scan mode configuration
import_scan_mode int_edt_mode -fast_capture_mode off

# Remove all PI/PO except the SSN bus
set_current_mode int_edt_top_stuck
```

The internal mode ensures that the patterns use only the SSN bus inputs and outputs. This is required to achieve isolation from other dies and make the pattern retargetable. External scan mode tests the connections between the dies.

Drive the desired static DFT signals:

```
# Use BoundaryScan as isolation
set_static_dft_signal_values output_pad_disable 1
set_static_dft_signal_values bscan_input_isolation_enable 1

# Use a BondingConfiguration with all ports
set_static_dft_signal_values external_bonding_config_en 0
```

You must restore all port constraints at the package level during the retargeting. Use the boundary scan chains of other dies in IJTAG mode to restore the constraints on ports that connect to another die. The steps to generate and simulate internal mode ATPG patterns are identical to the normal hierarchical design.

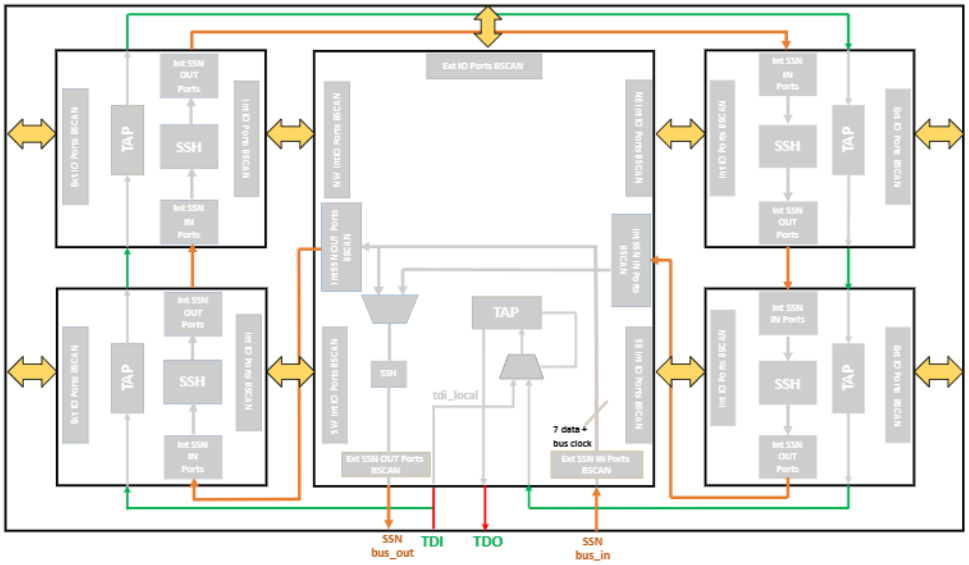
Create patterns for external mode ATPG in a similar way, then concatenate the results of the two modes to see the full coverage.

Note:
Do not retarget external mode patterns to the package level.

Package-Level Integration and Verification

This section describes the Tessent Shell flow for the package level. This part of the flow differs from the die-level flow, as there is no insertion of the DFT logic at the package level. The package-level flow focuses on package integration, interconnect pattern generation, retargeting the die patterns to the package level, and creating the at-speed die-to-die patterns.

Figure 26. 2.5D Interposer Connectivity



Note:
You are responsible for creating the wired connections between the dies and to the package boundary (marked in red, green, and orange).

You must instantiate the dies in a single package module and manually describe the package level. There is no automation in the HDL integration for the Tessent 2.5D flow, but you can use the Tessent introspection and insertion flows to integrate the package.

The obligatory connections at the package level shown in the previous figure are as follows:

- From a die to the package boundary.
- From one die to another die.
- The daisy-chained TAP connections between the dies.
- The daisy-chained SSN bus between the dies.

Extracting the Unified Package BSDL
Retargeting the ATPG Patterns
Generating the Die-to-Die Boundary Scan Patterns
Generating the SSN Patterns

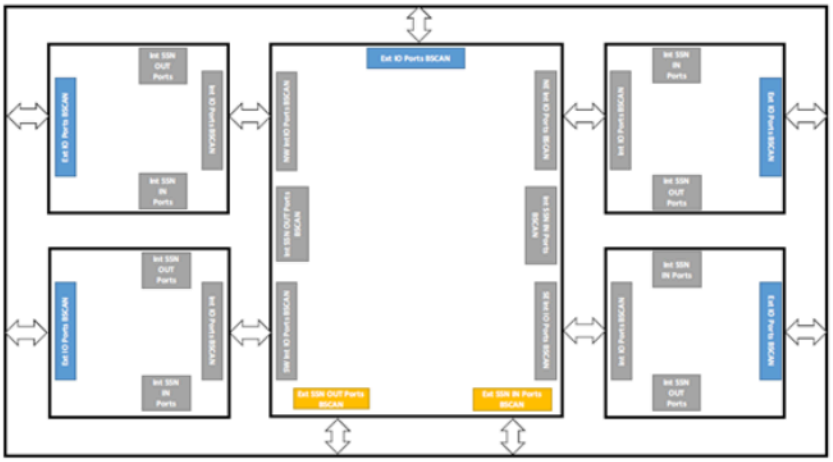
Extracting the Unified Package BSDL

BSDL extraction is required to generate the unified BSDL description for the package level.

Extract the BSDL by loading the interposer netlist to integrate the ICL and package BSDL descriptions and reading the interface view of all dies using the read_design command.

The following figure shows the external bonding configuration to the package pins:

Figure 27. External Bonding to Package Pins



Procedure

1. Read the design files:

   read_design satellite_die -design_id gate1 -view interface

   read_design center_die -design_id gate1 -view interface

   read_verilog ../data/design/chip.v

   set_current_design chip

2. Integrate the package by setting the current design level to chip. Next, create a TSDB container for the package level with the extract_icl command.

   set_design_level chip

   extract_icl -write_in_tsdb on

3. Read the die BSDL and extract the package BSDL:

   read_bsdl satellite_die_external_bscan_group.bsdl

   read_bsdl center_die_external_bscan_group.bsdl

   set_dft_specification_requirements -bsdl_extraction on

   check_design_rules

   The check_design_rules command triggers the package BSDL extraction:

```
// command: check_design_rules
// BSDL extraction from the BSDL descriptions of dies.
// Warning: Rule FN4 violation occurs 140 times
// Flattening process completed, cell instances=260, gates=3118,
 PIs=78, POs=69, CPU time=0.00 sec.
// ------------------------------------------------------------
// Begin circuit learning analyses.
// --------------------------------
// Learning completed, CPU time=0.00 sec.
// Extracted bsdl sequence:
// SW_satellite_die_inst
// NW_satellite_die_inst
// NE_satellite_die_inst
// SE_satellite_die_inst
// center_die_inst
// Extracted BSDL
 file: ./
tsdb_chip/dft_inserted_designs/chip_rtl1.dft_inserted_design/chip.b
sdl
```

Related Topics

read_bsdl [Tessent Shell Reference Manual]

delete_bsdl [Tessent Shell Reference Manual]

report_bsdl [Tessent Shell Reference Manual]

Retargeting the ATPG Patterns

You can retarget non-primary die ATPG patterns and primary die patterns simultaneously.

Prerequisites

- Generate die-level ATPG patterns.

Procedure

1. To retarget the die stuck-at ATPG patterns, add all die instances as the core instances:

   ```
   add_core_instances -instances [get_instances -of_module satellite_die] \
   -mode int_edt_top_stuck
   add_core_instances -instances [get_instances -of_module center_die] \
   -mode int_edt_top_stuck
   ```

   Tip
   Change the fault type to transition when calling the add_core_instances and read_patterns commands to retarget the transition type patterns.

2. Specify the clocks:

```
add_clocks CENTER_g1_clk -period 10ns
add_clocks CENTER_g1_clk_mem -period 10ns
```

3. Open the SSN multiplexer to include the satellite_dies into the active SSN path:

```
set_test_setup_icall
{center_die_inst.center_die_rtl2_tessent_ssn_mux_from_SE_die_inst.setup
select_secondary_bus 1} -append -end
```

4. Run Design Rule Checking to elaborate the design and call the test setup procedure:

```
check_design_rules
```

5. Read the existing satellite_die and center_die ATPG patterns and retarget them to the package level:

```
read_patterns -modules satellite_die -mode int_edt_top_stuck \
-fault_type stuck
read_patterns -modules center_die -mode int_edt_top_stuck \
-fault_type stuck
```

Note:
The mode is specified with the add_core_instances command.

6. Write the retargeted ATPG patterns:

```
write_patterns package_2_5D_retargeting_satellite_die_stuck_parallel.v \
  -verilog -replace -parameter_list {SIM_COMPARE_SUMMARY 1}

# Write SSH loopback testbench
write_patterns package_2_5D_retargeting_satellite_die_stuck_ssh_loopback.v \
  -verilog -serial -replace -parameter_list {SIM_COMPARE_SUMMARY 1} \
  -pattern_set ssh_loopback

# Use this option for signoff simulations to verify all chains with a single chain pattern
write_patterns package_2_5D_retargeting_satellite_die_stuck_serial_chain.v \
  -verilog -serial -replace -end 0 -parameter_list {SIM_COMPARE_SUMMARY 1} \
  -pattern_set chain
write_patterns package_2_5D_retargeting_satellite_die_stuck_serial_scan.v  \
  -verilog -serial -replace -end 0 -parameter_list {SIM_COMPARE_SUMMARY 1
ALL_EXCLUDE_UNUSED 0} \
  -pattern_set scan
```

7. Write the SSH loopback testbench:

```
write_patterns package_2_5D_retargeting_satellite_die_stuck_ssh_loopback.v \
  -verilog -serial -replace -parameter_list {SIM_COMPARE_SUMMARY 1} -pattern_set
ssh_loopback
```

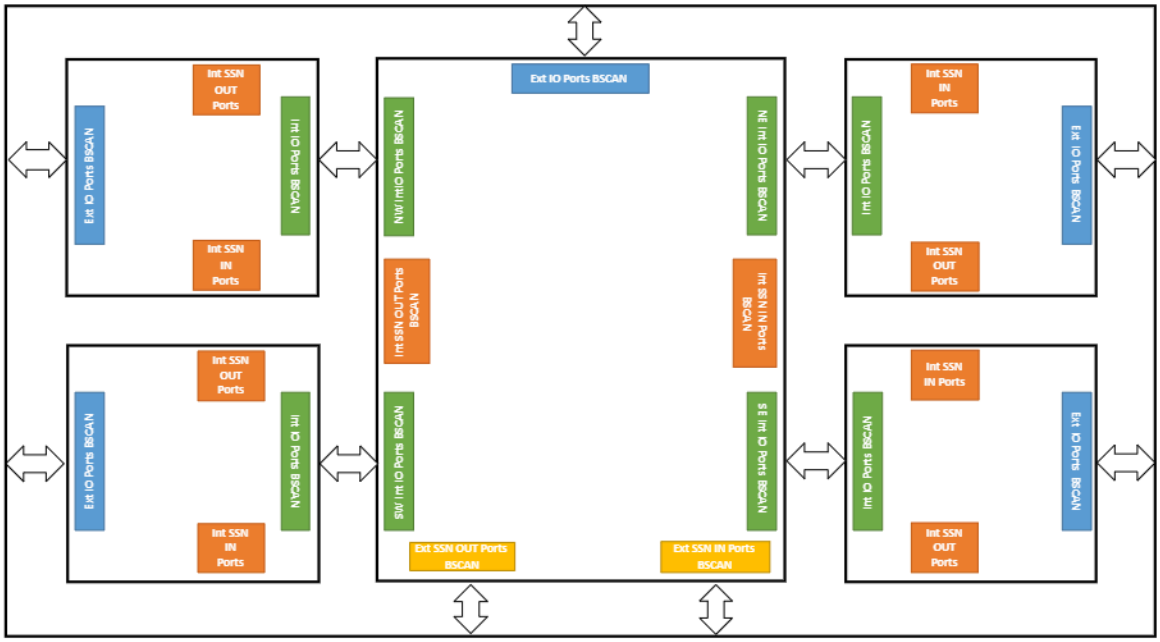
8. Write signoff simulations to verify all chains with a single chain pattern:

```
write_patterns package_2_5D_retargeting_satellite_die_stuck_serial_chain.v \
  -verilog -serial -replace -end 0 -parameter_list {SIM_COMPARE_SUMMARY 1} -pattern_set
chain
write_patterns package_2_5D_retargeting_satellite_die_stuck_serial_scan.v  \
  -verilog -serial -replace -end 0 -parameter_list {SIM_COMPARE_SUMMARY 1
ALL_EXCLUDE_UNUSED 0} -pattern_set scan
```

Generating the Die-to-Die Boundary Scan Patterns

After loading the BSDL files, use the configuration-based flow to generate test patterns. The Tessent Shell 2.5D methodology enables the package interconnectivity test.

Figure 28. 2.5D Package Design Example



Restrictions and Limitations

Use the all ports BondingConfiguration of the primary and non-primary dies.

Prerequisites

- A 2.5D design with BSDL data loaded using the read_bsdl command.

Procedure

1. To simulate the ICL verification patterns in all dies in the package, set the simulate_instruments_in_lower_physical_instances attribute to "on" as in the standard chip flow:

   ```
   set_defaults_value PatternsSpecification/SignOffOptions/
   simulate_instruments_in_lower_physical_instances on
   ```

2. Read the BSDL files with the all port BondingConfigurations:

```
read_bsdl ../satellite_die/tsdb_satellite_die/instruments/satellite_die_rtl_bscan.instrument/
satellite_die_all_ports_bonding_config.bsdl
read_bsdl ../center_die/tsdb_center_die/instruments/center_die_rtl_bscan.instrument/
center_die_all_ports_bonding_config.bsdl
set patt_spec [create_patterns_specification -replace]
```

3. Use the create_patterns_specification command to create a template specification from the BSDL data loaded into the design:

```
set patt_spec [create_patterns_specification -replace]

# Set the bonding enable ports to desired constant values for the DIE interconnect ports - all
ports bonding configuration
read_config_data -in_wrapper $patt_spec/Patterns(MultiDieInterconnect) \
  -from_string {
  DftControlSettings {
    SW_satellite_die_inst.external_bonding_config_en : 0 ;
    NW_satellite_die_inst.external_bonding_config_en : 0 ;
    NE_satellite_die_inst.external_bonding_config_en  : 0 ;
    SE_satellite_die_inst.external_bonding_config_en  : 0 ;
    center_die_inst.external_bonding_config_en          : 0 ;
  }
}
```

   Calling the create_pattern_specification command after reading the BSDL descriptions of the bonding configurations triggers the creation of the MultiDieInterconnect patterns.

Note:
It is necessary to explicitly assign values to the DFT signals driving bonding configurations in every boundary scan pattern, not only for the bonding configuration patterns.

   Modify the template specification as required.

4. Create the patterns with the process_patterns_specification command:

```
process_patterns_specification
```

   This results in the generation of the ICL verification patterns for each die in a package, the BScan patterns for the package, and the interconnect patterns for the bonding configuration.

5. Verify the patterns:

```
run_testbench_simulations
```

Examples

Pattern Generation Script

```
# This script generates BoundaryScan patterns for die_b

set_context patterns -ijtag -rtl -design_id rtl
read_design chip -view interface
set_current_design chip
```

```
# Check the design
check_design_rules

set spec [create_patterns_specification]

# Select all ports on die_a
read_config_data -in_wrapper $spec/Patterns(MultiDieInterconnect) \
  -from_string "
  AdvancedOptions {
    ConstantPortSettings {
      bypass_inter_die : 0;
    }
  }"

report_config_data $spec

# 2.5D interconnect pattern generation
process_patterns_specification

run_testbench_simulations
```

Patterns Specification

```
PatternsSpecification(package_top,rtl,signoff) {
  Patterns(ICLNetwork) {
    ICLNetworkVerify(package_top) {
    }
  }
  Patterns(MultiDieInterconnect) {
     SimulationOptions {
      LowerPhysicalBlockInstances {
        die_a_inst1 : full;
        die_a_inst2 : full;
        die_b_inst1 : full;
        die_b_inst2 : full;
      }
    }
    TestStep(MultiDieInterconnect) {
      MultiDieInterconnect {
        Die(die_a_inst1) {
          bsdl_file : instrument(die_a_rtl_bscan)/die_a_all_ports.bsdl;
        }
        Die(die_a_inst2) {
          bsdl_file : instrument(die_a_rtl_bscan)/die_a_all_ports.bsdl;
        }
        Die(die_b_inst1) {
          bsdl_file : instrument(die_b_rtl_bscan)/die_b.bsdl;
        }
        Die(die_b_inst2) {
          bsdl_file : instrument(die_b_rtl_bscan)/die_b.bsdl;
        }
        RunTest(inst_reg) {
        }
```

```
        RunTest(bypass_reg) {
        }
        RunTest(id_reg) {
        }
        RunTest(bscan_reg) {
        }
        RunTest(interconnect) {
        }
      }
    }
    AdvancedOptions {
      ConstantPortSettings {
        bypass_inter_die : 0;
      }
    }
  }
}
```

Related Topics

read_bsdl [Tessent Shell Reference Manual]

delete_bsdl [Tessent Shell Reference Manual]

report_bsdl [Tessent Shell Reference Manual]

Generating the SSN Patterns

The Tessent Shell tool automatically creates the SSN patterns for the default scan path. In this design, the default scan path tests only the primary die SSN Host.

Prerequisites

- A 2.5D design with BSDL data loaded using the read_bsdl command.

Procedure

1. To create patterns for the whole SSN network, add a ProcedureStep wrapper that calls a PDL file with a procedure that opens the SSN multiplexer to include all non-primary dies:

   ```
   iProcsForModule package_2_5D
   iProc int_ssn_ports {} {
     iCall center_die_inst.center_die_rtl2_tessent_ssn_mux_from_SE_die_inst.setup \
       select_secondary_bus 0b1
   }
   ```

   The preceding PDL file calls a procedure to reconfigure the SSN network.

2. Specify the SSNContinuityVerify wrapper to create the patterns for the network after opening the SSN multiplexer:

   ```
   source ../../workspace/package_2_5D/ssn_datapath_configuration.pdl
   read_config_data -last -in_wrapper $patt_spec/Patterns(SSN) -from_string {
     ProcedureStep(cfg_datapath) {
       iCall(int_ssn_ports) {
       }
   ```

```
    }
    SSNContinuityVerify(include_int_ssn_pads) {
    }
  }
```

Comprehensive Boundary Scan and IJTAG Example Flows

This section shows 2.5D packages with daisy-chained dies with the main die first and last in the chain.

2.5D Package With Daisy-Chained Dies (Main Die First)

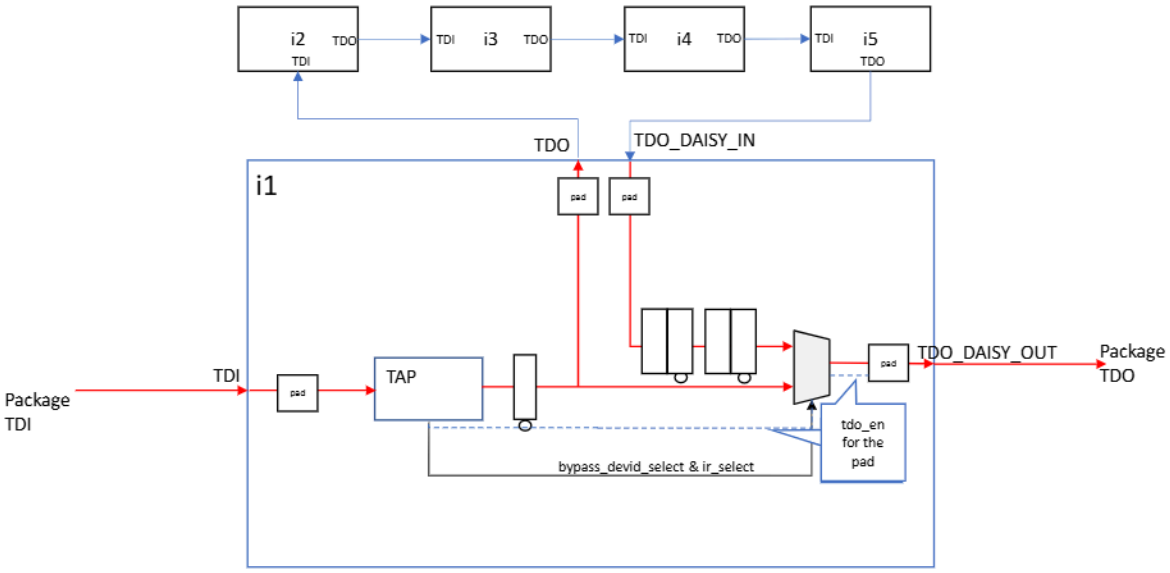
This section describes workflow examples with a daisy TDO multiplexer.

2.5D Example Workflow With a Fixed Daisy Chain

This workflow example shows how to deal with a TdoMux in a package. A TdoMux enables a second TDO for a 2.5D design, and is used when the TAP shifts through the BYPASS or DEVICE_ID register in the die closest to TDI in the package.

The following figure shows five dies in a single package. Instances i2, i3, i4, and i5 are instances of the same die, diec. Instance i1 is an instance of dieb. The boundary scan chain sequence is i1 → i2 → i3 → i4 → i5.

Figure 29. 2.5D Example With TdoMux



Note:
A 2.5D package is IEEE 1149.1 compliant with DftSpecification/IjtagNetwork/HostScanInterface/Tap/Interface/TdoMux/tdo_daisy_in and tdo_daisy_out ports and a TdoMux. The tool can generate a BSDL because the package has a one-bit BYPASS and a 32-bit DEVICE_ID register.

This workflow example demonstrates DFT insertion of TdoMux, package-level BSDL extraction, and pattern generation.

DFT Insertion Pass

Create the TAP and TdoMux and add the BoundaryScan cells in a DFT insertion pass.

Prerequisites

- Refer to the workflow example diagram in Figure 29.

Procedure

1. Set the context and read the design libraries and files:

   set_context dft -rtl

   # Read the libraries and the die design files
   …

2. Elaborate and specify the level of the design for all subsequent commands:

   set_current_design dieb
   set_design_level chip

3. Specify the tdo_daisy attributes to get the TdoMux in the DftSpecification:

   set_attribute_value -name function -value tdo_daisy_in tdo_daisy_in
   set_attribute_value -name function -value tdo_daisy_out tdo_daisy_out

   Note:
   If your TdoMux contains pipelines, you can specify additional or alternate pipeline attributes for the added pipelines with the add_die_logical_connections command. For example:

   ```
   add_die_logical_connections \
     -from tdo_daisy_in \
     -to tdo_daisy_out \
     -pipeline_stages 2 \
     -pipeline_capture_values 01 \
     -pipeline_safe_values 11
   ```

4. Specify the DftSpecification requirements:

   set_dft_specification_requirements -host_scan_interface_type tap -boundary_scan on

5. Run Design Rule Checking:

   check_design_rules

6. Create the DFT specification:

   set dftspec [create_dft_specification]

   Note:
   You can also specify additional DftSpecification settings in the TDO Mux using the DftSpecification/IjtagNetwork/HostScanInterface/Tap/Interface/TdoMux wrapper, such as pipelines.

   The TdoMux wrapper uses the default external_source for the connection_mode property.

7. Connect the IJTAG network interfaces:

process_dft_specification

8. Extract the ICL for the next DFT insertion pass:

   extract_icl

9. Create and verify the patterns for IJTAG and BoundaryScan:

   set_context patterns -ijtag -rtl

   check_design_rules

   set patspec [create_patterns_specification]

   Verify the generated patterns:

   report_config_data $patspec

   process_patterns_specification

   run_testbench_simulations

Package-Level BSDL Extraction

This section demonstrates how to extract package-level BSDL containing the die instances.

Prerequisites

- The DFT insertion has been completed. Refer to DFT Insertion Pass.
- Refer to the workflow example diagram in Figure 29.

Procedure

1. Set the context and read the design libraries and files:

   ```
   set_context patterns dft -rtl
   # Read the libraries and the design – full view of the package level
   …
   ```

2. Elaborate and specify the level of the design for all subsequent commands:

   ```
   set_current_design pack
   set_design_level chip
   ```

3. Extract ICL of the package:

   extract_icl

4. Extract the BSDL from the die BSDL:

   set_dft_specification_requirements -bsdl_extraction on

5. Read the BSDL files for the dies:

```
read_bsdl \
   tsdb_outdir/instruments/diec_rtl_bscan.instrument/diec.bsdl
read_bsdl \
   tsdb_outdir/instruments/dieb_rtl_bscan.instrument/dieb.bsdl
```

6. Run Design Rule Checking to extract the BSDL:

```
check_design_rules
```

Package-Level Pattern Generation

This section shows how to generate MultiDieInterconnect patterns and BoundaryScan patterns for the extracted BSDL file.

Prerequisites

- Package-level BSDL extraction has been completed. Refer to “Package-Level BSDL Extraction” on page 79.
- Refer to Figure 29 for design details.

Procedure

1. Set the context and read the design libraries and files:

```
set_context patterns -ijtag -rtl
# Read the libraries and the design – full view of the package level
…
```

2. Specify the level of the design for all subsequent commands:

```
set_current_design pack
set_design_level chip
```

3. Read the BSDL files for the dies:

```
read_bsdl \
   tsdb_outdir/instruments/diec_rtl_bscan.instrument/diec.bsdl
read_bsdl \
   tsdb_outdir/instruments/dieb_rtl_bscan.instrument/dieb.bsdl
```

4. Run Design Rule Checking:

```
check_design_rules
```

5. Create patterns for BoundaryScan and MultiDieInterconnect tests, and verify:

```
set spec [create_patterns_specification -bscan_sim_views ijtag_graybox]
report_config_data $spec

process_patterns_specification

run_testbench_simulations
```

2.5D Example With Satellite Die Bypasses

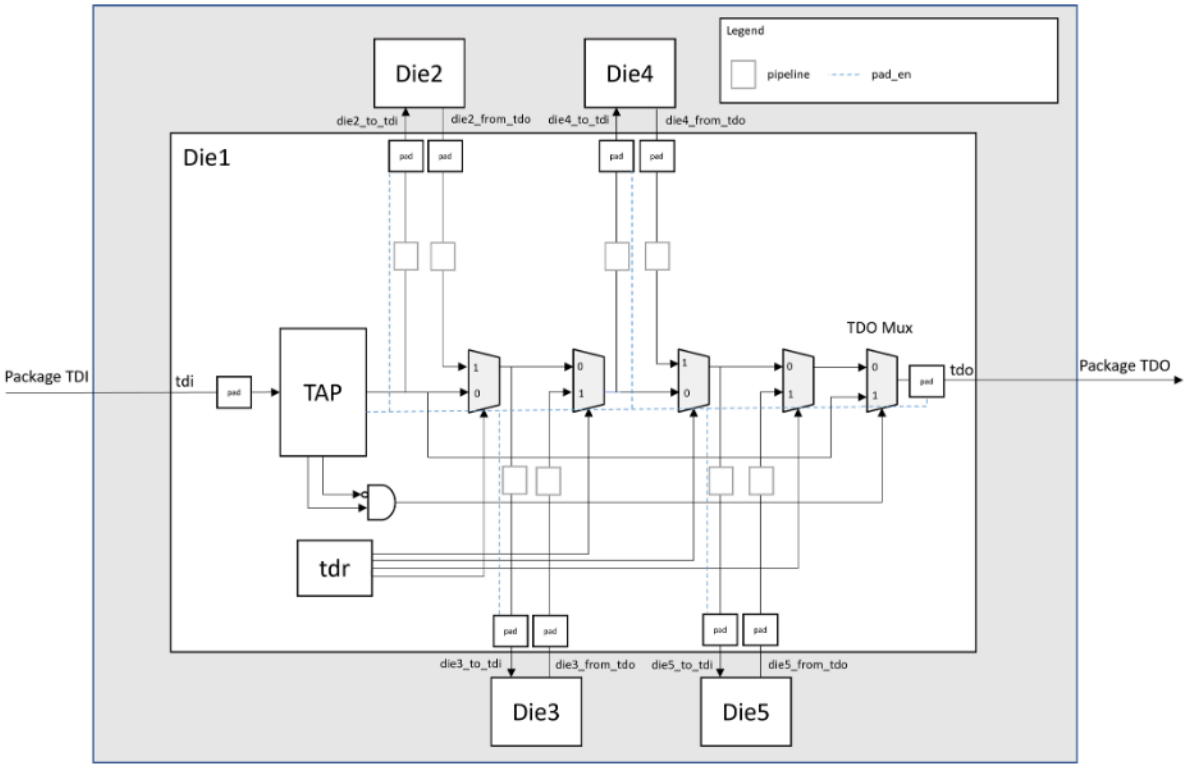
This workflow example does a DFT insertion pass to a design and extracts the BSDL file. Then, it performs package-level BSDL extraction and pattern generation.

The DFT insertion pass inserts the TAP, scan muxes, pipelines, and it includes a block with the boundary scan already inserted (see the DftSpecification/IjtagNetwork/HostScanInterface/Interface and DftSpecification/IjtagNetwork/Pipeline wrappers in the *Tessent Shell Reference Manual*). The DFT insertion also connects those existing boundary scan cells.

This example has five dies in a single package. Die2, Die3, Die4, and Die5 are four instances of the same die. The boundary scan chain sequence is configurable. In this example, all dies are active, and the boundary scan chain sequence is Die1 → Die2 → Die3 → Die4 → Die5. All TAP connections route through the center die.

The following figure shows a graphical representation of the workflow example:

Figure 30. 2.5D Example With Die-Level Logical Connections



Note:
This figure does not show the package-level TCK, TMS, and TRST connections.

They connect to Die1 and, from there, go through the secondary scan interfaces to dies 2, 3, 4, and 5. If you bypass a die, the toTMS port going through that die is switched off, and the TAP controller of that die stays in the Run-Test/Idle state.

DFT Insertion
Die-Level BSDL Extraction

DFT Insertion

The section describes the DFT insertion pass for a design where the block-level design BoundaryScan DFT has already completed.

Prerequisites

Restriction:
You must ensure that you do not define boundary scan cells for the die HostScanInterface ports, such as die2_to_tdi, die2_from_tdo, and others.

- The BoundaryScan DFT insertion for the block-level design has already been completed.
- Refer to the workflow example diagram in Figure 30.

Procedure

1. Set the context and read the design libraries and files:

```
set_context dft -rtl

# Read the libraries, and the central_die and block-level design files
…
```

2. Elaborate and specify the level of the design for all subsequent commands:

```
set_current_design central_die
set_design_level chip
```

3. Register the DFT signals:

```
set dft_signals {bp2 bp3 bp4 bp5}
register_static_dft_signal_names $dft_signals
add_dft_signals $dft_signals -create_with_tdr
```

4. Run Design Rule Checking:

```
check_design_rules
```

5. Create the DFT specification:

```
set spec [create_dft_specification]
```

6. Add the required signals and the TdoMux in "tdo_source_interception" connection_mode to the design:

```
read_config_data \
  -in_wrapper $spec/ljtagNetwork/HostScanInterface(tap)/Tap(main)/Interface \
  -from_string "
capture_ir_dr_en_present : on;
```

```
shift_ir_dr_en_present   : on;
TdoMux {
 connection_mode : tdo_source_interception;
}
"
```

Tip
Refer to the DftSpecification/IjtagNetwork/HostScanInterface/Tap/Interface/connection_mode property for more details.

7. Read the satellite die ScanMuxes into the design along with the pipelines:

```
read_config_data \
-in_wrapper $spec/IjtagNetwork/HostScanInterface(tap) \
-first \
-from_string "
ScanMux(die5) {
 select : DftSignal(bp5);
 Input(1) {
  Pipeline(from_die5) {
   length : 2;
// 01 value for the instruction register capture value to be compliant with IEEE 1149.1
   capture_value : 2'b01;
   si_retiming : on;
   Connections {
    shift_ir_dr_en    : Tap(main);
    from_tdo_en       : Tap(main);
   }
  }
  SecondaryHostScanInterface(die5) {
   to_tck            : die5_to_tck;
   to_tdi            : die5_to_tdi;
   from_tdo          : die5_from_tdo;
   from_tdo_en       : \"\";
   to_tms            : die5_to_tms;
   to_trst           : die5_to_trst;
  }
  Pipeline(to_die5) {
   Connections {
    shift_ir_dr_en    : Tap(main);
    from_tdo_en       : Tap(main);
   }
  }
 }
}
ScanMux(die4) {
 select : DftSignal(bp4);
 Input(1) {
  Pipeline(from_die4) {
   length : 2;
// 01 value for the instruction register capture value to be compliant with IEEE 1149.1
   capture_value : 2'b01;
   si_retiming : on;
   Connections {
    shift_ir_dr_en    : Tap(main);
    from_tdo_en       : Tap(main);
   }
```

```
    }
    SecondaryHostScanInterface(die4) {
      to_tck          : die4_to_tck;
      to_tdi          : die4_to_tdi;
      from_tdo        : die4_from_tdo;
      from_tdo_en     : \"\";
      to_tms          : die4_to_tms;
      to_trst         : die4_to_trst;
    }
    Pipeline(to_die4) {
      Connections {
        shift_ir_dr_en    : Tap(main);
        from_tdo_en       : Tap(main);
      }
    }
  }
}
ScanMux(die3) {
  select : DftSignal(bp3);
  Input(1) {
    Pipeline(from_die3) {
      length : 2;
// 01 value for the instruction register capture value to be compliant with IEEE 1149.1
      capture_value : 2'b01;
      si_retiming : on;
      Connections {
        shift_ir_dr_en    : Tap(main);
        from_tdo_en       : Tap(main);
      }
    }
    SecondaryHostScanInterface(die3) {
      to_tck          : die3_to_tck;
      to_tdi          : die3_to_tdi;
      from_tdo        : die3_from_tdo;
      from_tdo_en     : \"\";
      to_tms          : die3_to_tms;
      to_trst         : die3_to_trst;
    }
    Pipeline(to_die3) {
      Connections {
        shift_ir_dr_en    : Tap(main);
        from_tdo_en       : Tap(main);
      }
    }
  }
}
ScanMux(die2) {
  select : DftSignal(bp2);
  Input(1) {
    Pipeline(from_die2) {
      length : 2;
// 01 value for the instruction register capture value to be compliant with IEEE 1149.1
      capture_value : 2'b01;
      si_retiming : on;
      Connections {
        shift_ir_dr_en    : Tap(main);
        from_tdo_en       : Tap(main);
      }
    }
```

```
  SecondaryHostScanInterface(die2) {
    to_tck          : die2_to_tck;
    to_tdi          : die2_to_tdi;
    from_tdo        : die2_from_tdo;
    from_tdo_en     : \"\";
    to_tms          : die2_to_tms;
    to_trst         : die2_to_trst;
  }
  Pipeline(to_die2) {
    Connections {
      shift_ir_dr_en    : Tap(main);
      from_tdo_en       : Tap(main);
    }
  }
 }
}
"
```

8. Read the DftSpecification with the required BoundaryScan cells to be connected:

```
read_config_data -in_wrapper $spec -from_string "
 EmbeddedBoundaryScan {
  HostBScanInterface(block1) {
   Interface {
    ijtag_host_interface : Tap(main)/HostBScan;
   }
   EBScanInstance(block1_i) {
   }
  }
 }
"
```

9. Connect the TDI → TDO chains:

```
process_dft_specification
```

10. Extract the ICL description:

```
extract_icl
```

Die-Level BSDL Extraction

This section describes the die-level BSDL extraction of a design that has already completed the DFT insertion pass with BoundaryScan.

Prerequisites

- DFT insertion has been completed. Refer to DFT Insertion.
- Refer to the workflow example diagram in Figure 30.

Procedure

1. Set the context and read the design libraries and files:

   ```
   set_context dft -rtl -design_id bsdl_extraction

   # Read the libraries and the design – full view of the package level
   …
   ```

2. Extract the ICL description:

   ```
   extract_icl
   ```

3. Move the system mode to setup:

   ```
   set_system_mode setup
   ```

4. Synthesize the required blocks and emulate the loadboard loopbacks, if any.

5. Define the TDI/TDO/TCK/TMS/TRST attributes:

   ```
   set_attribute_value -name function -value tdi tdi
   set_attribute_value -name function -value tdo tdo
   set_attribute_value -name function -value tck tck
   set_attribute_value -name function -value tms tms
   set_attribute_value -name function -value trst trst
   ```

   Note:
   If the design has a TdoMux and ports with "tdo_daisy_in" and "tdo_daisy_out" are not defined, it automatically adds the TAP_SCAN_OUT_DAISY extension to the BSDL file.

6. Set the DftSpecification requirements and the bonding configuration for the BSDL:

   ```
   set_dft_specification_requirements \
     -bsdl_extraction on
   ```

7. Set the DFT signals to control the multiplexer configurations:

   ```
   # Set the DFT signals for the bypasses
   set dft_signals {bp2 bp3 bp4 bp5}
   set dft_signals_and_values {bp2 0 bp3 0 bp4 0 bp5 0}
   set_static_dft_signal_values $dft_signals_and_values
   set_bsdl_extraction_options -dft_signals_for_bonding_configurations $dft_signals
   ```

8. Extract the BSDL file:

   ```
   check_design_rules
   ```

9. Create patterns for IJTAG and BoundaryScan:

   ```
   set_context patterns -ijtag

   set spec [create_patterns_specification -separate_tap_and_bscan_patterns on]
   report_config_data $spec
   ```

```
process_patterns_specification
```

10. Verify the generated patterns:

```
run_testbench_simulations
```

Package-Level BSDL Extraction

Extract package-level BSDL containing the five die instances.

Prerequisites

- Die-level BSDL extraction pass has been completed. Refer to Die-Level BSDL Extraction.
- Refer to the workflow example diagram in Figure 30.

Procedure

1. Set the context and read the design libraries and files:

```
set_context dft -rtl

# Read the libraries and the  design – full view of the package level
…
```

2. Elaborate the design and ICL, and specify the level of the design for all subsequent commands:

```
set_current_design chip
set_design_level chip
extract_icl
```

3. Extract the BSDL from the die BSDL:

```
set_dft_specification_requirements -bsdl_extraction on
```

4. Read the BSDL files for the dies:

```
read_bsdl tsdb_outdir/dft_inserted_designs/
central_die_bsdl_extraction.dft_inserted_design/central_die.bsdl

read_bsdl tsdb_outdir/instruments/satellite_die_rtl_bscan.instrument/satellite_die.bsdl
```

5. Define the die configuration requirements:

```
# All satellite dies are active
set_bsdl_extraction_options -override_bonding_enable_values  {
  die1.bp2 1 die1.bp3 1 die1.bp4 1 die1.bp5 1
}
```

6. Add the logical connections:

```
# TCK, TMS, and TRST
foreach to_die { die2 die3 die4 die5 } {
   add_die_logical_connection -from die1/tms  -to die1/${to_die}_to_tms
```

```
    add_die_logical_connection -from die1/tck  -to die1/${to_die}_to_tck
    add_die_logical_connection -from die1/trst -to die1/${to_die}_to_trst
}

# Add the die logical connections
add_die_logical_connections \
  -from die1/die2_from_tdo \
  -to die1/die3_to_tdi \
  -pipeline_stages 3 \
  -pipeline_capture_values 01x
add_die_logical_connections
  -from die1/die3_from_tdo \
  -to die1/die4_to_tdi \
  -pipeline_stages 3 \
  -pipeline_capture_values 01x
add_die_logical_connections \
  -from die1/die4_from_tdo \
  -to die1/die5_to_tdi \
  -pipeline_stages 3 \
  -pipeline_capture_values 01x
add_die_logical_connections \
  -from die1/die5_from_tdo \
  -to die1/tdo -pipeline_stages 2 \
  -pipeline_capture_values 01
add_die_logical_connections \
  -from die1/tdo \
  -to die1/die2_to_tdi \
  -pipeline_stages 1

# report the added logical connections
report_die_logical_connections
```

7. Run Design Rule Checking to extract the BSDL:

```
check_design_rules
```

Package-Level Pattern Generation

Generate BoundaryScan patterns for the extracted BSDL file and MultiDieInterconnect patterns.

Prerequisites

- Package-level BSDL extraction has been completed. Refer to Package-Level BSDL Extraction.
- Refer to the workflow example diagram in Figure 30.

Procedure

1. Set the context and read the design libraries and files:

```
set_context dft -ijtag -rtl

# Read the libraries and the design, full view of the package level
…
```

2. Elaborate and specify the level of the design for all subsequent commands:

```
set_current_design chip
set_design_level chip
```

3. Read the BSDL files for the dies:

```
read_bsdl tsdb_outdir/dft_inserted_designs/
central_die_bsdl_extraction.dft_inserted_design/central_die.bsdl

read_bsdl tsdb_outdir/instruments/satellite_die_rtl_bscan.instrument/satellite_die.bsdl
```

4. Add the logical connections:

```
# TCK, TMS, and TRST
foreach to_die { die2 die3 die4 die5 } {
   add_die_logical_connection -from die1/tms  -to die1/${to_die}_to_tms
   add_die_logical_connection -from die1/tck  -to die1/${to_die}_to_tck
   add_die_logical_connection -from die1/trst -to die1/${to_die}_to_trst
}

# Add the die logical connections
add_die_logical_connections \
 -from die1/die2_from_tdo \
 -to die1/die3_to_tdi \
 -pipeline_stages 3 \
 -pipeline_capture_values 01x
add_die_logical_connections
 -from die1/die3_from_tdo \
 -to die1/die4_to_tdi \
 -pipeline_stages 3 \
 -pipeline_capture_values 01x
add_die_logical_connections \
 -from die1/die4_from_tdo \
 -to die1/die5_to_tdi \
 -pipeline_stages 3 \
 -pipeline_capture_values 01x
add_die_logical_connections \
 -from die1/die5_from_tdo \
 -to die1/tdo -pipeline_stages 2 \
 -pipeline_capture_values 01
add_die_logical_connections \
 -from die1/tdo \
 -to die1/die2_to_tdi \
 -pipeline_stages 1

# report the added logical connections
report_die_logical_connections
```

5. Run Design Rule Checking:

```
check_design_rules
```

6. Create patterns for BoundaryScan and MultiDieInterconnect tests:

```
set spec [create_patterns_specification]
report_config_data $spec
```

7. Delete the RunTest(test_logic_reset) test from the pattern specification and update the DftSignals for the multi-die interconnect flow:

```
foreach_in_collection pattern [get_config_elements -in_wrapper $spec
Patterns(JtagTap*Patterns)] {
  delete_config_element -in_wrapper $pattern TestStep(JtagBscanTestStep)/BoundaryScan/
RunTest(test_logic_reset)
}

delete_config_element -in_wrapper $spec Patterns(MultiDieInterconnect)/DftControlSettings

read_config_data -in_wrapper $spec/Patterns(MultiDieInterconnect) -from_string "
   DftControlSettings {
     die1.bp2 : 1;
     die1.bp3 : 1;
     die1.bp4 : 1;
     die1.bp5 : 1;
   }
"
```

8. Verify the generated patterns:

```
process_patterns_specification
run_testbench_simulations
```

2.5D Package With Daisy-Chained Dies (Main Die Last)

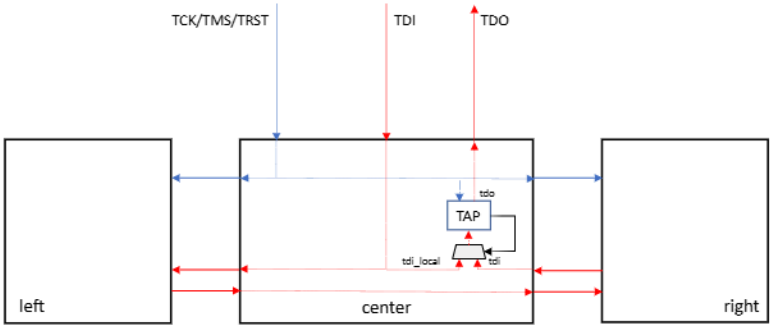
This section describes workflow examples with a local TDI mux.

2.5D Example Workflow With a Fixed Daisy Chain

This workflow example has a single DFT insertion pass, package-level BSDL extraction, and pattern generation. There are three dies in a single package, and the left die and the right die are two instances of the same die. The boundary scan chain sequence is left die -> right die -> center die, and all TAP connections route through the center die.

Figure 31 shows a graphical representation of the workflow example.

Figure 31. 2.5D Example



The example uses a DftSpecification with SecondaryHostScanInterface wrappers to automate the connections between the main TAP scan interface and the left and right scan interfaces.

The center die has the following ports:

- **Primary TAP Boundary Scan Interface** — tdi, tdo, tck, tms, and trst.
- **Left Secondary TAP Boundary Scan Interface** — left_tdi, left_tdo, left_tdo_en, left_tck, left_tms, and left_trst.
- **Right Secondary TAP Boundary Scan Interface** — right_tdi, right_tdo, right_tdo_en, right_tck, right_tms, and right_trst.

Single DFT Insertion Pass

Create the TAP controller, secondary TAP scan interfaces, tdi_local mux, and the boundary scan cells.

Prerequisites

- Refer to the workflow example diagram in Figure 31 on page 92.

Procedure

1. Set the context and read the design libraries and files:

   ```
   set_context dft -rtl

   # Read the libraries and the center_die design
   …
   ```

2. Specify the level of the design for all subsequent commands:

   ```
   set_current_design center_die
   set_design_level chip
   ```

3. Avoid adding BoundaryScan cells to the secondary scan interfaces:

   ```
   set_boundary_scan_port_options -cell_options dont_touch {left_* right_*}
   ```

4. Create a tdi_local bypass to bypass the satellite dies:

   ```
   set_attribute_value -name function tdi -value tdi_local
   ```

5. Specify the tdi as identical to the tdi_local for IJTAG:

   ```
   set_defaults_value DftSpecification/ijtagNetwork/HostScanInterface/Tap/tdi tdi
   ```

6. Request BoundaryScan insertion:

   ```
   set_dft_specification_requirements -boundary_scan on
   ```

7. Add the logical connections for TCK, TMS, and TRST connections going through the center die and for the center die:

   ```
   add_die_logical_connections -from tck -to left_tck
   add_die_logical_connections -from tck -to right_tck
   add_die_logical_connections -from tms -to left_tms
   add_die_logical_connections -from tms -to right_tms
   add_die_logical_connections -from trst -to left_trst
   add_die_logical_connections -from trst -to right_trst
   # TDI/TDO chain
   add_die_logical_connections -from tdi -to left_tdi
   add_die_logical_connections -from left_tdo -to right_tdi
   ```

   These logical connections appear in the BSDL file under the TESSENT_LOGICAL_CONNECTIONS extension.

8. Run Design Rule Checking:

   ```
   check_design_rules
   ```

9. Create the DftSpecification with TAP controller, TDI local mux, and boundary scan cells:

   ```
   set spec [create_dft_specification]
   ```

10. Add host scan interfaces for the left and right satellite dies:

```
read_config_data -after $spec/IjtagNetwork/HostScanInterface(tap)/Tap(main) -from_string "
SecondaryHostScanInterface(right) {
  to_tck          : right_tck;
  to_tdi          : right_tdi;
  from_tdo        : right_tdo;
  to_tms          : right_tms;
  to_trst         : right_trst;
  from_tdo_en     : \"\";
}
SecondaryHostScanInterface(left) {
  to_tck          : left_tck;
  to_tdi          : left_tdi;
  from_tdo        : left_tdo;
  to_tms          : left_tms;
  to_trst         : left_trst;
  from_tdo_en     : \"\";
}"
```

11. Add a scan interface for the BSDL file (right_tdo is the die-level TDI):

```
read_config_data -in_wrapper $spec/IjtagNetwork -from_string "
HostScanInterface(bscan) {
  Interface {
    tck       : tck;
    trst      : trst;
    tms       : tms;
    tdi       : right_tdo;
    tdi_local : tdi;
    tdo       : tdo;
  }
}"
set_config_value -in_wrapper $spec/BoundaryScan bsdl_tap_interface bscan
```

12. Create and insert the DFT hardware:

```
process_dft_specification
```

13. Create signoff patterns for IJTAG and BoundaryScan:

```
set_context patterns -ijtag
```

14. Extract the ICL description and check the design rules:

```
extract_icl
check_design_rules
```

15. Create and simulate the patterns:

```
set spec [create_patterns_specification]
process_patterns_specification
run_testbench_simulations
```

The DftSpecification for the center die after the single DFT insertion pass is as follows:

```
DftSpecification(center_die,rtl) {
 IjtagNetwork {
   HostScanInterface(tap) {
     Interface {
       tck       : tck;
       trst      : trst;
       tms       : tms;
       tdi       : tdi;
       tdi_local : tdi;
       tdo       : tdo;
     }
     Tap(main) {
       HostIjtag(1) {
       }
       HostBscan {
       }
     }
     SecondaryHostScanInterface(right) {
       to_tck      : right_tck;
       to_tdi      : right_tdi;
       from_tdo    : right_tdo;
       to_tms      : right_tms;
       to_trst     : right_trst;
       from_tdo_en : \"\";
     }
     SecondaryHostScanInterface(left) {
       to_tck      : left_tck;
       to_tdi      : left_tdi;
       from_tdo    : left_tdo;
       to_tms      : left_tms;
       to_trst     : left_trst;
       from_tdo_en : \"\";
     }
    }
    HostScanInterface(bscan) {
      Interface {
        tck       : tck;
        trst      : trst;
        tms       : tms;
        tdi       : right_tdo;
        tdi_local : tdi;
        tdo       : tdo;
      }
    }
  }
  BoundaryScan {
    ijtag_host_interface : Tap(main)/HostBscan;
    bsdl_tap_interface : bscan;
    BoundaryScanCellOptions {
      left_tck   : dont_touch;
      left_tdi   : dont_touch;
      left_tdo   : dont_touch;
```

```
        left_tms   : dont_touch;
        left_trst  : dont_touch;
        right_tck  : dont_touch;
        right_tdi  : dont_touch;
        right_tdo  : dont_touch;
        right_tms  : dont_touch;
        right_trst : dont_touch;
      }
    }
}
```

Note:
The tdi_local mux is outside the TAP controller.

Package-Level BSDL Extraction

Extract package-level BSDL containing the three die instances.

Prerequisites

- The single-pass DFT insertion has been completed. Refer to "Single DFT Insertion Pass" on page 92.
- Refer to the workflow example diagram in Figure 31 on page 92.

Procedure

1. Set the context and read the design libraries and files:

   set_context dft -rtl

   # Read the libraries and the design – full view of the package level
   …

2. Elaborate and specify the level of the design for all subsequent commands:

   set_current_design pack
   set_design_level chip

3. Extract the BSDL from the die BSDL:

   set_dft_specification_requirements -bsdl_extraction on

Note:
You may extract the BSDL, excluding one or more of the dies.

For example, to extract the BSDL for just the center and left satellite dies, you must read the correct BSDL files, assign them to the correct instance, and specify the correct die logical connections.

4. Read the BSDL files for the dies:

```
read_bsdl \
   tsdb_outdir/instruments/center_die_rtl_bscan.instrument/center_die.bsdl
read_bsdl \
   tsdb_outdir/instruments/satellite_die_rtl_bscan.instrument/satellite_die.bsdl
```

Note:
The tool loads logical connections for the bonding configuration directly from the BSDL TESSENT_LOGICAL_CONNECTIONS extension. If you need new logical connections, define them with the add_die_logical_connections command.

5. Run Design Rule Checking to extract the BSDL:

```
check_design_rules
```

Package-Level Pattern Generation

Generate BoundaryScan patterns for the extracted BSDL file and MultiDieInterconnect patterns.

Prerequisites

- Package-level BSDL extraction has been completed. Refer to "Package-Level BSDL Extraction" on page 96.
- Refer to the workflow example diagram in Figure 31 on page 92.

Procedure

1. Set the context and read the design libraries and files:

```
set_context patterns -ijtag -rtl

# Read the libraries and the design – full view of the package level
…
```

2. Specify the level of the design for all subsequent commands:

```
set_current_design pack
set_design_level chip
```

3. Read the BSDL files for the dies to trigger the MultiDieInterconnect pattern generation:

```
read_bsdl \
   tsdb_outdir/instruments/center_die_rtl_bscan.instrument/center_die.bsdl
read_bsdl \
   tsdb_outdir/instruments/satellite_die_rtl_bscan.instrument/satellite_die.bsdl
```

Note:
The tool loads logical connections for the bonding configuration directly from the BSDL TESSENT_LOGICAL_CONNECTIONS extension. If you need new logical connections, define them with the add_die_logical_connections command.

4. Run Design Rule Checking:

check_design_rules

5. Create patterns for BoundaryScan and MultiDieInterconnect tests, and verify:

   set spec [create_patterns_specification -bscan_sim_views ijtag_graybox]

   report_config_data $spec

   process_patterns_specification

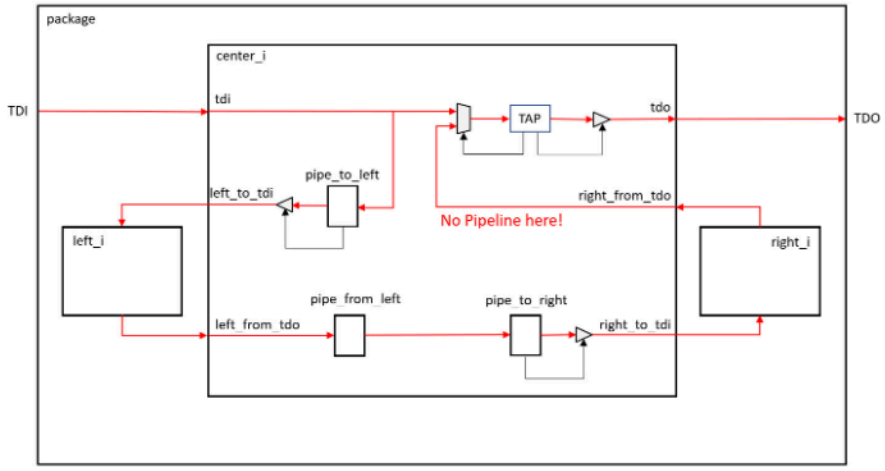
   run_testbench_simulations

Using Pipeline Stages

You may need to use pipeline stages instead of wire connections in your design to meet timing requirements. This section shows an example of how to create pipeline stages in a design during DFT.

The following figure shows a package that consists of a center die and two satellite dies.

Figure 32. Example Design With Pipeline Stages



Note:
The tck, tms, and trst connections to the satellite dies are routed through the center die, but these are not shown in the figure.

Some notes on the preceding design:

- The satellite dies, left_i and right_i, have standard chip-level BoundaryScan DFT hardware with a TAP controller.
- The center_i die has a local TDI mux and three pipeline stages to connect the satellite dies through the center die.
- The center die's die-level BoundaryScan test uses "right_from_tdo" as TAP_SCAN_IN and "tdi" as TAP_SCAN_IN_LOCAL.

Restriction:
It is not possible to add pipeline stages between "tdi" and the TDI mux, between the TDI mux and the TAP controller, or between the TAP controller and "tdo." The IEEE 1149.1 standard requires the BYPASS Shift-DR path to be 1 bit and the DEVICE_ID Shift-DR path to be 32 bits, which would be violated with any pipeline stages on these paths.

Furthermore, you cannot add a pipeline stage between the "right_from_tdo" and the TDI mux. This restriction will be removed in a future release.

The Center Die

This section describes the TAP controller, local TDI mux, and BoundaryScan DFT creation and insertion for the center die.

Tip
The IEEE 1149.1 standard mandates that TDO provide data on the negative edge of TCK. To ease timing closure, you can use "Pipeline/si_retiming=on" on the from_tdo pipeline.

The following is the center die RTL before DFT insertion:

```
module center(
  input wire  tck, tms, trst, tdi,
  output wire tdo,
  output wire left_to_tck, left_to_tms, left_to_trst, left_to_tdi,
  input wire  left_from_tdo,
  output wire right_to_tck, right_to_tms, right_to_trst, right_to_tdi,
  input wire  right_from_tdo,
  ...
);
  INPAD      jtag_pad1 (.IO(tck), .FP());
  INPAD      jtag_pad2 (.IO(tms), .FP());
  INPAD      jtag_pad3 (.IO(trst), .FP());
  INPAD      jtag_pad4 (.IO(tdi), .FP());
  OUTPAD_EN1 jtag_pad5 (.IO(tdo), .TP(), .EN1());

  OUTPAD_EN1 left_pad1 (.IO(left_to_tck), .TP(), .EN1(1'b1));
  OUTPAD_EN1 left_pad2 (.IO(left_to_tms), .TP(), .EN1(1'b1));
  OUTPAD_EN1 left_pad3 (.IO(left_to_trst), .TP(), .EN1(1'b1));
  OUTPAD_EN1 left_pad4 (.IO(left_to_tdi), .TP(), .EN1(1'b1));
  INPAD      lef_padt5 (.IO(left_from_tdo), .FP());

  OUTPAD_EN1 right_pad1(.IO(right_to_tck), .TP(), .EN1(1'b1));
  OUTPAD_EN1 right_pad2(.IO(right_to_tms), .TP(), .EN1(1'b1));
  OUTPAD_EN1 right_pad3(.IO(right_to_trst), .TP(), .EN1(1'b1));
  OUTPAD_EN1 right_pad4(.IO(right_to_tdi), .TP(), .EN1(1'b1));
  INPAD      right_pad5(.IO(right_from_tdo), .FP());
  ...
endmodule
```

The following example dofile performs DFT insertion of the TAP controller, the TDI mux, the pipeline stages, and BoundaryScan hardware in one Tessent Shell run:

```
# DFT insertion
set_context dft -rtl

# Read the libraries
...

# Read the center_die files
...

set_current_design center
set_design_level chip

set_dft_specification_requirements -boundary_scan on

# No BoundaryScan cells and no HIGHZ control of the SecondaryHostScanInterface pads
set_boundary_scan_port_options -cell_options dont_touch {left_* right_*}

check_design_rulesset

spec [create_dft_specification]

# Define the TDI local mux
set_config_value -in_wrapper $spec/IjtagNetwork/HostScanInterface(tap)/Interface tdi_local tdi
add_config_element -in_wrapper \
  $spec/IjtagNetwork/HostScanInterface(tap)/Tap(main)/Interface TdiMux

# For the pipelines
set_config_value -in_wrapper \
  $spec/IjtagNetwork/HostScanInterface(tap)/Tap(main)/Interface shift_ir_dr_en_present on

# Define the chain
read_config_data -after $spec/IjtagNetwork/HostScanInterface(tap)/Tap(main) -from_string "
SecondaryHostScanInterface(right) {
  to_tck              : right_to_tck;
  to_tdi              : right_to_tdi;
  from_tdo          : right_from_tdo;
  from_tdo_en     : \"\";
  to_tms              : right_to_tms;
  to_trst             : right_to_trst;
}
Pipeline(to_right) {
  Connections {
    shift_ir_dr_en  : Tap(main);
    from_tdo_en     : Tap(main);
  }
}
Pipeline(from_left) {
  si_retiming : on;
  Connections {
    shift_ir_dr_en  : Tap(main);
    from_tdo_en     : Tap(main);
  }
}
SecondaryHostScanInterface(left) {
  to_tck              : left_to_tck;
  to_tdi              : left_to_tdi;
  from_tdo          : left_from_tdo;
  from_tdo_en     : \"\";
```

```
  to_tms            : left_to_tms;
  to_trst           : left_to_trst;
 }
 Pipeline(to_left) {
  Connections {
   shift_ir_dr_en  : Tap(main);
   from_tdo_en     : Tap(main);
  }
 }"

 # Define different TAP ports for the BSDL
 read_config_data -in_wrapper $spec/IjtagNetwork -from_string "
 HostScanInterface(bscan) {
  Interface {
   tck         : tck;
   trst        : trst;
   tms         : tms;
   tdi         : right_from_tdo;
   tdi_local   : tdi;
   tdo         : tdo;
  }
 }"
 set_config_value -in_wrapper $spec/BoundaryScan bsdl_tap_interface bscan

 report_config_data $spec

 process_dft_specification

 set_system_mode setup

 extract_icl

 # Signoff patterns
 set_context patterns -ijtag

 check_design_rules

 set spec [create_patterns_specification]

 process_patterns_specification

 run_testbench_simulations
```

The following is the DftSpecification from the DFT insertion:

```
DftSpecification(center,rtl) {
  IjtagNetwork {
    HostScanInterface(tap) {
      Interface {
        tck       : tck;
        trst      : trst;
        tms       : tms;
        tdi       : tdi;
        tdi_local : tdi;
        tdo       : tdo;
      }
      Tap(main) {
        Interface {
```

```
        shift_ir_dr_en_present : on;
        TdiMux {
        }
      }
      HostIjtag(1) {
      }
      HostBscan {
      }
    }
    SecondaryHostScanInterface(right) {
      to_tck      : right_to_tck;
      to_tdi      : right_to_tdi;
      from_tdo    : right_from_tdo;
      from_tdo_en : \"\";
      to_tms      : right_to_tms;
      to_trst     : right_to_trst;
    }
    Pipeline(to_right) {
      Connections {
        shift_ir_dr_en : Tap(main);
        from_tdo_en : Tap(main);
      }
    }
    Pipeline(from_left) {
      si_retiming : on;
      Connections {
        shift_ir_dr_en : Tap(main);
        from_tdo_en : Tap(main);
      }
    }
    SecondaryHostScanInterface(left) {
      to_tck      : left_to_tck;
      to_tdi      : left_to_tdi;
      from_tdo    : left_from_tdo;
      from_tdo_en : \"\";
      to_tms      : left_to_tms;
      to_trst     : left_to_trst;
    }
    Pipeline(to_left) {
      Connections {
        shift_ir_dr_en : Tap(main);
        from_tdo_en : Tap(main);
      }
    }
  }
  HostScanInterface(bscan) {
    Interface {
      tck       : tck;
      trst      : trst;
      tms       : tms;
      tdi       : right_from_tdo;
      tdi_local : tdi;
      tdo       : tdo;
```

```
      }
    }
  }
  BoundaryScan {
    ijtag_host_interface : Tap(main)/HostBscan;
    bsdl_tap_interface : bscan;
    BoundaryScanCellOptions {
      left_from_tdo  : dont_touch;
      left_to_tck    : dont_touch;
      left_to_tdi    : dont_touch;
      left_to_tms    : dont_touch;
      left_to_trst   : dont_touch;
      right_from_tdo : dont_touch;
      right_to_tck   : dont_touch;
      right_to_tdi   : dont_touch;
      right_to_tms   : dont_touch;
      right_to_trst  : dont_touch;
    }
  }
}
```

Package Level

The following is the package-level RTL before DFT insertion:

```
module package_top(
  input wire  tck, tms, trst, tdi,
  output wire tdo,
...
);
  wire left_to_tck, left_to_tms, left_to_trst, left_to_tdi, left_from_tdo;
  wire right_to_tck, right_to_tms, right_to_trst, right_to_tdi,
 right_from_tdo;

  center center_i(
    .tck(tck), .tms(tms), .trst(trst), .tdi(tdi), .tdo(tdo),
    .left_to_tck(left_to_tck), .left_to_tms(left_to_tms),
    .left_to_trst(left_to_trst), .left_to_tdi(left_to_tdi),
    .left_from_tdo(left_from_tdo), .right_to_tck(right_to_tck),
    .right_to_tms(right_to_tms), .right_to_trst(right_to_trst),
    .right_to_tdi(right_to_tdi), .right_from_tdo(right_from_tdo),
    ...
  );

  satellite left_i(
    .tck(left_to_tck), .tms(left_to_tms), .trst(left_to_trst),
    .tdi(left_to_tdi), .tdo(left_from_tdo),
    ...
  );

  satellite right_i(
    .tck(right_to_tck), .tms(right_to_tms), .trst(right_to_trst),
```

```
    .tdi(right_to_tdi), .tdo(right_from_tdo),
    ...
  );
endmodule
```

The following dofile uses the add_die_logical_connections command to include the pipeline stages for package-level boundary scan:

```
set_context dft -rtl

# Read the libraries
...

# Read the package_top files
...

set_current_design package_top
set_design_level chip

extract_icl -write_in_tsdb on

# BSDL extraction
set_dft_specification_requirements -bsdl_extraction on

read_bsdl tsdb_outdir/instruments/center_rtl_bscan.instrument/center.bsdl
read_bsdl tsdb_outdir/instruments/satellite_rtl_bscan.instrument/satellite.bsdl

# chain connections through the center die
add_die_logical_connections -from \
  center_i/tdi -to center_i/left_to_tdi -pipeline_stages 1
add_die_logical_connections -from \
  center_i/left_from_tdo -to center_i/right_to_tdi -pipeline_stages 2

# control connections
add_die_logical_connections -from center_i/tck -to center_i/left_to_tck
add_die_logical_connections -from center_i/tms -to center_i/left_to_tms
add_die_logical_connections -from center_i/trst -to center_i/left_to_trst
add_die_logical_connections -from center_i/tck -to center_i/right_to_tck
add_die_logical_connections -from center_i/tms -to center_i/right_to_tms
add_die_logical_connections -from center_i/trst -to center_i/right_to_trst

check_design_rules

set_system_mode setup

# signoff pattern generation
set_context patterns -ijtag

check_design_rules

set spec [create_patterns_specification]

report_config_data $spec

process_patterns_specification

run_testbench_simulations
```

2.5D Example With Satellite Die Bypasses

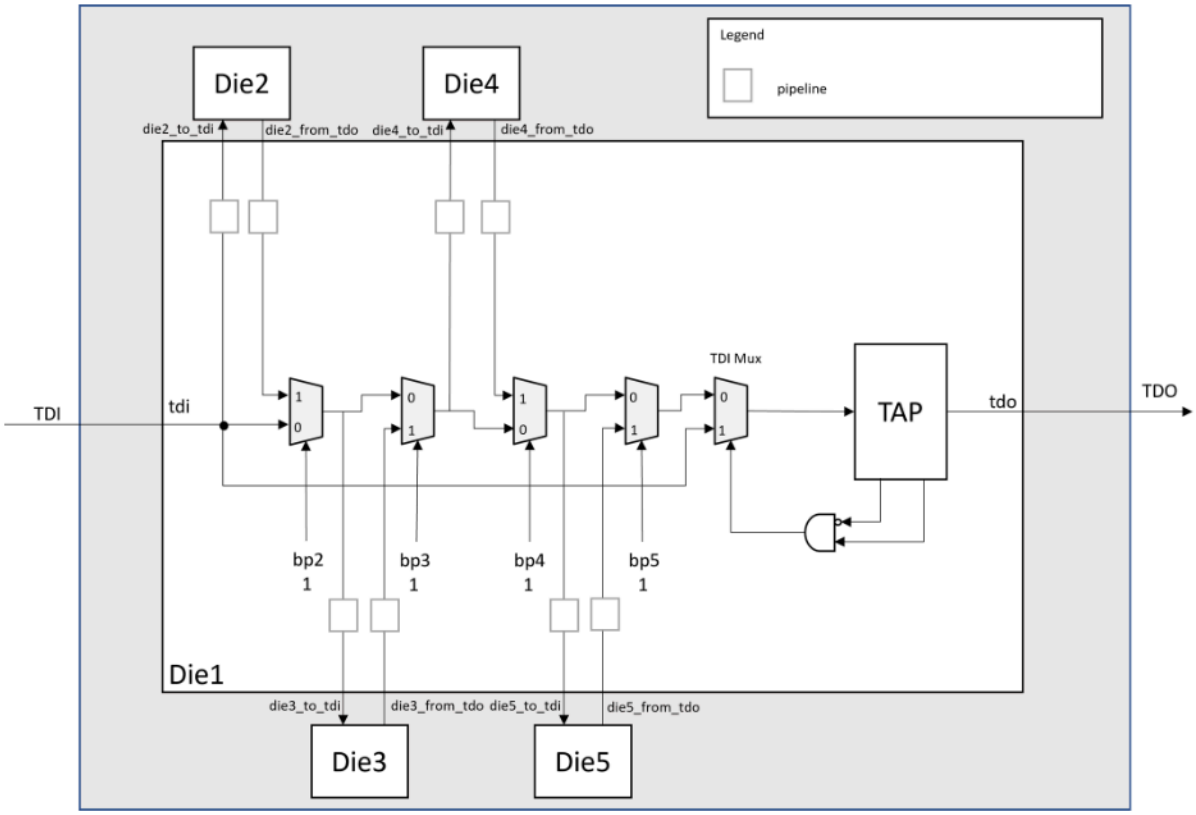
This workflow example does a DFT insertion pass to a design and extracts the BSDL file. Then, it performs package-level BSDL extraction and pattern generation.

The DFT insertion pass inserts the TAP, scan muxes, pipelines, and it includes a block with the boundary scan already inserted (see the DftSpecification/IjtagNetwork/HostScanInterface/Interface and DftSpecification/IjtagNetwork/Pipeline wrappers in the *Tessent Shell Reference Manual*). The DFT insertion also connects those existing boundary scan cells.

This example has five dies in a single package. Die2, Die3, Die4, and Die5 are four instances of the same die. The boundary scan chain sequence is configurable. In this example, all dies are active, and the boundary scan chain sequence is Die2 → Die3 → Die4 → Die5 → Die1. All TAP connections route through the center die.

The following figure shows a graphical representation of the workflow example:

Figure 33. 2.5D Example With Die-Level Logical Connections



Note:
This figure does not show the package-level TCK, TMS, and TRST connections.

They connect to die1 and, from there, go through the secondary scan interfaces to dies 2, 3, 4, and 5. If you bypass a die, the toTMS port going through that die is switched off, and the TAP controller of that die stays in the RunTestIdle state.

DFT Insertion

DFT Insertion

The section describes the DFT insertion pass on a design with block-level design BoundaryScan DFT already completed.

Prerequisites

Restriction:
You must ensure that you do not define boundary scan cells for the die HostScanInterface ports, such as die2_to_tdi, die2_from_tdo, and others.

- The BoundaryScan DFT insertion for the block-level design has already been completed.
- Refer to the workflow example diagram in Figure 33.

Procedure

1. Set the context and read the design libraries and files:

```
set_context dft -rtl

# Read the libraries and the central_die and block level design files
...
```

2. Elaborate and specify the level of the design for all subsequent commands:

```
set_current_design central_die
set_design_level chip
```

3. Specify the defaults for your design:

```
set_defaults_value DftSpecification/IjtagNetwork/HostScanInterface/Tap/tdi tdi
set_defaults_value DftSpecification/IjtagNetwork/HostScanInterface/Tap/tdi_local tdi
```

4. Register the DFT signals:

```
set dft_signals {bp2 bp3 bp4 bp5}
register_static_dft_signal_names $dft_signals
add_dft_signals $dft_signals -create_with_tdr
```

5. Run Design Rule Checking:

```
check_design_rules
```

6. Create the DFT specification:

```
set spec [create_dft_specification]
```

7. Read the TDI mux into the design:

```
read_config_data -in_wrapper $spec/ljtagNetwork/HostScanInterface(tap)/Tap(main)/
Interface -from_string "
 shift_ir_dr_en_present : on;
 TdiMux {
 }
"
```

8. Read the satellite die ScanMuxes into the design along with the pipelines:

```
read_config_data \
-in_wrapper $spec/ljtagNetwork/HostScanInterface(tap) \
-from_string "
ScanMux(die5) {
 select : DftSignal(bp5);
 Input(1) {
  Pipeline(from_die5) {
   si_retiming       : on;
   Connections {
    shift_ir_dr_en   : Tap(main);
    from_tdo_en      : Tap(main);
   }
  }
  SecondaryHostScanInterface(die5) {
   to_tck           : die5_to_tck;
   to_tdi           : die5_to_tdi;
   from_tdo         : die5_from_tdo;
   from_tdo_en      : \"\";
   to_tms           : die5_to_tms;
   to_trst          : die5_to_trst;
  }
  Pipeline(to_die5) {
   Connections {
    shift_ir_dr_en   : Tap(main);
    from_tdo_en      : Tap(main);
   }
  }
 }
}
ScanMux(die4) {
 select : DftSignal(bp4);
 Input(1) {
  Pipeline(from_die4) {
   si_retiming       : on;
   Connections {
    shift_ir_dr_en   : Tap(main);
    from_tdo_en      : Tap(main);
   }
  }
  SecondaryHostScanInterface(die4) {
   to_tck           : die4_to_tck;
   to_tdi           : die4_to_tdi;
   from_tdo         : die4_from_tdo;
   from_tdo_en      : \"\";
   to_tms           : die4_to_tms;
   to_trst          : die4_to_trst;
  }
```

```
    Pipeline(to_die4) {
     Connections {
      shift_ir_dr_en    : Tap(main);
      from_tdo_en       : Tap(main);
     }
    }
   }
  }
  ScanMux(die3) {
   select : DftSignal(bp3);
   Input(1) {
    Pipeline(from_die3) {
     si_retiming        : on;
     Connections {
      shift_ir_dr_en    : Tap(main);
      from_tdo_en       : Tap(main);
     }
    }
    SecondaryHostScanInterface(die3) {
     to_tck             : die3_to_tck;
     to_tdi             : die3_to_tdi;
     from_tdo           : die3_from_tdo;
     from_tdo_en        : \"\";
     to_tms             : die3_to_tms;
     to_trst            : die3_to_trst;
    }
    Pipeline(to_die3) {
     Connections {
      shift_ir_dr_en    : Tap(main);
      from_tdo_en       : Tap(main);
     }
    }
   }
  }
  ScanMux(die2) {
   select : DftSignal(bp2);
   Input(1) {
    Pipeline(from_die2) {
     si_retiming        : on;
     Connections {
      shift_ir_dr_en    : Tap(main);
      from_tdo_en       : Tap(main);
     }
    }
    SecondaryHostScanInterface(die2) {
     to_tck             : die2_to_tck;
     to_tdi             : die2_to_tdi;
     from_tdo           : die2_from_tdo;
     from_tdo_en        : \"\";
     to_tms             : die2_to_tms;
     to_trst            : die2_to_trst;
    }
    Pipeline(to_die2) {
     Connections {
      shift_ir_dr_en    : Tap(main);
      from_tdo_en       : Tap(main);
     }
    }
   }
```

```
}
"
```

9. Read the DftSpecification with the required BoundaryScan cells to be connected:

```
read_config_data -in_wrapper $spec -from_string "
 EmbeddedBoundaryScan {
  HostBScanInterface(block1) {
   Interface {
    ijtag_host_interface : Tap(main)/HostBScan;
   }
   EBScanInstance(block1_i) {
   }
  }
 }
"
```

10. Connect the TDI -> TDO chains:

```
process_dft_specification
```

11. Extract the ICL for the next DFT insertion pass:

```
extract_icl
```

Die-Level BSDL Extraction

This section describes the die-level BSDL extraction of a design that has already completed the DFT insertion pass with BoundaryScan.

Prerequisites

- DFT insertion has been completed. Refer to DFT Insertion.
- Refer to the workflow example diagram in Figure 33.

Procedure

1. Set the context and read the design libraries and files:

```
set_context dft -rtl -design_id bsdl_extraction

# Read the libraries and the design – full view of the package level
…
```

2. Extract the ICL description:

```
extract_icl
```

3. Move the system mode to setup:

```
set_system_mode setup
```

4. Synthesize the required blocks and emulate the loadboard loopbacks, if any.

5. Define the TDI/TDO/TCK/TMS/TRST attributes:

```
set_attribute_value -name function -value tdi tdi
set_attribute_value -name function -value tdo tdo
set_attribute_value -name function -value tck tck
set_attribute_value -name function -value tms tms
set_attribute_value -name function -value trst trst
```

Note:
If the design has a separate tdi_local, you can also define the "tdi_local" attribute for the port. If the tool finds a TdiMux during BSDL extraction and you did not previously define the "tdi_local" attribute, it assumes that the "tdi" of the design is also the "tdi_local" of the design.

6. Set the DftSpecification requirements:

```
set_dft_specification_requirements \
  -bsdl_extraction on
```

7. Set the DFT signals to control the mux configurations:

```
# Set the DFT signals for the bypasses
set dft_signals {bp2 bp3 bp4 bp5}
set dft_signals_and_values {bp2 0 bp3 0 bp4 0 bp5 0}
set_static_dft_signal_values $dft_signals_and_values
set_bsdl_extraction_options -dft_signals_for_bonding_configurations $dft_signals
```

8. Extract the BSDL file:

```
check_design_rules
```

9. Create patterns for IJTAG and BoundaryScan:

```
set_context patterns -ijtag

check_design_rules

set spec [create_patterns_specification -separate_tap_and_bscan_patterns on]
report_config_data $spec

process_patterns_specification
```

10. Verify the generated patterns:

```
run_testbench_simulations
```

Package-Level BSDL Extraction

Extract package-level BSDL containing the five die instances.

Prerequisites

- Die-level BSDL extraction pass has been completed. Refer to Die-Level BSDL Extraction.
- Refer to the workflow example diagram in Figure 33.

Procedure

1. Set the context and read the design libraries and files:

```
set_context dft -rtl

# Read the libraries and the  design – full view of the package level
…
```

2. Elaborate the design and ICL, and specify the level of the design for all subsequent commands:

```
set_current_design chip
set_design_level chip
extract_icl
```

3. Extract the BSDL from the die BSDL:

```
set_dft_specification_requirements -bsdl_extraction on
```

4. Read the BSDL files for the dies:

```
read_bsdl tsdb_outdir/dft_inserted_designs/
central_die_bsdl_extraction.dft_inserted_design/central_die.bsdl

read_bsdl tsdb_outdir/instruments/satellite_die_rtl_bscan.instrument/satellite_die.bsdl
```

5. Define the die configuration requirements:

```
# All satellite dies are active
set_bsdl_extraction_options -override_bonding_enable_values  {
  die1.bp2 1 die1.bp3 1 die1.bp4 1 die1.bp5 1
}
```

6. Add the logical connections:

```
# TCK, TMS, and TRST
foreach to_die { die2 die3 die4 die5 } {
   add_die_logical_connection -from die1/tms  -to die1/${to_die}_to_tms
   add_die_logical_connection -from die1/tck  -to die1/${to_die}_to_tck
   add_die_logical_connection -from die1/trst -to die1/${to_die}_to_trst
}

# Add the die logical connections
add_die_logical_connection -from die1/tdi -to die1/die2_to_tdi -pipeline_stages 1
add_die_logical_connection -from die1/die2_from_tdo -to die1/die3_to_tdi -pipeline_stages 2
add_die_logical_connection -from die1/die3_from_tdo -to die1/die4_to_tdi -pipeline_stages 2
add_die_logical_connection -from die1/die4_from_tdo -to die1/die5_to_tdi -pipeline_stages 2
add_die_logical_connection -from die1/die5_from_tdo -to die1/tdi -pipeline_stages 1
```

```
# report the added logical connections
report_die_logical_connections
```

7. Run Design Rule Checking to extract the BSDL:

```
check_design_rules
```

Package-Level Pattern Generation

Generate BoundaryScan patterns for the extracted BSDL file and MultiDieInterconnect patterns.

Prerequisites

- Package-level BSDL extraction has been completed. Refer to "Package-Level BSDL Extraction" on page 110.
- Refer to the workflow example diagram in Figure 33.

Procedure

1. Set the context and read the design libraries and files:

```
set_context dft -ijtag -rtl

# Read the libraries and the design, full view of the package level
…
```

2. Elaborate and specify the level of the design for all subsequent commands:

```
set_current_design chip
set_design_level chip
```

3. Read the BSDL files for the dies:

```
read_bsdl tsdb_outdir/dft_inserted_designs/
central_die_bsdl_extraction.dft_inserted_design/central_die.bsdl

read_bsdl tsdb_outdir/instruments/satellite_die_rtl_bscan.instrument/satellite_die.bsdl
```

4. Add the logical connections:

```
# TCK, TMS, and TRST
foreach to_die { die2 die3 die4 die5 } {
   add_die_logical_connection -from die1/tms  -to die1/${to_die}_to_tms
   add_die_logical_connection -from die1/tck  -to die1/${to_die}_to_tck
   add_die_logical_connection -from die1/trst -to die1/${to_die}_to_trst
}

# Add the die logical connections
add_die_logical_connection -from die1/tdi -to die1/die2_to_tdi -pipeline_stages 1
add_die_logical_connection -from die1/die2_from_tdo -to die1/die3_to_tdi -pipeline_stages 2
add_die_logical_connection -from die1/die3_from_tdo -to die1/die4_to_tdi -pipeline_stages 2
add_die_logical_connection -from die1/die4_from_tdo -to die1/die5_to_tdi -pipeline_stages 2
add_die_logical_connection -from die1/die5_from_tdo -to die1/tdi -pipeline_stages 1
```

```
# report the added logical connections
report_die_logical_connections
```

5. Run Design Rule Checking:

```
check_design_rules
```

6. Create patterns for BoundaryScan and MultiDieInterconnect tests:

```
set spec [create_patterns_specification -separate_tap_and_bscan_patterns on]
report_config_data $spec
```

7. Delete the RunTest(test_logic_reset) test from the pattern specification and update the DftSignals for the multi-die interconnect flow:

```
foreach_in_collection pattern [get_config_elements -in_wrapper $spec
Patterns(JtagTap*Patterns)] {
  delete_config_element -in_wrapper $pattern TestStep(JtagBscanTestStep)/BoundaryScan/
RunTest(test_logic_reset)
}

delete_config_element -in_wrapper $spec Patterns(MultiDieInterconnect)/DftControlSettings

read_config_data -in_wrapper $spec/Patterns(MultiDieInterconnect) -from_string "
   DftControlSettings {
     die1.bp2 : 1;
     die1.bp3 : 1;
     die1.bp4 : 1;
     die1.bp5 : 1;
   }
"
```

8. Verify the generated patterns:

```
process_patterns_specification
run_testbench_simulations
```

Tessent Shell Flow for 3D Stacked Designs

This section describes the DFT flow for dies in a 3D Stacked design.

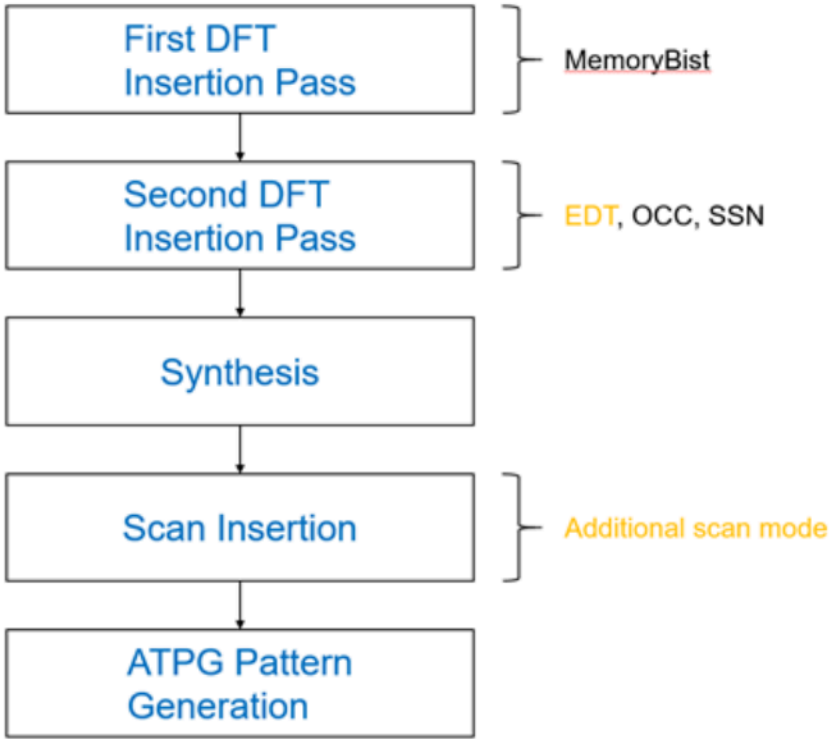
Core-Level DFT and Scan Insertion Flow

DFT insertion in a 3D-based design is die-oriented, so the core-level DFT insertion flow is similar to the standard insertion flow for physical blocks.

The areas in which the 3D DFT insertion flow differs are in scan test logic insertion and scan modes insertion. Refer to RTL and Scan DFT Insertion Flow for Physical Blocks in the *Tessent Shell User's Manual* for more information.

The following figure shows the block-level insertion flow, updated to show the additions to the second insertion pass. During EDT insertion, insert a second EDT controller for two external scan modes. The tool adds these additional modes during the scan insertion step.

Figure 34. DFT and Scan Insertion Flow



First DFT Insertion Pass: Inserting Core-Level IJTAG and MemoryBIST

The first DFT insertion pass is identical to the Tessent Shell Flow for Hierarchical designs.

This pass inserts the non-scan DFT elements such as MemoryBIST and IJTAG. For complete information about this step at the block level, see First DFT Insertion Pass Performing Block-Level MemoryBIST in the *Tessent Shell User's Manual*.

Second DFT Insertion Pass: Inserting Core-Level EDT, OCC, and SSN

This insertion pass is similar to the second DFT insertion pass in the SSN flow for physical blocks.

All ATPG clocks must have their own OCC Controllers. The EDT logic handles the scan tests inside the physical block and the wrapper chains. The SSN bus provides parallel test access to the EDT controllers.

You must register and add the parent_ext_edt_mode DFT signal for the new scan mode, and add the ext_edt_mode and parent_ext_edt _mode scan modes. Second DFT Insertion Pass Inserting Block-Level EDT, OCC, and SSN in the *Tessent Shell User's Manual* describes OCC and SSN insertion.

Procedure

1. Register and add the parent_ext_edt_mode DFT signal to be used in the new scan mode:

```
register_static_dft_signal_names \
  parent_ext_edt_mode \
  -usage scan_mode \
  -scan_mode_type external
add_dft_signals parent_ext_edt_mode
```

2. Add two scan modes to the external EDT controller: ext_edt_mode and parent_ext_edt_mode.

3. Insert OCC and SSN, as described in "Second DFT Insertion Pass: Inserting Block-Level EDT, OCC, and SSN" in the *Tessent Shell User's Manual*.

Examples

The following example shows the insertion of two EDT controllers, controller_int and controller_ext, connected to the EDT Sib. The internal mode scan chains attach to controller_int and are enabled with the int_edt_mode DFT signal. The external mode scan chains attach to controller_ext. The external scan chains are separated between the two modes, ext_edt_mode and parent_ext_edt_mode, enabled by their respective DFT signals.

```
EDT {
  ijtag_host_interface : Sib(edt);
  Controller(controller_int) {
    longest_chain_range    : 10, 50;
    scan_chain_count       : 8;
    input_channel_count    : 2;
    output_channel_count   : 1;
    Connections {
      mode_enables         : DftSignal(int_edt_mode);
    }
  }
  Controller(controller_ext) {
    longest_chain_range    : 10, 50;
```

```
    scan_chain_count       : 3;
    input_channel_count    : 1;
    output_channel_count   : 1;
    Connections {
      mode_enables         : DftSignal(ext_edt_mode),
                             DftSignal(parent_ext_edt_mode);
    }
  }
}
```

Performing Scan Insertion With Tessent Scan (Core Level)

At the gate level, you can add scan modes to the EDT controllers, analyze the scan chains, insert test logic, and write the graybox view of the design into the TSDB.

The logic that is present at the block level must be scan stitched into chains for three scan modes:

- Internal Scan Mode (internal logic scan flops)
- External Scan Mode (wrapper cells in the core)
- Parent External Scan Mode (wrapper cells that connect to the die boundary)

Note:
The three scan modes consist of the two conventional internal and external modes and the parent external scan mode. The input core wrapper cells that capture the die's primary inputs must be excluded from the external scan mode to prevent injecting unknowns into the scan chain. These input cells are part of the parent external mode, which you use if the die is in the external test mode.

Prerequisites

- You have performed synthesis as described in "Performing Synthesis" in the *Tessent Shell User Manual*.
- You have identified the wrapper cells that belong to the parent external mode and have generated a Tcl dictionary for them. You can do this manually or have the tool generate the dictionary automatically by using the tag_child_physical_block_pins, write_child_physical_block_pin_dictionary, and get_scan_elements_to_promote commands.
- For more details on scan insertion, see "Performing Scan Chain Insertion: Wrapped Core" in the *Tessent Shell User's Manual*.

Procedure

1. Add the three scan modes: int_edt_mode, ext_edt_mode, and parent_ext_edt_mode:

```
# Create a scan mode and specify the EDT instance
# to connect the scan chains to
add_scan_mode int_edt_mode \
  -edt_instances bd_corea_rtl2_tessent_edt_c1_int_inst
set parent_ext_elements \
  [get_scan_elements_to_promote \
  -child_physical_block_pin_dict_file \
  ./bottom_die.child_physical_block_pin_dict \
  -for_parent_block_ports selected]
set ext_elements [remove_from_collection \
  [get_scan_elements -class wrapper] \
  [filter_collection $parent_ext_elements "wrapper_type==input"]]
add_scan_mode ext_edt_mode \
  -edt_instances bd_corea_rtl2_tessent_edt_c1_ext_inst \
  -include_elements $ext_elements
if {[sizeof_collection $parent_ext_elements] > 0} {
  add_scan_mode parent_ext_edt_mode \
    -edt_instances bd_corea_rtl2_tessent_edt_c1_ext_inst \
    -include_elements $parent_ext_elements
}
analyze_scan_chains
insert_test_logic
```

Note:
The set of scan cells that are part of the parent_ext_edt_mode is excluded from the ext_edt_mode.

2. To improve retargeting performance and simulation of the scan test, write one scan_graybox for each external scan mode:

```
set_context patterns -scan
import_scan_mode ext_edt_mode
check_design_rules
analyze_graybox
write_design -tsdb -graybox
set_system_mode setup
import_scan_mode parent_ext_edt_mode
check_design_rules
analyze_graybox
write_design -tsdb -graybox \
  -variant_scan_graybox_id parent_ext_edt_mode
```

3. To improve IJTAG-based simulation performance, write the ijtag_graybox:

```
set_context patterns -ijtag
set_system_mode analysis
analyze_graybox -ijtag
write_design -tsdb -graybox
```

Generating Core-Level ATPG Patterns

Generating ATPG patterns at the chip level in a multi-die environment is similar to generating block-level ATPG patterns in the SSN workflow. You can retarget the internal scan mode patterns to the die level and the package level.

Refer to "Generating Block-Level ATPG Patterns" in the *Tessent Shell User's Manual* for more information.

Die-Level DFT and Scan Insertion Flow (3D)

The Tessent Shell flow enables independent testing of the dies, retargeting internal modes from dies, and creating consolidated patterns to detect faults on die-to-die connections.

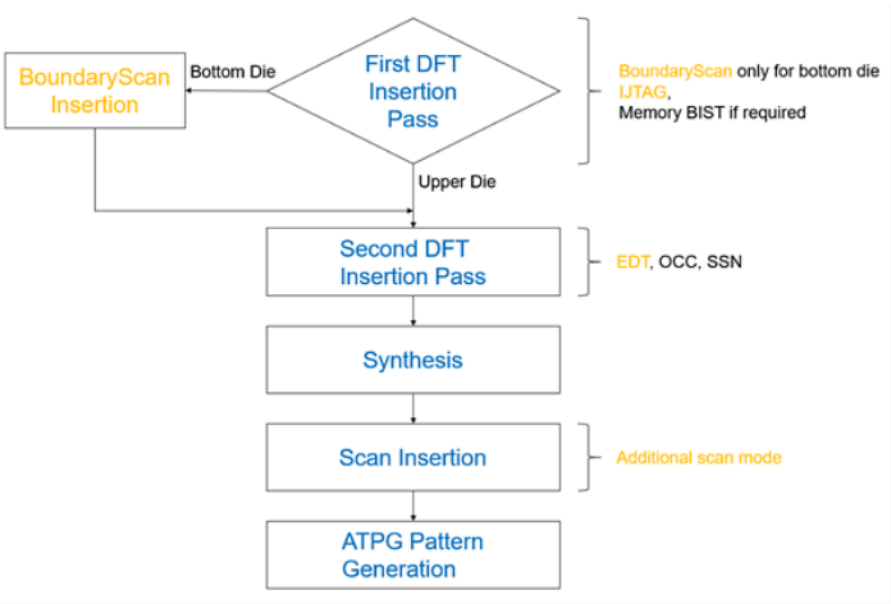
You create a design based on the 3D stack approach from dies. Each die is a chip with its own test logic and scan isolation. Dies are stacked on the next level of hierarchy, which is the package level, where only wired connections are present. The functional clock tree is localized within each die and controlled by dedicated OCCs.

A 3D-based design affects different aspects of testing, and the Tessent Multi-die flow provides the following capabilities:

- Boundary Scan
  - BSDL Extraction at the Package Level. You can extract the BSDL at the package level by reading the BSDL of the base die with boundary scan inserted.
- IJTAG
  - Sacrificial pads.
  - IJTAG insertion in the Tessent Multi-die flow includes the insertion of PTAP and STAP logic that provides serial test access as required by the IEEE 1838 standard.
- EDT
  - Dies in multi-die designs require two scan modes. The internal scan mode tests the die's internal logic and uses of the external mode of the cores. Stack-level die-to-die testing uses the external scan mode of the die.
- SSN
  - Tessent SSN is a Flexible Parallel Port (FPP) from the IEEE 1838 standard. This handles the die-to-die scan test communication.
- Functional clock tree localized within each die. 3D designs do not contain a central clock tree, because only the base die has contact with the package pads. The clock signal propagates through all dies in the stack. The clock signal output of the die is multiplexed to support parallel testing of the dies in internal modes, and to enable the synchronization of the DWR in external modes during the die-to-die test.

Figure 35 shows the standard top-level workflow you follow when not using Tessent Multi-die, with additions to indicate that you make changes in the first DFT insertion pass during BoundaryScan and IJTAG insertion. The second DFT insertion pass inserts EDT clock muxes to bypass the OCC for the localized clock tree during INTEST of the dies. During EDT insertion, you insert a second EDT controller for the die EXTEST mode. The Tessent scan insertion step adds the respective external scan mode.

Figure 35. First DFT Insertion Pass: Inserting Die-Level Boundary Scan, IJTAG, and MemoryBIST



First DFT Insertion Pass: Inserting Die-Level Boundary Scan, IJTAG, and MemoryBIST

During the first DFT insertion pass at the die level, you insert BoundaryScan, IJTAG, and MemoryBIST.

BoundaryScan cells are inserted only on the bottom die, and the upper dies do not have BoundaryScan. This step is similar to the first DFT insertion pass of the top-level SSN insertion and verification flow. However, BoundaryScan insertion excludes any port not connecting directly to the package boundary. Refer to Top-Level SSN Insertion and Verification in the *Tessent Shell User's Manual* for more information.

Note:
The tool adds BoundaryScan auxiliary logic to use the functional ports of the die as the SSN access interface.

In the Tessent Multi-die flow, the TAP is enhanced to PTAP if you specify the STAP insertion. To insert the STAP, use the create_dft_specification -stap_host_list command. This switch specifies a list of STAP ids and pipelines. The tool places the first element in the list closest to the stap_to_si pin of the PTAP, and the last element in the list is placed closest to the stap_from_so pin of the PTAP.

Prerequisites

- For a complete description of BoundaryScan insertion using auxiliary ports for SSN, refer to First DFT Insertion Pass Performing Top-Level MemoryBIST in the *Tessent Shell User's Manual*.
- You can perform MemoryBIST insertion in parallel with BoundaryScan if it is inserted in a particular die. Refer to First DFT Insertion Pass Performing Top-Level MemoryBIST and Boundary Scan in the *Tessent Shell User's Manual* for information and prerequisites.

Procedure

1. Enable MemoryBIST and BoundaryScan insertion:

```
set_dft_specification_requirements \
  -boundary_scan on -memory_test on
```

2. Specify which bottom-die ports should not be touched during BoundaryScan insertion:

```
# Set BoundaryScan insertion to not touch
# interconnect ports without I/O pads
set bottom_ports \
  [get_ports {my_tck my_tdi my_tdo my_tms my_trst clk gpio}]
# Get all ports and remove the set of ports that need BoundaryScan
set tsv_ports [remove_from_collection [get_ports] $bottom_ports]
set_boundary_scan_port_options $tsv_ports -cell_options dont_touch
check_design_rules
```

3. For all dies in the stack, generate the DftSpecification and use the -stap_host_list switch to specify the STAPs and pipelines that you want to insert:

```
# Create the DftSpecification with two 3D STAPs and two pipelines:
set spec [create_dft_specification \
  -stap_host_list [list right pipe(between) left pipe(end)]]
```

Tip
The top-most die in a stack does not need an STAP. To avoid inserting STAP in the top-most die, do not specify the -stap_host_list switch.

4. Add auxiliary logic on the inputs and outputs used for the SSN bus and bus clock to utilize functional ports:

```
read_config_data -in_wrapper ${spec}/BoundaryScan -from_string {
  AuxiliaryInputOutputPorts {
    auxiliary_input_ports  : gpio[11:8];
    auxiliary_output_ports  : gpio[2:0];
  }
}
```

Second DFT Insertion Pass: Inserting Die-Level EDT, OCC, and SSN

During the second DFT insertion pass at the die level, you insert any EDT, OCC, and SSN elements your design requires.

Each die has a localized clock tree. Insert clock muxes to enable parallel testing of the dies in internal mode and chain synchronization across all dies' DWR. The muxes bypass OCC to provide a free-running clock to the next die in a stack or source the clock from the functional clock tree.

The EDT logic described by the DFT specification specifies the insertion of two EDT controllers. The DWRs are used to ensure good isolation in a die, and these DWR scan chains differ in length compared to the scan chains stitched across the internal logic of the die. Two controllers are needed to ensure optimal compression.

The tool creates the SSN hardware based on the SSN wrapper in the DftSpecification. For a description of the SSN wrapper, see SSN in the *Tessent Shell Reference Manual*.

In this step, the SSN network is inserted in the base die to enable interaction with the upper dies in the stack.

Prerequisites

- For a detailed description of SSH, EDT, and OCC insertion done in one DFT insertion pass, refer to "Second DFT Insertion Pass: Inserting Top-Level EDT, OCC, and SSN" in the *Tessent Shell User's Manual*.
- The clock multiplexing for output clocks is also inserted in this step. For more information, see "Clocking During Logic Test" in the *Tessent Shell User's Manual*.

Procedure

1. Insert the clock muxes. The left_clk_out and right_clk_out signals are sourced from clk_buf, and the int_ltest_en DFT signal drives the multiplexer. This bypasses the OCC, and the upper die can run in parallel with the bottom die when in internal test mode.

```
add_dft_clock_mux left_clk_out \
  -test_clock_source clk_buf/C -dft_signal_source_name int_ltest_en
add_dft_clock_mux right_clk_out \
  -test_clock_source clk_buf/C -dft_signal_source_name int_ltest_en
```

2. Insert EDT using the DftSpecification. The following example shows the insertion of two EDT controllers connected to the EDT Sib. The internal mode scan chains attach to c1_int and are enabled with the int_edt_mode DFT signal. The connect_bscan_segments_to_lsb_chains property is set to on to test the BoundaryScan chain like a scan chain. The c1_ext signal connects the scan chains related to the external mode.

```
EDT {
  ijtag_host_interface : Sib(edt);
  Controller(c1_int) {
    longest_chain_range            : 40, 50;
    scan_chain_count               : 8;
    input_channel_count            : 2;
    output_channel_count           : 1;
```

```
    connect_bscan_segments_to_lsb_chains : on;
    Connections {
      mode_enables                 : DftSignal(int_edt_mode);
    }
  }
  Controller(c1_ext) {
    longest_chain_range            : 10, 20;
    scan_chain_count               : 3;
    input_channel_count            : 1;
    output_channel_count           : 1;
    Connections {
      mode_enables                 : DftSignal(ext_edt_mode);
    }
  }
}
```

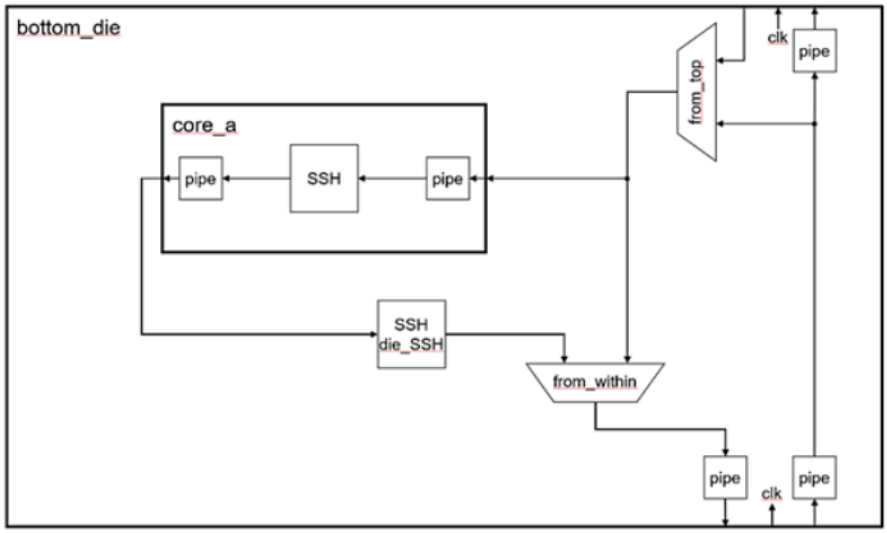
3. In a 3D stack, reconfiguring the SSN network can be useful for retargeting and simulating patterns. To provide your die with greater flexibility and the least loopback, you should insert a multiplexer that enables bypassing the ScanHost. The following example shows the SSN network with a secondary SSN interface, a physical block with SSH inside, and the main SSH for this die.

```
SSN {
  ijtag_host_interface : Sib(ssn);
  DataPath(1) {
    output_bus_width :3;
    Connections {
      bus_clock_in : ssn_bus_clock_in;
      bus_data_in  : ssn_bus_data_in[%d];
      bus_data_out : ssn_bus_data_out[%d];
    }
    OutputPipeline(out) {
    }
    Multiplexer(from_within) {
      ScanHost(die_SSH) {
        input_chain_count : 2;
        output_chain_count : 2;
      }
      DesignInstance(core_a) {
      }
    }
    Multiplexer(from_top) {
      Connections {
        secondary_bus_data_in : ssn_data_in_up[2:0];
      }
    }
    pipeline(in) {
      ExtraOutputPath {
        Connections {
          bus_clock_out : ssn_clock_out_up;
          bus_data_out  : ssn_data_out_up[2:0];
        }
        pipeline(top) {
        }
```

```
            }
        }
    }
}
```

This DftSpecification results in the logic shown in the following figure.

Figure 36. Reconfigured SSN



Performing Scan Insertion With Tessent Scan (Die Level)

At the gate level, you can add scan modes to the EDT controllers, analyze the scan chains, insert test logic, and write the graybox view of the design into the TSDB.

The logic at the top level must be scan stitched along with the external-mode wrapper chains of the cores.

Note:
The tool only promotes the scan cells of ports that connect to the top level to shared wrapper cells. The ports that connect to other dies do not have dedicated wrapper cells.

Unlike hierarchical chip-level flow in 3D designs at the die level, the tool adds two scan modes. The die's internal scan mode tests the die's internal logic. In this mode, the core's external scan mode should be active. Die-to-die test at the package level uses the die's external scan mode. The core's parent external scan mode should be active during die external mode.

Prerequisites

- Before inserting the scan chains, perform synthesis as described in "Performing Synthesis" in the *Tessent Shell User's Manual*.

Procedure

1. Specify the wrapper analysis options to only promote flops that interact with top-level ports to shared wrapper cells:

```
set tsv_ports [get_ports {left_* right_*}]
set_wrapper_analysis_options \
  -exclude_ports [remove_from_collection [get_port] $tsv_ports]
set_dedicated_wrapper_cell_options off -ports $tsv_ports
```

2. Specify active scan mode in child physical blocks before performing DRC checks. In the die's int_edt_mode, the child block's active mode is ext_edt_mode. In the die's ext_edt_mode, the child block's active mode is parent_ext_edt_mode. Use the die's ext_edt_mode for die-to-die tests at the package level.

```
# Set the active scan mode for child physical blocks
set_attribute_value {corea_left corea_right} \
  -name active_child_scan_mode -value ext_edt_mode
check_design_rules
analyze_wrapper_cells
# Create a scan mode and specify the EDT instance to
# connect the scan chains to
add_scan_mode int_edt_mode -edt_instance \
  bottom_die_rtl2_tessent_edt_c1_int_inst
set_attribute_value {corea_left corea_right} \
  -name active_child_scan_mode -value parent_ext_edt_mode
add_scan_mode ext_edt_mode \
  -edt_instance bottom_die_rtl2_tessent_edt_c1_ext_inst \
  -include_elements [get_scan_elements *occ*] \
 -associate_chains [get_scan_elements -type chain]
analyze_scan_chains
insert_test_logic
```

3. Write out the scan_graybox for die external scan mode and ijtag_graybox:

```
# Create the ijtag graybox and scan graybox
# for the scan inserted netlist
set_context patterns -scan
set_system_mode setup
read_design bd_corea -view graybox \
  -variant_scan_graybox_id parent_ext_edt_mode -force
set_current_design bottom_die
import_scan_mode ext_edt_mode
set_test_setup_icall{
  bottom_die_rtl2_tessent_ssn_mux_from_within_inst.setup
  select_secondary_bus 1
}
check_design_rules
analyze_graybox
write_design -tsdb -graybox -verbose
```

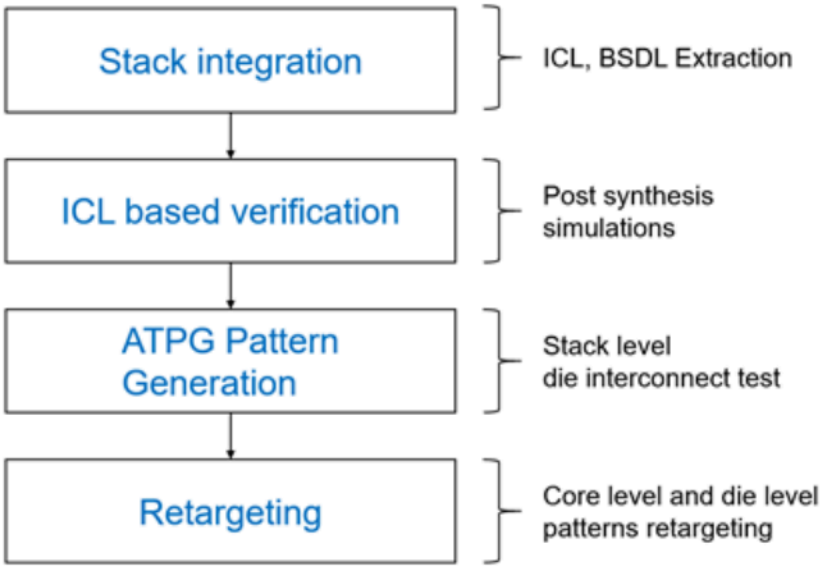
```
set_context patterns -ijtag
set_system_mode analysis
analyze_graybox -ijtag
write_design -tsdb -graybox
```

Stack-Level Integration and Verification

The flow at the stack level focuses on design integration and verification. The package-level netlist provides signal connectivity between the dies, and from the dies to the package pins. The dies are instantiated on top of each other, and all signals to the upper dies go through the bottom die. The connectivity consists of wires only (there is no logic outside the dies). There is no DFT insertion at the stack level as all required DFT logic is already inserted in the dies.

Figure 37 shows the default flow for 3D stacks.

Figure 37. Default Flow for 3D Stacks



Stack Integration With ICL and BSDL Extraction

Integrate the stack after inserting all required DFT into the dies. Tessent traces through ports to extract the ICL. The BSDL of the bottom die is extracted at the package level.

Note:
Only the die interface view is required for ICL descriptions to extract ICL.

You should exclude floating pins (pins facing upwards at the top-most die) from ICL extraction.

Prerequisites

- All dies used in the stack have DFT inserted per the Die-Level DFT and Scan Insertion Flow (3D).
- The design level is set with the set_design_level chip command. Tessent uses the chip level as the design level for stack integration.

Procedure

1. Read the stack netlist and dies in the interface view. Elaborate the design and set the design level:

```
read_verilog ../data/stack.v
read_design bottom_die -design_id gate -view interface \
  -include_child_blocks_icl_and_pdl
read_design upper_die -design_id gate -view interface \
  -include_child_blocks_icl_and_pdl
set_current_design stack
set_design_level chip
```

2. Exclude the floating pins from ICL extraction:

```
set_attribute_value \
  [get_pins {*_up} -of_instances upper_die_*_top] \
  -name ignore_during_icl_extraction
```

3. Specify the clock ports:

```
add_clocks clk -period 5ns
```

4. Update the TSDB with the integrated design and extract ICL:

```
write_design -tsdb
extract_icl
```

5. Extract the Unified BSDL for the stack:

```
set_dft_specification_requirements -bsdl_extraction on
read_bsdl \
  tsdb_outdir/instr/bottom_die_rtl1_bscan.instr/bottom_die.bsdl
check_design_rules
```

Related Topics

read_bsdl [Tessent Shell Reference Manual]

delete_bsdl [Tessent Shell Reference Manual]

report_bsdl [Tessent Shell Reference Manual]

Verifying the ICL-Based Patterns After Stack Integration

Resimulating the MemoryBIST and ICL verification patterns after synthesis ensures that the ICL network is fully functional and able to be reliably used during the ATPG and Scan retargeting setup. The ICL network must be verified before ATPG and retargeting because the SSN setup relies on IJTAG.

Refer to "Verifying the ICL Model" in the *Tessent Shell User's Manual* for more information about ICL model verification.

Note:
To verify the ICL model, reading only the ijtag_graybox view is sufficient. This reduces the simulation computation time.

Procedure

1. Read the ijtag_graybox view of the dies and the physical blocks instantiated in the dies with the read_design -include_child_blocks_icl_and_pdl command:

```
read_design upper_die  -design_id gate -view ijtag_graybox \
  -include_child_blocks_icl_and_pdl
read_design bottom_die  -design_id gate -view ijtag_graybox \
  -include_child_blocks_icl_and_pdl
read_design stack -design_id gate1 -view ijtag_graybox
set_current_design stack
```

2. Specify the attribute to simulate the instruments in the lower physical instances and create the PatternsSpecification:

```
set_defaults_value \

 /PatternsSpecification/SignoffOptions/simulate_instruments_in_lower_physical_instances
 on
set spec [create_patterns_specification]
```

3. Configure the SSN path to test all possible paths:

```
set SSNContinuityVerify \
  [get_config_element SSNContinuityVerify(default) -in $spec -hier]
read_config_data -before $SSNContinuityVerify -from_string {
  ProcedureStep(ssn_setup) {
    iCall(bottom_die.bottom_die_rtl2_tessent_ssn_mux_from_within_inst.setup) {
      iProcArguments {
        args : "select_secondary_bus 1";
      }
    }

    iCall(bottom_die.bottom_die_rtl2_tessent_ssn_mux_from_top_left_inst.setup) {
      iProcArguments {
        args : "select_secondary_bus 1";
      }
    }
```

```
    iCall(bottom_die.bottom_die_rtl2_tessent_ssn_mux_from_top_right_inst.setup) {
      iProcArguments {
        args : "select_secondary_bus 1";
      }
    }

    iCall(upper_die_left_mid.upper_die_rtl2_tessent_ssn_mux_from_within_inst.setup){
      iProcArguments {
        args : "select_secondary_bus 1";
      }
    }

    iCall(upper_die_left_mid.upper_die_rtl2_tessent_ssn_mux_from_top_inst.setup) {
      iProcArguments {
        args : "select_secondary_bus 1";
      }
    }

    iCall(upper_die_right_mid.upper_die_rtl2_tessent_ssn_mux_from_within_inst.setup){
  iProcArguments {
    args : "select_secondary_bus 1";
  }
}

iCall(upper_die_right_mid.upper_die_rtl2_tessent_ssn_mux_from_top_inst.setup) {
  iProcArguments {
    args : "select_secondary_bus 1";
  }
}

iCall(upper_die_left_top.upper_die_rtl2_tessent_ssn_mux_from_within_inst.setup) {
  iProcArguments {
    args : "select_secondary_bus 1";
  }
}

iCall(upper_die_left_top.upper_die_rtl2_tessent_ssn_mux_from_within_inst.setup) {
  iProcArguments {
    args : "select_secondary_bus 1";
  }
}
  }
}
```

4. Verify the ICL network for stack with all dies as ijtag_graybox:

```
read_config_data -in $spec -
from_string {
  Patterns(ICLNetwork_all) {
    ICLNetworkVerify(stack) {
    }
    SimulationOptions {
```

```
      LowerPhysicalBlockInstances {
        bottom_die : ijtag_graybox;
        upper_die_left_mid : ijtag_graybox;
        upper_die_left_top : ijtag_graybox;
        upper_die_right_mid : ijtag_graybox;
        upper_die_right_top : ijtag_graybox;
      }
    }
  }
}
```

5. Process the PatternsSpecification wrapper and simulate patterns:

```
process_patterns_specification
set_simulation_library_sources \
  -v ../data/adk.lib/verilog/adk.v -y ../data/mems -extension v
run_testbench_simulations
```

Generating Stack-Level ATPG Patterns

You can create scan test patterns to test the interconnect between devices in a 3D design. Verify the connection between the package and the bottom die with the BoundaryScan test.

Note:
The view of the dies in the stack must match the specified scan mode. You must configure the SSN patch to access all SSHs at the die level to perform the die-to-die test.

Prerequisites

- You have a validated ICL model of the current design.

Procedure

1. Read the scan_graybox view of the dies:

```
read_design bottom_die -design_id gate -view scan_graybox
read_design upper_die -design_id gate -view scan_graybox
```

2. Read the integrated stack design and set it as the current design:

```
read_design stack -design_id gate1
set_current_design stack
```

3. Specify the scan modes for the dies and set the scan mode for the stack:

```
add_core_instances \
  -instances bottom_die -mode ext_edt_mode_slow_capture
add_core_instances \
  -instances upper_die_*_top -mode ext_mode_top_die
add_core_instances -instances upper_die_*_mid -mode ext_mode
set_current_mode stack_full_stuck -type internal
```

4. Configure the SSN path to access SSH in each die:

```
set_test_setup_icall {
  bottom_die.bottom_die_rtl2_tessent_ssn_mux_from_within_inst.setup
  select_secondary_bus 1
} -append -merge -non_retargetable

set_test_setup_icall {
  bottom_die.bottom_die_rtl2_tessent_ssn_mux_from_top_left_inst.setup
  select_secondary_bus 1
} -append -merge -non_retargetable

set_test_setup_icall {
  bottom_die.bottom_die_rtl2_tessent_ssn_mux_from_top_right_inst.setup
  select_secondary_bus 1
} -append -merge -non_retargetable

set_test_setup_icall {
   upper_die_left_mid.upper_die_rtl2_tessent_ssn_mux_from_within_inst.setup
   select_secondary_bus 1
} -append -merge -non_retargetable

set_test_setup_icall {
  upper_die_left_mid.upper_die_rtl2_tessent_ssn_mux_from_top_inst.setup
  select_secondary_bus 1
} -append -merge -non_retargetable

set_test_setup_icall {
  upper_die_right_mid.upper_die_rtl2_tessent_ssn_mux_from_within_inst.setup
  select_secondary_bus 1
} -append -merge -non_retargetable

set_test_setup_icall {
  upper_die_right_mid.upper_die_rtl2_tessent_ssn_mux_from_top_inst.setup
  select_secondary_bus 1
} -append -merge -non_retargetable

set_test_setup_icall {
  upper_die_left_top.upper_die_rtl2_tessent_ssn_mux_from_within_inst.setup
  select_secondary_bus 1
} -append -merge -non_retargetable
```

```
set_test_setup_icall {
  upper_die_right_top.upper_die_rtl2_tessent_ssn_mux_from_within_inst.setup
  select_secondary_bus 1
} -append -merge -non_retargetable
```

5. Perform a DRC check of the design, set the fault type, and create patterns. The tool writes the stack's scan mode (stack_full_stuck) and patterns to the TSDB.

```
check_design_rules
set_fault_type stuck
create_patterns
write_tsdb_data -replace
```

6. Write the patterns.

```
write_patterns \
  stack_atpg.testbenches/stack_stack_full_stuck_parallel.v \
  -verilog -replace -parameter_list "SIM_COMPARE_SUMMARY 1" \
  -pattern_set scan
write_patterns \
  stack_atpg.testbenches/stack_stack_full_stuck_loopback.v \
  -serial -verilog -replace -param_list "SIM_COMPARE_SUMMARY 1" \
  -pattern_sets ssn_loopback
write_patterns \
  stack_atpg.testbenches/stack_stack_full_stuck_serial_chain.v \
  -verilog -serial -replace -end 1 \
  -parameter_list "SIM_COMPARE_SUMMARY 1 SIM_PARALLEL_MONITOR 1" \
  -pattern_set chain
write_patterns \
  stack_atpg.testbenches/stack_stack_full_stuck_serial_scan.v \
  -verilog -serial -replace -end 1 \
  -parameter_list "SIM_COMPARE_SUMMARY 1 SIM_PARALLEL_MONITOR 1" \
  -pattern_set scan
```

Note:
Refer to Generating Block-Level ATPG Patterns in the SSN Workflow for more details on the patterns written in this step.

7. Create the PatternsSpecification for the ATPG testbenches using the create_patterns_specification command with the -testbench_files switch and provide a list of written patterns.

```
create_patterns_specification -replace -testbench_files { \
  stack_atpg.testbenches/stack_stack_full_stuck_loopback.v \
  stack_atpg.testbenches/stack_stack_full_stuck_parallel.v \
  stack_atpg.testbenches/stack_stack_full_stuck_serial_chain.v \
  stack_atpg.testbenches/stack_stack_full_stuck_serial_scan.v}
```

Simulating the patterns is automated with the PatternsSpecification flow. The -testbench_files switch ensures the simulator compiles and uses the correct design view for each child block of the design.

The following code shows the created PatternsSpecification:

```
PatternsSpecification(stack,gate,stack_full_stuck_signoff) {
  Patterns(stack_stack_full_stuck_serial_scan) {
    SimulationOptions {
      LowerPhysicalBlockInstances {
        bottom_die : scan_graybox;
        upper_die_left_mid : scan_graybox;
        upper_die_left_top : scan_graybox;
        upper_die_right_mid : scan_graybox;
        upper_die_right_top : scan_graybox;
      }
    }
    TestbenchVerify(1) {
      testbench_file_name : \
        stack_atpg.testbenches/stack_stack_full_stuck_serial_scan.v;
    }
  }
…
}
```

Note:
The die design views are set as scan grayboxes because this view is sufficient for simulating the die-to-die patterns.

8. Process the PatternsSpecification and run testbench simulations.

Tip
Simulate the parallel load scan patterns first, followed by the serial SSH loopback patterns, one Serial Chain pattern, and one Serial Scan pattern.

Retargeting Core and Die-Level Patterns

After verifying the ICL model of the stacked design, you do not need to generate new scan patterns for die and core internal scan modes. You can retarget the previously created patterns for internal scan mode to the stack level.

The retargeting at the stack level is similar to that at the chip level with SSN. Refer to "Retargeting ATPG Patterns" in the *Tessent Shell User's Manual* for more details.

Note:
This procedure has certain restrictions:

- You can perform retargeting on one block at a time, or multiple blocks if they are properly isolated.
- Only the internal scan modes are retargetable.
- You must read the block as the full view to retarget the internal scan mode.
- You must read any block that contains a feedthrough to access another currently retargeted block in at least the ijtag_graybox view. You can read in blocks that do not contain such feedthroughs as interface view.
- Read the scan_graybox view with different variants if the block is in external test mode.

Procedure

1. To retarget scan patterns on groups of blocks, read the ijtag_graybox view of all physical blocks.
2. Import the retargetable modes for any groups.
3. Read the precomputed patterns for all selected instances.
4. Write out the retargeted patterns.

Examples

Figure 38 shows the views during the retargeting of the internal test of core instances. The ijtag_graybox view of the dies is sufficient as it contains the IJTAG network, including PTAP/STAP and SSN bus.

Figure 38. Design Views During Retargeting of Internal Test Patterns for All Core Instances

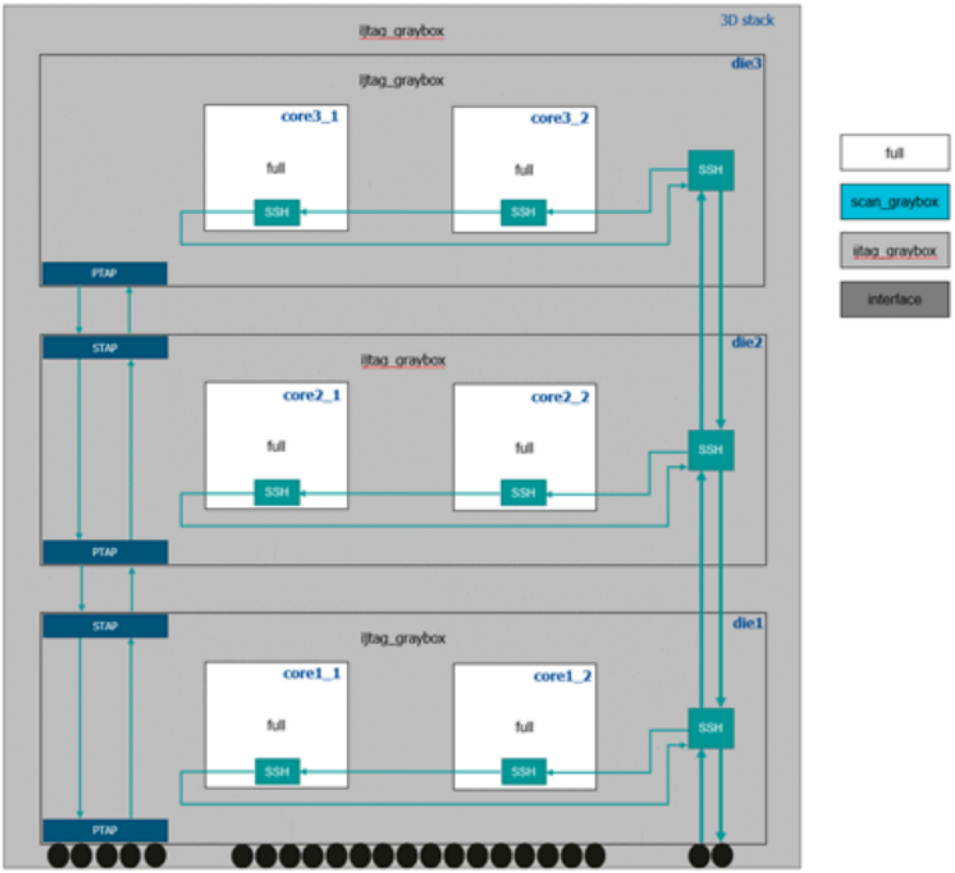


Figure 39 shows the views during the retargeting of a single core. The retargetable core 3_1 is in full view. Assuming that the SSN bus access to core3_1 is through the core3_2, the ijtag_graybox view is required. All the cores in other instances may be read as interface views because the SSN bus can access core3_1 without needing to go through them.

Figure 39. Design Views During Internal Test Pattern Retargeting of a Single Core Instance

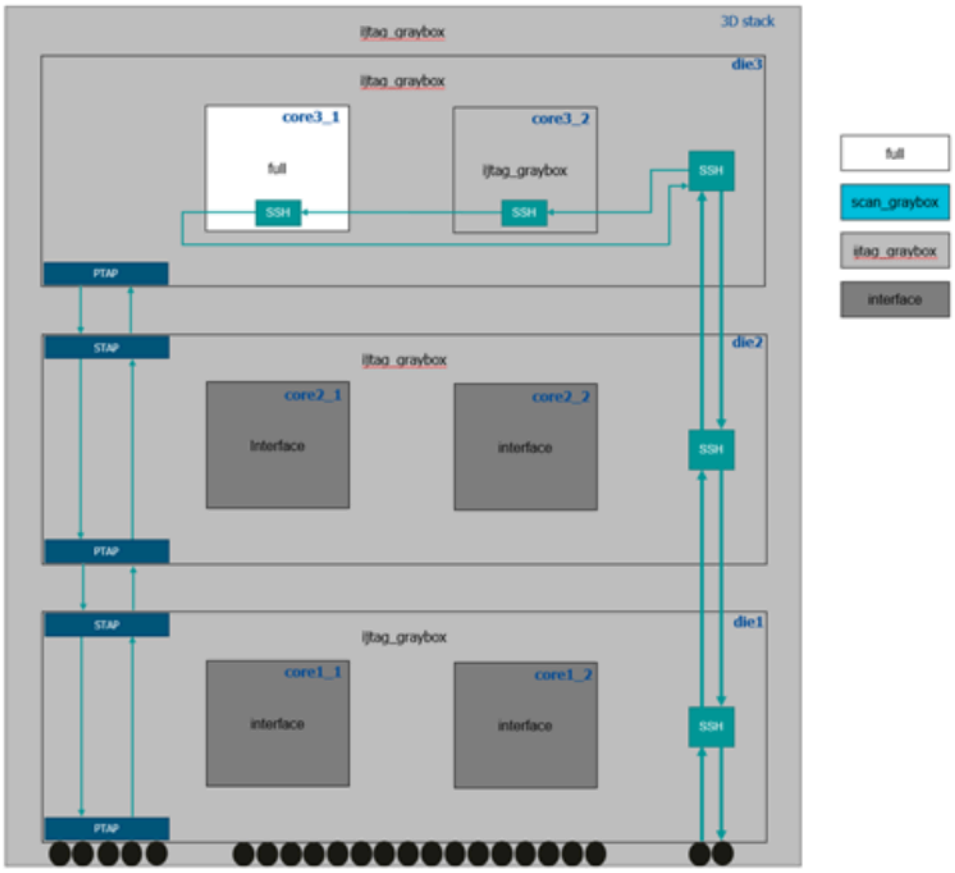


Figure 40 shows the views during the retargeting of the internal test of the dies. The die instances must be read as full view. You can read the core instances inside the dies with the scan_graybox view as it contains the PTAP/STAP, SSN bus, and scan chains with wrapper cells.

Figure 40. Design Views During Internal Test Pattern Retargeting for All Die Instances

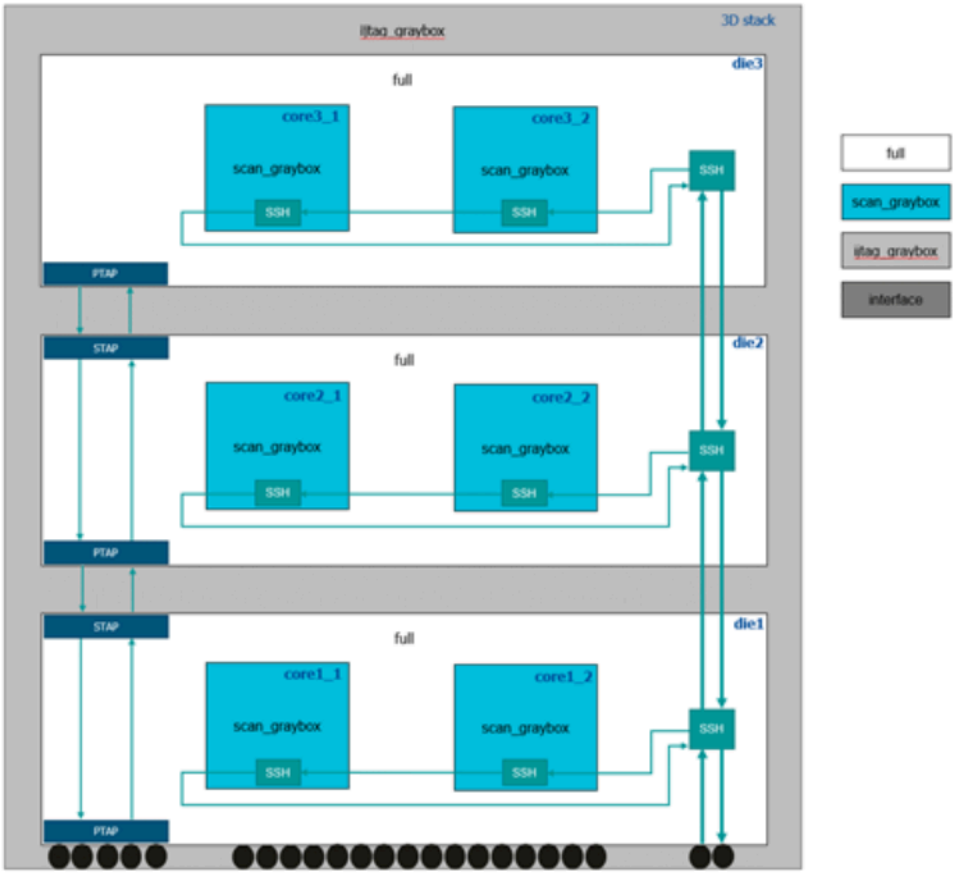
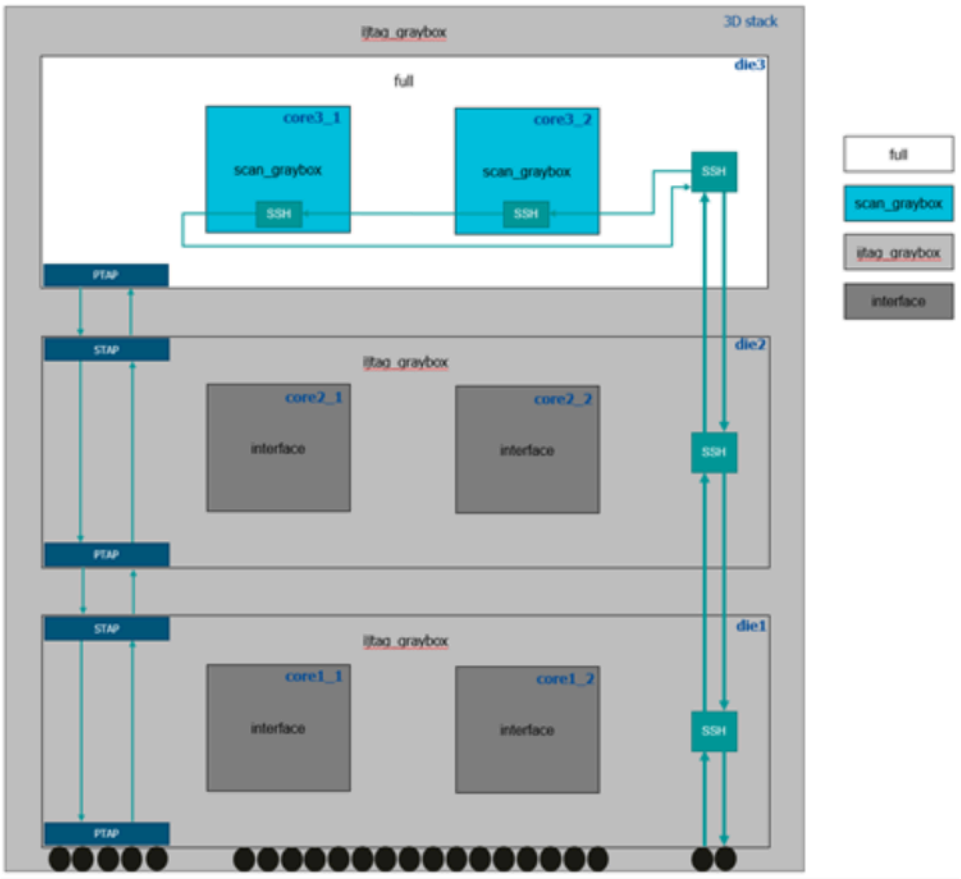


Figure 41 shows the views during the retargeting of a single die. Read the retargeted die in full view, and the cores in the retargeted die as scan_graybox. Read all other dies that provide SSN bus access to the top die as the ijtag_graybox view. You can read the cores in these dies as interface views because they are not needed for retargeting of the top-most die.

Figure 41. Design Views During Internal Test Pattern Retargeting For a Single Die Instance



Appendix A
Getting Help

There are several ways to get help when setting up and using Tessent software tools. Depending on your need, help is available from documentation, online command help, and Siemens EDA Support.

Tessent Documentation System

From the Tessent 2024.1 release, Siemens provides an improved method to access your Siemens Digital Industries Software product documentation. The default option serves your product documentation from Support Center, giving you immediate access to the latest release-specific documentation, and an enhanced search of HTML and PDF files. This new method also eliminates the requirement to install product documentation as part of the local software installation.

For easy access, you can now view Support Center documentation using a documentation proxy on your network. This documentation proxy removes the need for your users to have a Support Center account or to log into Support Center to view documentation.

We understand that some customers use our products on restricted networks without internet access. You have the option to download the documentation and set up a Siemens Documentation Server to view the documentation package on your local network.

Refer to "Configuring Documentation Access" in the *Managing Tessent Software* manual for instructions to set up the documentation proxy or the documentation server.

Global Customer Support and Success

A support contract with Siemens Digital Industries Software is a valuable investment in your organization's success. With a support contract, you have 24/7 access to the comprehensive and personalized Support Center portal.

Support Center features an extensive knowledge base to quickly troubleshoot issues by product and version. You can also download the latest releases, access the most up-to-date documentation, and submit a support case through a streamlined process.

`https://support.sw.siemens.com`

If your site is under a current support contract, but you do not have a Support Center login, register here:

`https://support.sw.siemens.com/register`

Glossary

2.5D

Similar to a printed circuit board (PCB and 2D), but more complex. With 2.5D, the PCB is called an interposer and made in a fab from silicon. 2.5D technology achieves a much finer bump pitch than standard IC bump pitch by using microbumps. Microbumps enable many thousands of interconnects between ICs, and achieve better performance than standard PCBs. The devices in a SiP use I/O buffers to communicate between the ICs assembled on the interposer.

3D

3D devices are individual dies vertically stacked on each other. The dies are connected using Through-Silicon Vias (TSVs), which optionally go through a simplified I/O buffer with ESD protection. The advantage of 3D compared to 2.5D is that even more die-to-die (D2D) interconnects and you can achieve better performance with 3D interconnects. The die that communicates to devices or package pins outside of the 3D stack is called the base die. The base die contains full I/O buffers such as GPIO, LVDS, or SerDes. Typically, the base die is the bottom die in the 3D stack.

5.5D

A combination of 2.5D and 3D technologies. For example, an ASIC on an interposer with four 3D HBM DRAM stacks also mounted on the interposer and communicate with the ASIC.

BSDL

Boundary Scan Description Language. The IEEE-1149.1 design file describes the Test Access Port (TAP) Controller, TAP I/Os, the boundary scan chain architecture, and the length of any TDRs. The BSDL language is based on VHDL.

Chiplet

A chip from a third-party supplier that usually has a specific function. For example, a chiplet could be an Ethernet subsystem comprising a SerDes and MAC but also includes a PLL. Chiplets are supplied as fully-tested bare dies and are assembled into the SiP with one or more SoCs.

Interposer

A silicon-based substrate used like a PCB to connect multiple ICs but with a much finer pitch and higher connectivity. Interposers are manufactured in an IC fab process line. Also refer to 2.5D.

KGD

Known Good Die testing. A wafer sort test strategy where you run the full die-level test program. This can be challenging due to the finer bump pitch for 2.5D/3D devices and the noisy and high RLC test environment.

MCM

Multi-Chip Module. MCM is an older technology similar to 2.5D, but instead of silicon, the substrate material is based on FR-4 (fiberglass), PTFE (Teflon), or ceramic. The MCM bump and wire pitch are similar to printed circuit boards, so the complexity is simpler than 2.5D.

PCB

Printed Circuit Board. Typically, an FR-4 (fiberglass) substrate board with one or more layers of conductive traces used to connect multiple ICs, active electronic components, or passive electronic components.

SiP

System in a Package. Multiple dies, including SoCs and chiplets, are instantiated in a single chip package.

SoC

System on a Chip. Extends ICs from simple devices comprised entirely of general purpose I/O (GPIO), logic, memories, and may include third-party IP cores such as SerDes, DDR PHYs, and special I/O such as LVDS and PECL.

TSDB

Tessent Shell Database: The design repository that the Tessent toolset uses to store or transfer the DFT database.

Unified BSDL

Use unified BSDL for board or system testing. SiPs have two categories of I/O. The first category is I/O that connect to other dies, and the second category is I/O that connect to package pins. With unified BSDL, you can build a single package-level BSDL comprised of I/O that talk to package pins. A BYPASS register and 32-bit DEVICE_ID register from the primary TAP are also used for IEEE-1149.1 compliance.

Third-Party Information

Details on open source and third-party software that may be included with this product are available in the *<your_software_installation_location>/legal* directory.